

第4章: Next.js の基礎

0. 前提知識: URL・ルーティング・サーバーレンダリング

Next.js（読み: ネクストジェイエス）を理解するには「URL（ユーアールエル：Web上の住所）」「ルーティング（routing：URLとページを結びつける仕組み）」「レンダリング（rendering：HTMLを組み立てて画面に出す処理）」の3つの基本概念を押さえる必要があります。ここから始めましょう。

この章で前提となる用語の超ざっくり予習: - **サーバー**：インターネットの向こうにあるコンピューター。リクエストを受けて返事をする。 - **クライアント／ブラウザ**：ユーザーの手元のコンピューターで動くChromeやSafariなどのアプリ。 - **HTML**：画面に出る文字や画像のレイアウト情報を書いた文書。 - **JavaScript**：ブラウザの中で動くプログラム。React や Next.js のコードもこれにコンパイルされる。

0.1 URL の構造

`https://example.com:443/products/123?sort=price&page=2#reviews`

https	プロトコル（通信方式）。 https は暗号化された通信、 http は暗号化されていない通信。今は基本 https を使う。
example.com	ホスト名（ドメイン）。サーバーの住所。 example.com のような人間に読める形と、実際の IP アドレスが DNS で結ばれている。
:443	ポート番号（省略時は https なら 443、http なら 80）。普段は省略されているので意識しないことが多い。
/products/123	パス（path: ページの場所）。Next.js ではこの部分が app/ フォルダの構造とそのまま対応する。
? sort=price&page=2	クエリパラメータ（? 以降）。 sort=price&page=2 のように「キー=値」を & で繋いで送る追加情報。
#reviews	フラグメント（ページ内位置）。 #reviews のように # の後に書く部分。ページ内の ID 指定に使う。

Next.js で書く各ページは、このパス（**/products/123** の部分）に対応します。つまり「URL のパスをどう設計するか＝どんなフォルダ構造にするか」という関係が成り立ちます。

0.2 ルーティングって何？

「URL を受け取って、どのページを表示するか決める仕組み」が **ルーティング**（routing：道案内のこと。URLに対応するページを「探して呼び出す」処理）です。Next.js の **App Router**（アップルーター：Next.js 13.4以降で正式採用された新しいルーティング方式。 **app/** フォルダ配下の構造で URL を決める）では、 **app/** フォルダの中のフォルダ構成がそのまま URL になります。これを **ファイルベースルーティング**（File-based Routing：ファイルやフォルダの配置自体がそのままルート定義になる方式）と呼びます。

app/	この app/ フォルダが Next.js の出発点			
└─ page.tsx	→ /	トップページ。 page.tsx があるフォルダ = その URL でアクセスできる		
└─ about/		フォルダ名が URL のセグメント（区切り）になる		
└─ page.tsx	→ /about	app/about/page.tsx は /about のページ本体		
└─ books/		/books 以下のグループ		
└─ page.tsx	→ /books	/books の一覧ページ		
└─ [id]/		角括弧 [id] は「動的セグメント」: 数字でも文字でも何でも受ける		
└─ page.tsx	→ /books/123	例えば /books/42 でアクセスすると id="42" として受け取れる ([id] は動的な値)		

ファイル＝ページ: 「 **app/about/page.tsx** を作っただけで **/about** というURLでそのページが見られるようになる」というのが Next.js のキモです。第3章までの React だけでは、 **react-router-dom** などのライブラリを別途インストールし、自分で `<Route path="/about" element={...} />` のようにルーティング設定を書く必要がありました。Next.js ではその設定作業が「ファイルを置く」という直感的な操作に置き換わります。

0.3 レンダリング3兄弟（CSR / SSR / SSG）

Webページを「画面に出るHTMLにする」処理を **レンダリング**（rendering：データやテンプレートから最終的な見た目のHTMLを組み立てること）と呼びます。これがいつ・どこで行われるかで3種類あります。

種類	フルネーム	いつHTMLができる？	どこで？	メリット
CSR	Client Side Rendering（クライアントサイドレンダリング：ブラウザ側でJSを動かしてHTMLを組み立てる方式）	アクセス時	ブラウザ	動的、軽量サーバー
SSR	Server Side Rendering（サーバーサイドレンダリング：サーバー側で完成HTMLを作って送る方式）	アクセス時	サーバー	SEO良、初回表示速い
SSG	Static Site Generation（スタティックサイトジェネレーション：ビルド時にHTMLを事前生成しておく方式）	ビルド時に1回	サーバー	最速、安価

このほか **ISR**（Incremental Static Regeneration：インクリメンタル・スタティック・リジェネレーション。SSGをベースに「指定秒ごとに再生成」する仕組み）、**RSC**（React Server Components：リアクト・サーバー・コンポーネンツ。サーバーでだけ動く新しい種類のReactコンポーネント）、**ストリーミング**（Streaming：完成した部分から順次ブラウザに送る方式）、**ハイドレーション**（Hydration：サーバーで出来上がったHTMLにブラウザ側のJSが「水を与える」ように後付けで動きを付ける処理）という言葉もよく出てくるので、本章で順番に扱います。

本書での使い分け: Next.js の App Router では、`page.tsx` をシンプルに書くとサーバーで実行される（SSR/SSG相当）のがデフォルトです。インタラクティブな部品（ボタンや入力欄など）だけ `"use client"` を付けて Client Component（クライアントコンポーネント：ブラウザ側で動くコンポーネント）にし、CSR的な動きを実現します。

はじめに

この章では、React ベースのフルスタックフレームワーク（Full-stack Framework：フロントエンド=画面もバックエンド=サーバー処理も両方カバーするフレームワーク）である **Next.js**（ネクストジェイエス）の基礎を学びます。

Next.js を使うことで、React 単体では実現が難しい**サーバーサイドレンダリング**（SSR：Server Side Rendering = サーバー側でHTMLを生成してからブラウザに送る方式。ページの初回表示が速くなる）、**静的サイト生成**（SSG：Static Site Generation = ビルド時にHTMLを事前生成する方式。最も高速）、**ファイルベースルーティング**（ファイルを配置するだけでURLとページが自動的に対応する仕組み）などを簡単に実装できます。

なぜ React だけではなく Next.js を使うのか

React は「UIの部品を作るライブラリ」であり、それ以外の機能（ページのURL管理、サーバーとの通信、SEO対策など）は自分で用意する必要があります。Next.js はこれらを**最初から備えている**ため、開発者はアプリのロジックに集中できます。

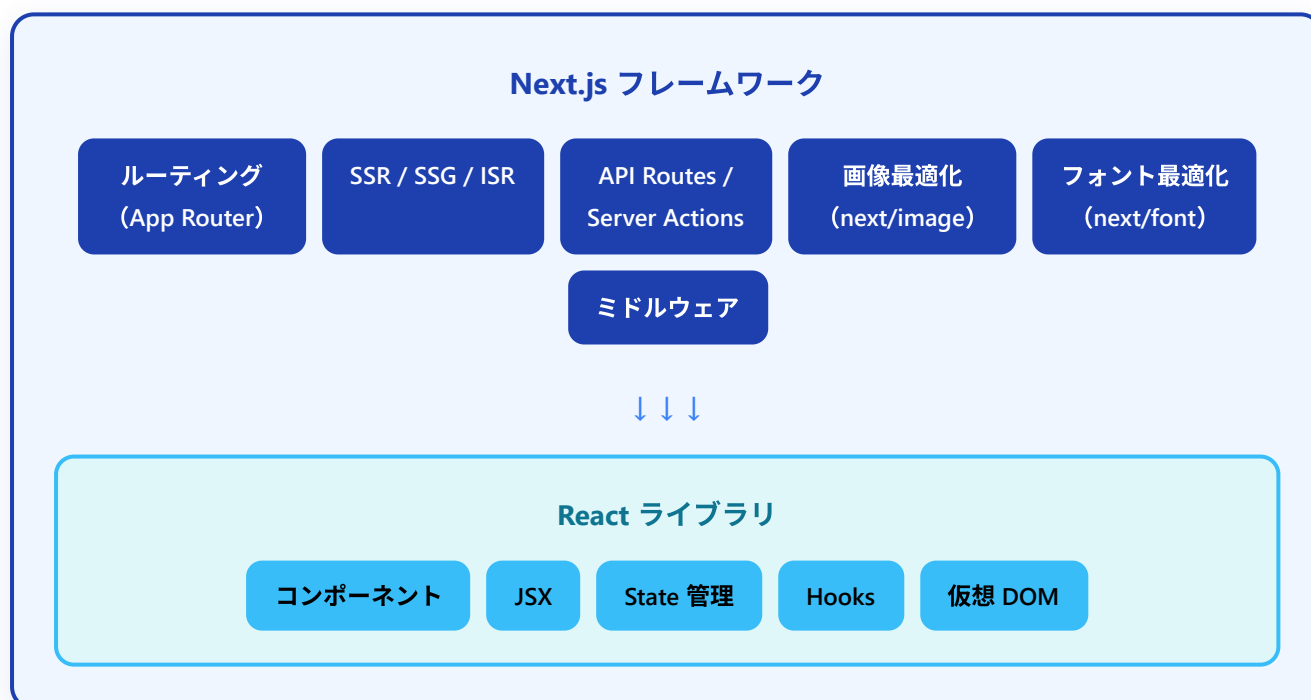
機能	React のみ	Next.js
ページのURL管理（ルーティング）	別途ライブラリ（react-router等）が必要	ファイルを置くだけで自動対応
サーバーでのデータ取得	自分でAPIサーバーを構築	Server Components で直接取得
SEO（検索エンジン対策）	追加設定が必要	標準で対応
ページ読み込み速度	初回が遅い（CSR）	高速（SSR/SSG）
デプロイ（公開）	別途設定が必要	Vercel で数クリック

この章で学ぶこと: - Next.js の概念と React との関係 - **App Router**（アップルーター：Next.js 13以降の新しいルーティング方式）によるルーティング - **Server Components**（サーバー上で実行されるコンポーネント）と **Client Components**（ブラウザ上で実行されるコンポーネント）の使い分け - レイアウト、ナビゲーション、**データフェッチ**（Data Fetch：サーバーやAPIからデータを取得すること）の基本 - **Server Actions**（サーバーアクション：フォーム送信などのサーバー処理をシンプルに書ける仕組み）によるサーバー処理 - プロジェクト構成の**ベストプラクティス**（Best Practice：最も良いとされるやり方・慣習）

1. Next.js とは

1.1 React との関係

Next.js は **Vercel 社** が開発・メンテナンスしている React ベースのフレームワークです。React はあくまで「UIライブラリ」であり、ルーティングやサーバーサイド処理などは含まれていません。Next.js はその React の上に、Web アプリケーション開発に必要な機能を包括的に提供します。



たとえ話で理解する:

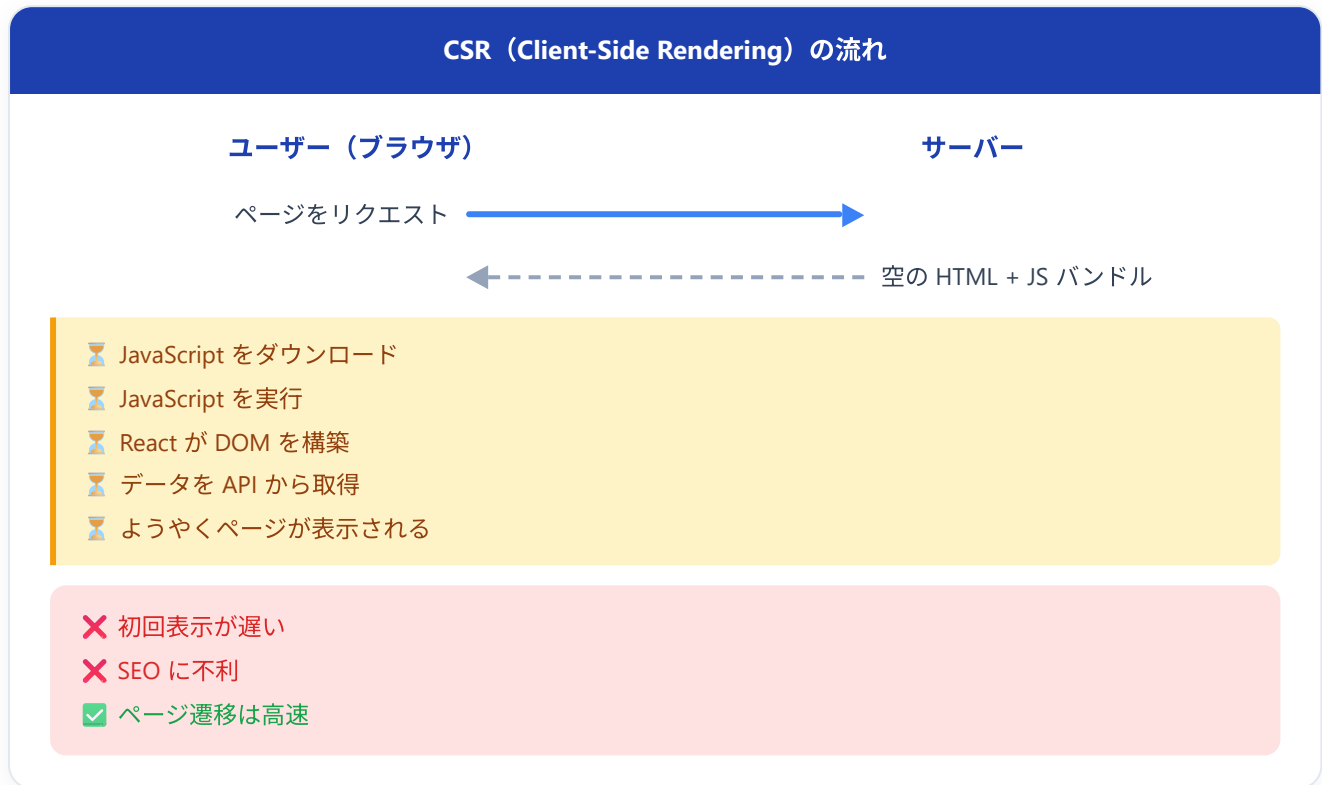
- **React** = エンジン（車を動かす核心部分）
- **Next.js** = 完成した車（エンジンに加え、ハンドル、ブレーキ、ナビなど全部入り）

React 単体でアプリを作ることできますが、ルーティングやデータ取得の仕組みを自分で選定・構築する必要があります。Next.js を使えば、それらが最初から統合されています。

1.2 SSR, SSG, CSR の違い

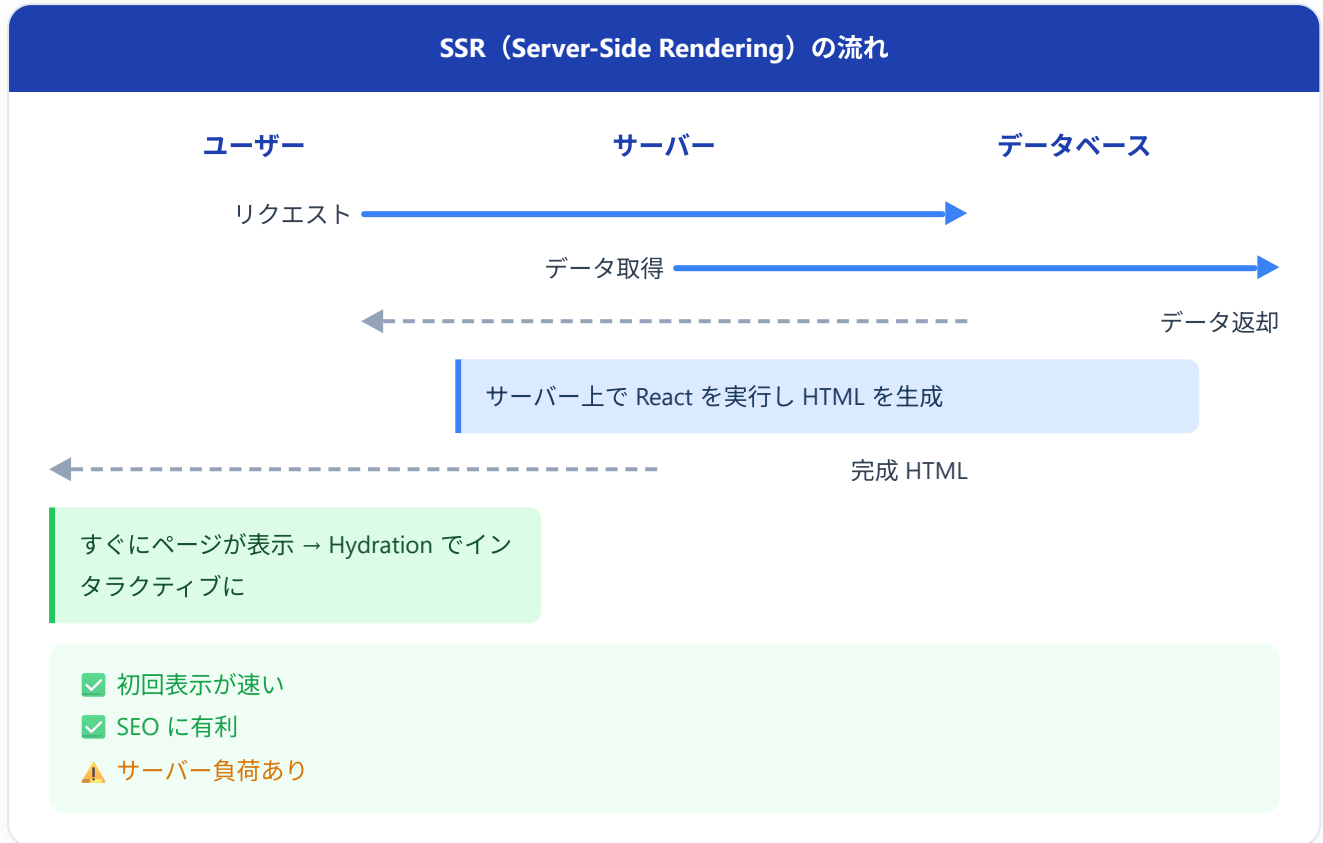
Web ページのレンダリング方法は大きく3つあります。Next.js はこれらすべてに対応しています。

CSR (Client-Side Rendering) - クライアントサイドレンダリング



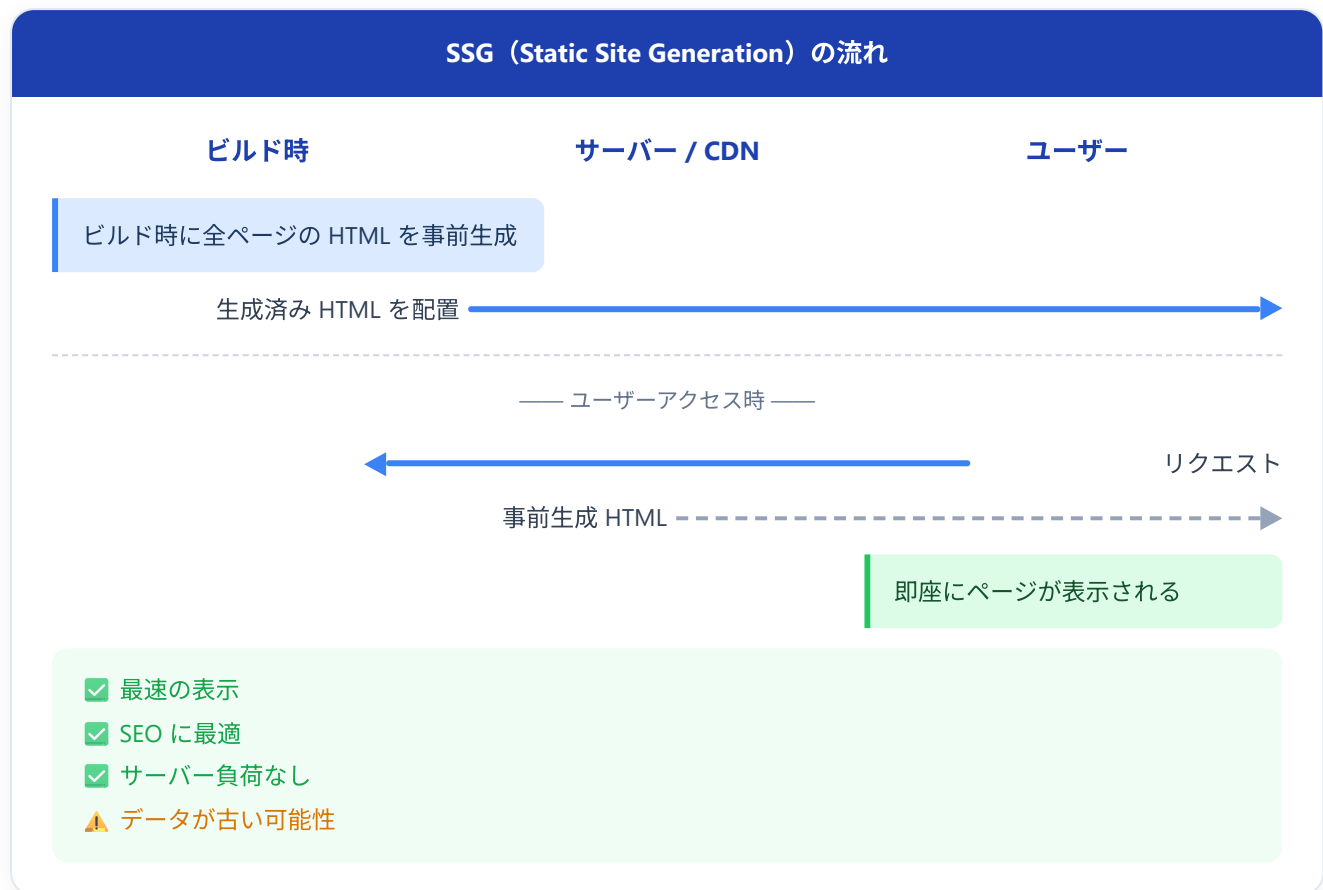
画面にはこう表示される: 最初に一瞬だけ白い画面やローディングスピナーが表示され、JavaScript の処理が完了してからコンテンツが表示されます。素の React (Create React App) はデフォルトでこの方式です。

SSR (Server-Side Rendering) - サーバサイドレンダリング



画面にはこう表示される: リクエストのたびにサーバーで HTML が生成されるため、最新のデータが反映された完成形のページがすぐに表示されます。ブラウザが JavaScript を読み込むと、ボタンクリックなどのインタラクションが可能になります (この過程を **Hydration** と呼びます)。

SSG（Static Site Generation） - 静的サイト生成



画面にはこう表示される: ビルド時に生成済みの HTML がそのまま返されるため、表示速度は最速です。ただし、ビルド後にデータが変わっても、再ビルドするまで古いデータが表示され続けます。ブログや企業サイトなど、更新頻度の低いページに最適です。

3つのレンダリング方式の比較

特性	CSR	SSR	SSG
初回表示速度	遅い	速い	最速
SEO	不利	有利	最も有利
データの鮮度	常に最新	常に最新	ビルド時点
サーバー負荷	低い	高い	最も低い
使用例	ダッシュボード	EC サイト商品ページ	ブログ記事

1.3 なぜ Next.js を使うのか（素の React との比較）

機能	素の React	Next.js
ルーティング	react-router 等を別途インストール	App Router が組み込み
SSR / SSG	自分で構築が必要（非常に複雑）	設定なしで利用可能
コード分割	手動で React.lazy 等を使う	自動で最適化
画像最適化	自分で実装	<code>next/image</code> が自動最適化
フォント最適化	自分で実装	<code>next/font</code> が自動最適化
API エンドポイント	Express 等の別サーバーが必要	Route Handlers / Server Actions
TypeScript	設定が必要	初期設定済み
ESLint	設定が必要	初期設定済み
環境変数	dotenv 等を別途設定	<code>.env.local</code> が組み込み
デプロイ	ホスティング先の選定・設定が必要	Vercel にワンクリックデプロイ

結論: 2024年以降、新規の React プロジェクトを始める場合は Next.js（App Router）を使うのがスタンダードです。React の公式ドキュメントでも Next.js が推奨フレームワークの1つとして紹介されています。

2. App Router

2.1 App Router vs Pages Router

Next.js には2つのルーティングシステムがあります。

特性	Pages Router (従来)	App Router (推奨)
ディレクトリ	<code>pages/</code>	<code>app/</code>
導入バージョン	Next.js 初期～	Next.js 13.4～ (安定版)
デフォルトコンポーネント	Client Component	Server Component
レイアウト	<code>_app.tsx</code> , <code>_document.tsx</code>	<code>layout.tsx</code> (ネスト可能)
データ取得	<code>getServerSideProps</code> , <code>getStaticProps</code>	async/await で直接
ローディング UI	自分で実装	<code>loading.tsx</code>
エラー UI	<code>_error.tsx</code> (グローバル)	<code>error.tsx</code> (ルートごと)
Server Actions	非対応	対応
Streaming	非対応	対応

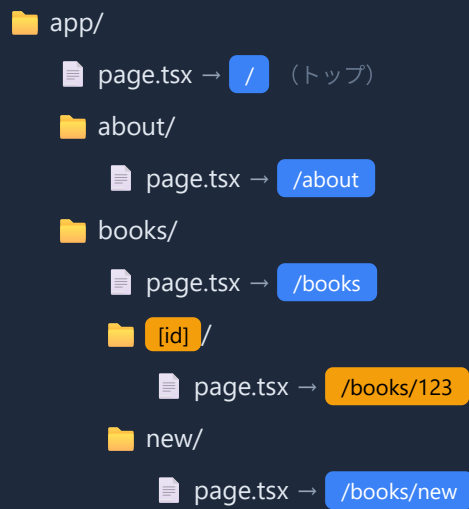
本チュートリアルでは **App Router** のみを使用します。Pages Router (ページズルーター: `pages/` フォルダを使う Next.js の初期からあるルーティング方式) は旧バージョンとの互換性のために残されていますが、新規プロジェクトでは App Router を選択してください。Pages Router では「コンポーネントは原則 Client、データ取得は `getServerSideProps` などの専用関数で外側から差し込む」という設計でしたが、App Router では「コンポーネント自体を async にして直接 await する」という根本的に新しい設計になっています。

なぜデフォルトが Server Component なのか: App Router は React Server Components (RSC: サーバーで実行されブラウザに HTML だけが送られる新種のコンポーネント) を基盤にしています。ブラウザに送る JS の量を減らし、機密情報の漏えいを防ぎ、初回表示を速くするために「まずは全部サーバー側で動かす」のがデフォルトなのです。クライアントでの動きが必要な箇所だけを `"use client"` で明示的に切り出していきます。

2.2 ファイルベースルーティングの仕組み

Next.js の App Router では、`app/` ディレクトリ内のフォルダ構造がそのまま URL のパスに対応します。これが **ファイルベースルーティング** です。

ファイル構造 → URL パス



重要なルール: - フォルダ がURLのセグメント（パスの一部）になる - `page.tsx` があるフォルダだけがアクセス可能なルートになる - `page.tsx` がないフォルダは URL として公開されない（コンポーネントの整理用に使える）

2.3 page.tsx, layout.tsx, loading.tsx, error.tsx の役割

App Router では、特別な名前を持つファイルにそれぞれ役割があります。

app/books/ ディレクトリ

layout.tsx (共通レイアウト)

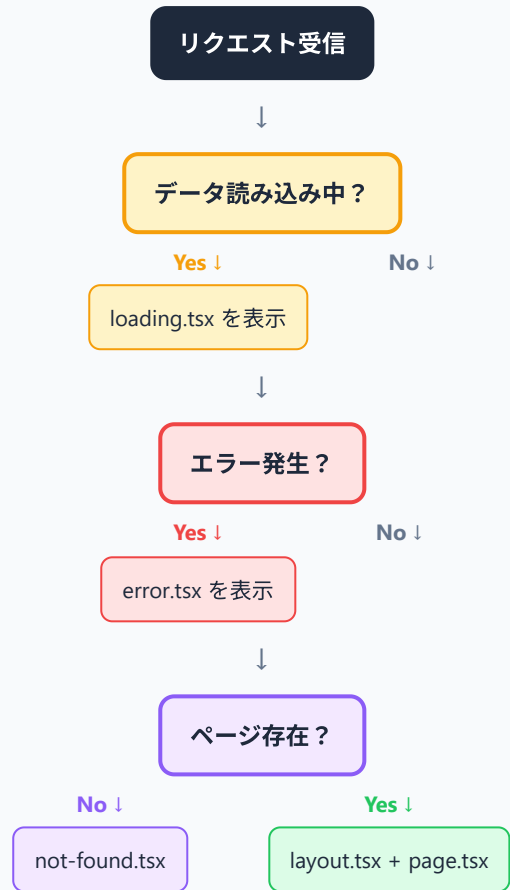
page.tsx (ページ本体)

loading.tsx (読み込み中の表示)

error.tsx (エラー時の表示)

not-found.tsx (404の表示)

レンダリングの流れ



page.tsx - ページ本体

page.tsx (ページ・ティーエスエックス) は Next.js が「これがこのフォルダのページ本体」と認識する予約名のファイルです。そのルート (URL) にアクセスしたときに表示されるメインコンテンツを書きます。**page.tsx** がないフォルダは URL として公開されない、というルールも合わせて覚えておきましょう。

▼このコードがやること (先に日本語で) : **app/page.tsx** を作って「トップページ (URL は /) の中身」を定義します。コンポーネント (画面の部品) は「JSX を返す関数」で、**export default** を付けて「このファイルの主演」として外に公開するのがポイントです。見出しと段落を表示するだけのシンプルなページです。詳しい1行ずつの意味はコード内コメントを見てください。

```
// ファイルパス。`app/` 直下なので URL は `/` (ルート) になる
// app/page.tsx
// ブラウザでアクセスする URL の説明コメント
// URL: /

// `export default` でこのコンポーネントを「このファイルの主演」として外に公開
export default function HomePage() {
    // 関数名 `HomePage` は何でもよい (慣習的にページ名を付ける)。Next.js は名前ではなく default export を見ている
    // JSX (HTMLに似たReactの記法) を返す
    return (
        // `

` : レイアウト用の汎用ブロック要素。中身を1つにまとめるために使う
        <div>
            // `

# ` : ページの一番大きな見出し。1ページに1つだけ置くのが推奨 <h1>書籍管理アプリへようこそ</h1> // ` ` : 段落 (paragraph)。普通の文章ブロック <p>あなたの読書記録を管理しましょう。</p> // 閉じタグ。開いた ` ` と必ず対応させる </div> // return の終わり ); // 関数の終わり }


```

画面にはこう表示される: ブラウザで <http://localhost:3000/> にアクセスすると、「書籍管理アプリへようこそ」という見出しと「あなたの読書記録を管理しましょう。」という段落が表示されます。

補足: なぜデフォルトが Server Component なのか? App Router では `page.tsx` の中身はサーバーで実行され、結果のHTMLだけがブラウザに送られます。これは「JavaScript バンドル (ブラウザに送られるJSの塊) を小さく保つ」「DB接続情報など秘密の値をブラウザに漏らさない」「初回表示を速くする」という3つの利点があるからです。ボタンクリックなどユーザー操作が必要になったときだけ `"use client"` で Client Component に切り替えます。

layout.tsx - 共通レイアウト

複数のページで共有される UI (ヘッダー、サイドバーなど) を定義します。ページ遷移してもレイアウトは再レンダリングされず、状態が保持されます。

▼ このコードがやること (先に日本語で) : すべてのページを「共通ヘッダー + 本文 + フッター」という枠で包む、アプリの一番外側の土台を作ります。 `children` (チルドレン) という名前の引数に「各ページの中身」が自動で差し込まれる仕組みがキモで、ページを移動してもこの枠は作り直されません。 `<html>` と `<body>` タグはこのルートレイアウトにだけ書く、という決まりも押さえてください。

```
// =====
// ファイルパス: app/layout.tsx
// 役割      : すべてのページを「ヘッダー+本文+フッター」の枠で包む
//            = Next.jsアプリの **一番外側の共通レイアウト**
// -----
// このファイルは Next.js が自動的に認識する「予約名」のひとつ。
// app/ フォルダ直下に layout.tsx が必ず1つ必要。
// =====

// `Metadata` 型を import する (Next.js 標準の型)。
// `import type` は「型だけ取り込む」記法。実行時のJSには残らないので軽量。
import type { Metadata } from "next";

// -----
// (1) メタデータの定義
// -----
// metadata という名前で export すると、Next.js が <head> 内の <title> や
// <meta name="description" ...> を自動生成してくれる。
// 自分で <head> を書く必要がない。
export const metadata: Metadata = {
  // ブラウザタブの文字
  title: "書籍管理アプリ",
  // SEO・SNS共有時の説明文
  description: "あなたの読書記録を管理するアプリケーション",
};

// -----
// (2) ルートレイアウトのコンポーネント本体
// -----
// `RootLayout` は予約名ではないが、慣習的にこの名前を付ける。
//
// 引数 (Props) :
//   - children: このレイアウトの中に差し込まれる「各ページの中身」
//   React.ReactNode 型は「JSX/文字列/数値/null/...などReactで扱える何でも」を表す。
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    // — 必須: <html> と <body> はルートレイアウトでだけ書く —
    // lang="ja" は「このページは日本語ですよ」という指定。
    // スクリーンリーダーや翻訳ツールがこれを参照する。
    <html lang="ja">
      <body>

        { /* — 共通ヘッダー (全ページに表示される) — */ }
        <header>

          <nav>

```

```

    <h1><img alt="book icon" data-bbox="248 65 268 75"/> 書籍管理アプリ</h1>
  </nav>
</header>

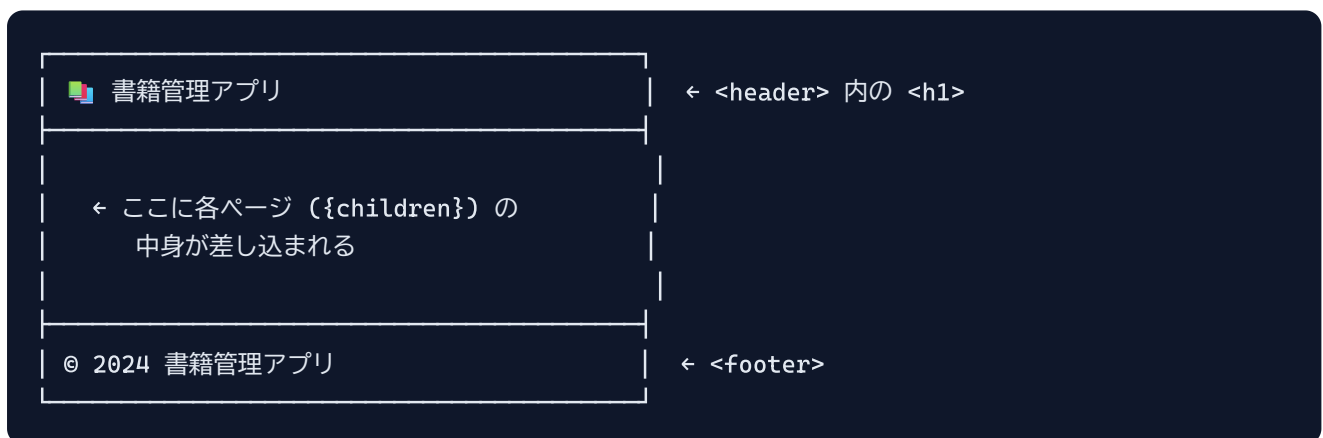
  {/*
    — ページ本体が差し込まれる場所 —
    各 page.tsx の return 値が、ここに自動で入る。
    /books にアクセスしたら books/page.tsx の内容が、
    /books/123 にアクセスしたら books/[id]/page.tsx の内容がここに入る。
  */}
  <main>{children}</main>

  {/* — 共通フッター（全ページに表示される） — */}
  <footer>
    <p>&copy; 2024 書籍管理アプリ</p>
  </footer>

</body>
</html>
);
}

```

▼ ブラウザでの見た目（どのページでも上下にこの枠が出る）：



▼ 動作の仕組み:

URL	layout.tsx	中央 (<code>{children}</code>)
<code>/</code>	表示される	<code>app/page.tsx</code> の中身
<code>/books</code>	表示される	<code>app/books/page.tsx</code> の中身
<code>/books/123</code>	表示される	<code>app/books/[id]/page.tsx</code> の中身

ページ遷移してもヘッダーとフッターは再描画されない（=スクロール位置や状態が保たれる）のが Next.js のレイアウト機能のメリットです。

▼ コードを1つずつ分解して解説

上のレイアウトには「メタデータの export」「children を受け取る型」「`<html>` / `<body>` の特別扱い」という、初心者がつまづきやすいポイントが入っています。順番に見ていきましょう。

解説1: `metadata` を export するだけで `<head>` が自動生成される

```
export const metadata: Metadata = {
  title: "書籍管理アプリ",
  description: "あなたの読書記録を管理するアプリケーション",
};
```

- `metadata`（メタデータ）は「ページに関する情報」のことです。ブラウザのタブに出るタイトルや、検索エンジン・SNS共有で使われる説明文などがこれにあたります。
- ポイントは「`metadata` という決まった名前で `export` するだけ」という点です。これだけで Next.js が自動的に `<head>` の中の `<title>` や `<meta name="description">` を作ってくれます。自分で `<head>` を書く必要がありません。
- `: Metadata` の部分は「このオブジェクトは `Metadata` という型ですよ」とTypeScriptに教える型注釈です（`Metadata` 型はファイル冒頭で `import type { Metadata } from "next"` して取り込んでいます）。

用語: メタデータ（metadata） ... 「データについてのデータ」という意味。ページ本体の中身ではなく「このページは何か」を説明する付帯情報（タイトル・説明文など）を指します。

解説2: `children` を受け取って「各ページの中身」を差し込む

```
export default function RootLayout({
  children,
}): {
  children: React.ReactNode;
} {
```

- `RootLayout` は、すべてのページを包む一番外側の枠（レイアウト）です。`export default` を付けて「このファイルの主演」として公開しています。
- 引数の `{ children }` は分割代入で、props（親から渡される値）の中から `children` だけを取り出しています。`children`（チルドレン＝子）には「このレイアウトの内側に入る、各ページの中身」が自動で入ってきます。
- `: { children: React.ReactNode }` は引数の型注釈です。`React.ReactNode`（リアクト・リアクトノード）は「JSX・文字列・数値・`null` など、Reactが画面に描画できるものなら何でも」を表す便利な型です。

用語: children (チルドレン) ... React で「あるコンポーネントのタグの内側に書かれた中身」を指す特別な props 名。Next.js のレイアウトでは「各ページの内容」がここに渡されます。

解説3: `<html>` と `<body>` はルートレイアウトにだけ書く

```
<html lang="ja">
  <body>

    <header>
      <nav>
        <h1>📖 書籍管理アプリ</h1>
      </nav>
    </header>

    <main>{children}</main>

    <footer>
      <p>© 2024 書籍管理アプリ</p>
    </footer>

  </body>
</html>
```

- 普通の React コンポーネントでは `<html>` や `<body>` を書きませんが、Next.js のルートレイアウトだけは例外で、ここに `<html>` と `<body>` を書きます（ページ全体の最も外側の枠だからです）。
- `lang="ja"` は「このページは日本語ですよ」という指定で、翻訳ツールやスクリーンリーダー（読み上げソフト）がこれを参照します。
- `<header>`（ヘッダー）と `<footer>`（フッター）は全ページ共通で表示される枠です。その間の `<main>{children}</main>` の `{children}` の位置に、各ページの中身がはめ込まれます。`/books` を開けば books のページが、`/` を開けばトップページの中身が、この同じ場所に入れ替わりで表示されます。

用語: `<header>` / `<main>` / `<footer>` ... HTMLの「セマンティック要素」（意味を持つタグ）。それぞれ「上部の見出し領域」「主要な本文領域」「下部の補足領域」を表し、検索エンジンや支援技術がページ構造を理解する助けになります。

loading.tsx - 読み込み中の表示

`loading.tsx` は Next.js が「データ取得中に表示するUI」と認識する予約名のファイルです。データの読み込み中に自動的に表示される UI を書きます。仕組みとしては内部で React の `<Suspense>`（サスペンス：データ待ちの間に代

わりのUIを出す機能) を使っており、対応する `page.tsx` の Server Component が `await` でデータを待っている間、この `loading.tsx` の中身が代わりに描画されます。

▼ このコードがやること (先に日本語で) : データ取得中に「読み込み中」を伝える画面 (ローディングUI) を作ります。 `loading.tsx` という予約名のファイルを置くだけで、Next.js が自動で「ページの準備ができるまでの間」にこれを表示してくれます。回転するスピナーと案内メッセージを出すだけのシンプルな部品です。詳細はコード内コメントを参照してください。

```
// ファイルパス。`/books` のロード中UIになる
// app/books/loading.tsx

// フォルダごとに置けるので、ページ単位で異なるローディング表現が可能

// default export された関数がそのまま「ローディングUI」として使われる
export default function BooksLoading() {
  // 関数名は何でもよいが、慣習的に`xxxLoading` と付ける
  // 返す JSX
  return (
    // `className` は HTML の `class` 属性のJSX版 (`class` はJSの予約語なので別名)
    <div className="loading-container">
      // CSSアニメーションで作る回転するスピナー (自己終了タグ `</>` で書ける)
      <div className="spinner" />
      // ユーザーへの説明文
      <p>書籍データを読み込んでいます...</p>
      // ラッパーdivの閉じタグ
    </div>
  );
  // 関数の終わり
}
```

画面にはこう表示される: `/books` にアクセスした直後、データベースからデータを取得している間、スピナーアニメーションと「書籍データを読み込んでいます...」というメッセージが表示されます。データの取得が完了すると、自動的に `page.tsx` の内容に切り替わります。これは内部的に「サーバーがHTMLを少しずつ送る = ストリーミング」と呼ばれる挙動で実現されています。

error.tsx - エラー時の表示

`error.tsx` は Next.js が「そのフォルダ内でエラーが起きたときに表示するUI」と認識する予約名のファイルです。ページの描画中に例外が `throw` された場合などに表示される UI を書きます。 `"use client"` が必須です (React の Error Boundary (エラーバウンダリ: エラーをキャッチして代替UIを出す仕組み) はクラスコンポーネントを内部で使うため、Client Component でないと動かないため) 。

▼このコードがやること（先に日本語で）：ページ表示中にエラー（不具合）が起きたとき、白い画面で固まる代わりに「エラーが発生しました」という案内と「もう一度試す」ボタンを出す画面を作ります。`error.tsx` には必ず先頭に `"use client"` を書く必要があり、Next.js が `error`（起きたエラー情報）と `reset`（再挑戦用の関数）を自動で渡してくれます。ボタンを押すと `reset()` が呼ばれてその部分の再表示を試みます。詳細はコード内コメントを参照してください。

```
// `./books` 配下でエラーが起きたときに自動で表示される
// app/books/error.tsx
// この1行でこのファイルを「ブラウザ側で動くコンポーネント」に切り替える
"use client";

// error.tsx ではこれを必ず先頭に書く（ファイルの一番上、importより前）

// default export 関数。Next.js が自動で呼び出す
export default function BooksError({
  // props.error: 発生した Error オブジェクト
  error,
  // props.reset: 「もう一度試す」用に Next.js が渡してくれる関数
  reset,
// ↓ ここから TypeScript の型注釈
}): {
  // 標準 Error 型に、Next.js が独自に付ける `digest`（任意）を合成した型
  error: Error & { digest?: string };
  // digest はサーバーログとの紐付け用ID（本番環境で詳細メッセージを隠したいとき使う）
  reset: () => void;
} {
  return (
    // エラー表示の枠
    <div className="error-container">
      // タイトル
      <h2>エラーが発生しました</h2>
      // `error.message` : 例外が持つメッセージ文字列を表示
      <p>{error.message}</p>
      // `{}` 内はJSの式。文字列補間に使う
      <button onClick={() => reset()}>もう一度試す</button>
      // ボタン押下で `reset()` を呼ぶ → Next.js が該当セグメントを再描画してくれる
      // アロー関数で包んでいるのは「呼び出し時の引数 (event) を渡さないため」
    </div>
  );
}
```

画面にはこう表示される: 書籍データの取得中にエラーが発生すると、「エラーが発生しました」という見出しとエラーメッセージ、そして「もう一度試す」ボタンが表示されます。ボタンを押すと、そのセグメントの再レンダリングが試みられます。

補足: `not-found.tsx` もある: `notFound()` 関数（後述）が呼ばれたときや404相当の状態のときに表示するUIとして `not-found.tsx` という予約ファイルも置けます。`error.tsx` と兄弟関係にあります。

2.4 動的ルーティング ([id])

URL の一部をパラメータ（parameter：動的に変わる値）として受け取りたい場合、フォルダ名を角括弧 `[...]` で囲みます。これを **動的セグメント**（Dynamic Segment：URLの中で変化する部分）と呼びます。

```
app/                                # アプリのルートフォルダ
  books/                            # `/books` 以下のグループ
    [id]/                          # 角括弧で囲むと「ここは可変」を意味する。フォルダ名の `id`
    が変数名になる
      page.tsx    → /books/1, /books/2, /books/abc などにマッチ  # 数字でも文字列でも何でも
    受ける
```

▼ このコードがやること（先に日本語で）：URL の一部（例: `/books/42` の「42」）をプログラムの中で受け取って画面に表示します。フォルダ名を `[id]` のように角括弧で囲むと、その部分が変化する値（パラメータ）になり、`params` という引数で受け取れます。Next.js 15 以降では `params` は `await` で取り出す必要がある点に注意してください（受け取った値は数値ではなく文字列で来ます）。詳細はコード内コメントを参照してください。

```

// ファイルパス。フォルダ名 [id] が動的セグメント
// app/books/[id]/page.tsx

// params はオブジェクトとして渡される
// Next.js 15 では params は Promise として渡されるため await が必要
// `async` 関数にすると本体で `await` が使える
export default async function BookDetailPage({
  // この `async function` をコンポー
  // ーネントに使えるのは Server Component の特権
  // props.params: 動的セグメントの値が入ったオブジェクト
  params,
  // ↓ TypeScript 型注釈
}) {
  // Next.js 15+ では params は Promise でラップされて渡される
  params: Promise<{ id: string }>;
  // `{ id: string }` は `id` という文
  // 字列プロパティを持つオブジェクト」型
  // `await` で Promise を解決し、分割代入で `id` だけ取り出す
  const { id } = await params;
  // 例: URLが `/books/42` なら id は
  // "42" (数値ではなく文字列!)

  return (
    // ページ全体のラッパー
    <div>
      // 見出し
      <h1>書籍詳細</h1>
      // 取り出した id を JSX に埋め込んで表示
      <p>書籍 ID: {id}</p>
      /* 実際にはここで id を使ってデータベースから書籍情報を取得する */
      // JSX 中のコメントは `{/* ...`
    </div>
  );
}

```

画面にはこう表示される: - `/books/1` にアクセスすると「書籍 ID: 1」と表示されます。 - `/books/42` にアクセスすると「書籍 ID: 42」と表示されます。 - `/books/abc` にアクセスすると「書籍 ID: abc」と表示されます。

URL のその部分がそのまま `id` パラメータとして使えるわけです。

▼ コードを1つずつ分解して解説

この動的ルートのコードには「`async` コンポーネント」「`params` が Promise である」「`await` で取り出す」という3つの新しい考え方が詰まっています。1つずつ見ていきましょう。

解説1: コンポーネント関数に `async` を付けられるのは Server Component の特権

```
export default async function BookDetailPage({
  params,
}): {
  params: Promise<{ id: string }>;
} {
```

- 関数名の前に付いている `async` (エイシク=非同期) は、「この関数の中で `await` (待つ) が使えるようにする」キーワードです。
- 普通の React (Client Component) では、コンポーネント関数を `async` にできません。関数を `async` にできるのは Server Component だけの特権です (このファイルは `"use client"` を書いていないので Server Component です)。
- `params` (パラムス=パラメータの複数形) は、URLの動的セグメント (`[id]` の部分) の値が入った箱です。引数の分割代入で受け取っています。

用語: `async` / `await` ... 「時間がかかる処理 (データ取得など) の結果を待つ」ための仕組み。`async` を付けた関数の中でだけ `await` が使え、`await ○○` は「○○の結果が出るまでここで待つ」という意味になります。

解説2: Next.js 15 では `params` は「Promise」で渡される

```
const { id } = await params;
```

- `params: Promise<{ id: string }>` の `Promise` (プロミス=約束) は「今すぐではなく、少し後に値が届く」ことを表す型です。Next.js 15 以降では `params` がこの Promise に包まれて渡されます。
- そのため、中身を取り出すには `await params` のように `await` で「値が届くのを待つ」必要があります。届いたオブジェクト (`{ id: "42" }` のような形) から、分割代入で `id` だけを取り出しています。
- 注意点:** 取り出した `id` は数値ではなく文字列です。URLが `/books/42` でも、`id` は数値の `42` ではなく文字列の `"42"` になります。数値として計算したいときは `Number(id)` で変換します。

用語: `Promise` (プロミス) ... 「将来この値を渡します」という約束を表すオブジェクト。`await` を付けると、その約束が果たされる (値が届く) まで待ってから次の行へ進みます。

複数の動的セグメント

```
app/  
  users/  
    [userId]/  
      books/  
        [bookId]/  
        page.tsx    → /users/5/books/12 にマッチ
```

▼ このコードがやること（先に日本語で）：URL に変化する部分が2つある場合（例: `/users/5/books/12`）に、その両方の値（`userId` と `bookId`）を同時に取り出して表示します。動的セグメントは2段以上ネストで、`params` の中に複数のキーとして入ってきます。`await` で取り出してから分割代入で2つの変数に展開するのがポイントです。詳細はコード内コメントを参照してください。

```
// =====
// ファイルパス: app/users/[userId]/books/[bookId]/page.tsx
// 役割      : URL 中の userId と bookId を取り出して表示するページ
// -----
// このファイルは Next.js が「動的セグメントを含むページ」と認識する。
// /users/5/books/12 にアクセスされると userId="5", bookId="12" になる。
// =====

// (1) 関数本体に async を付けると、内部で await が使える。
//     コンポーネント関数を async にできるのは Server Component だけの特権。
//
// (2) 引数の Props 型は { params: Promise<{ ... }> }
//     Next.js 15 以降では params が Promise でラップされて渡される。
//     なので await で「中身の値」を取り出してから使う。
export default async function UserBookPage({
  params,
}: {
  params: Promise<{ userId: string; bookId: string }>;
}) {
  // (3) Promise を await してオブジェクトを取り出し、
  //     さらに分割代入で userId と bookId に展開する。
  //     URL が /users/5/books/12 なら userId = "5", bookId = "12" となる。
  //     ※ 文字列で来ることに注意。数値が必要なら Number(userId) で変換する。
  const { userId, bookId } = await params;

  // (4) 取り出した値を JSX 内に埋め込んで表示
  return (
    <div>
      <p>ユーザー ID: {userId}</p>
      <p>書籍 ID: {bookId}</p>
    </div>
  );
}

// ▼ /users/5/books/12 にアクセスしたときの画面表示:
//   ユーザー ID: 5
//   書籍 ID: 12
```

キャッチオールセグメント

「何階層あっても全部受け取りたい」という場合は `[...slug]` のように `...`（スプレッド構文と同じ記号）を付けます。これを **Catch-all Segment**（キャッチオールセグメント：何階層でも一括で受け取る動的セグメント）と呼びます。

```

app/                # ルート
  docs/             # `/docs` 以下のグループ
    [...slug]/      # `...` 付きの動的セグメント。slug は配列
で渡る
    page.tsx        → /docs/a, /docs/a/b, /docs/a/b/c などにマッチ # 何階層でもOK

```

▼ このコードがやること（先に日本語で）： `/docs/a/b/c` のように何階層続くか分からない URL を、まとめて1つのページで受け取ります。フォルダ名を `[...slug]` のように `...` 付きにすると、それ以降のパスが「文字列の配列」として渡ってきます（例: `["react", "hooks", "useEffect"]`）。受け取った配列を `join(" / ")` でつなげて表示する例です。詳細はコード内コメントを参照してください。

```

// =====
// ファイルパス: app/docs/[...slug]/page.tsx
// 役割      : スラッシュ区切りで何階層でもマッチする「キャッチオール」ページ
// =====
// [...slug] のように ... を付けると、それ以降のすべてのパスが配列として渡る。
// /docs/react          → slug = ["react"]
// /docs/react/hooks    → slug = ["react", "hooks"]
// /docs/react/hooks/useEffect → slug = ["react", "hooks", "useEffect"]
// =====

export default async function DocsPage({
  params,
}: {
  // (1) slug は文字列の配列として渡る
  params: Promise<{ slug: string[] }>;
}) {
  // (2) Promise から slug 配列を取り出す
  const { slug } = await params;
  // 例: /docs/react/hooks/useEffect → slug = ["react", "hooks", "useEffect"]

  // (3) Array.prototype.join(" / ") で配列を文字列にする
  // ["react", "hooks", "useEffect"].join(" / ") → "react / hooks / useEffect"
  return (
    <div>
      <p>パス: {slug.join(" / ")}</p>
    </div>
  );
}

// ▼ /docs/react/hooks/useEffect にアクセスしたときの画面表示:
//   パス: react / hooks / useEffect

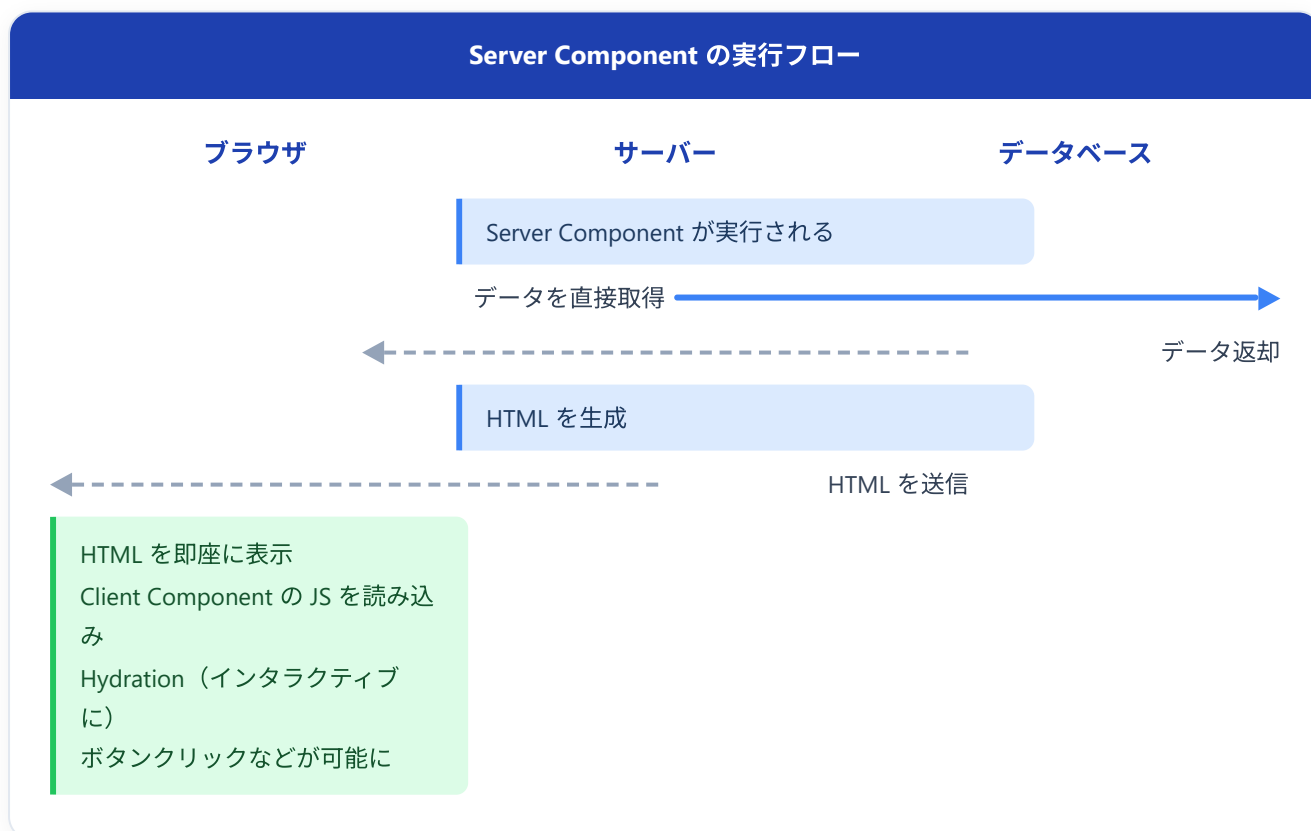
```


3. Server Components vs Client Components

3.1 概念の説明

Next.js App Router の最大の特徴は、デフォルトですべてのコンポーネントが **Server Component** であるということです。

Server Components (デフォルト)	Client Components ('use client')
✓ サーバー上で実行される ✓ データベースに直接アクセス可能 ✓ JS バンドルに含まれない ✗ <code>useState</code>, <code>useEffect</code> 使用不可 ✗ <code>onClick</code> 等のイベント処理不可 ✗ ブラウザ API 使用不可	🌐 ブラウザ上で実行される 🌐 API 経由でデータ取得 🌐 JS バンドルに含まれる ✓ <code>useState</code>, <code>useEffect</code> 使用可能 ✓ <code>onClick</code> 等のイベント処理可能 ✓ ブラウザ API 使用可能



3.2 "use client" ディレクティブ

Client Component にするには、ファイルの **先頭**（import 文よりも上）に **"use client"** と記述します。これを **ディレクティブ**（directive：コンパイラやランタイムに対する命令文）と呼びます。

'use client' の伝播ルール（重要）：- **"use client"** を書いたファイルは、その時点で「Client Component の境界（ボーダー）」になります。- その境界より下流（そのファイルが import する他のファイル）は、明示しなくても自動的に Client Component として扱われます。- つまり「Client から import される Server Component」は実質作れません。- 逆に「Server Component から import される Client Component」はOKです。これが普通の組み合わせ。- 1つのページ内で「Server の骨組みの中に Client 部品を埋め込む」という構造が App Router の基本パターンです。

▼このコードがやること（先に日本語で）：「**"use client"** を書かないと自動的に Server Component（サーバー側で動く部品）になる」というデフォルトの形を確認します。Server Component は関数を **async** にしてサーバー上で直接データを取りに行ける反面、**useState** やボタンのクリック処理などブラウザ側の機能は使えません。ここではデータ取得は省略し、文字を表示するだけの最小例です。詳細はコード内コメントを参照してください。

```
// =====
// Server Component の例 ("use client" を書かない=デフォルト)
// =====
// このファイルはサーバーでだけ実行される。
//   ✓ 関数を async にして await でDBやAPIにアクセスできる
//   ✓ 機密情報（APIキーなど）を扱える（ブラウザに送られない）
//   ✗ useState / useEffect / onClick などのインタラクティブ機能は使えない
//   ✗ window / document などブラウザ専用のAPIは使えない

export default function BookList() {
  // (1) 通常はここで await fetch(...) や DBクライアントを呼んでデータを取得する
  //   例: const books = await db.query("SELECT * FROM books");
  return <div>書籍一覧（サーバーで描画）</div>;
}
```

▼このコードがやること（先に日本語で）：ファイル先頭に **"use client"** を書いて、ブラウザ側で動く Client Component（クライアントコンポーネント）にする例です。これにより **useState**（状態を覚えておく仕組み）やユーザー入力への反応が使えるようになります。入力欄に文字を打つたびに状態が更新され、画面の表示もそれに追従する「検索バー」を作ります。詳細はコード内コメントを参照してください。

```
// =====
// Client Component の例 ("use client" をファイル先頭に書く)
// =====
// このディレクティブが書かれた瞬間から、このファイルとそこから import される
// すべてのコンポーネントはブラウザで動く扱いになる。
"use client";

// (1) Client Component ではブラウザ側で動く React のフックが使える
import { useState } from "react";

export default function SearchBar() {
  // (2) useState で「検索クエリ文字列」を状態として管理する
  //   [現在の値, 値を変える関数] = useState(初期値)
  //   初期値が "" なので最初の query は空文字。
  const [query, setQuery] = useState("");

  // (3) JSX で <input> を描画
  //   ・value={query}      : 入力欄に現在の状態を反映する (制御コンポーネント)
  //   ・onChange={e=>...}: ユーザーが文字を打つたびに呼ばれる関数
  //   ・e.target.value     : 入力欄の現在の値 (input要素のテキスト)
  //   ・setQuery(...)     : 状態を更新 → React が再描画 → value も更新される
  return (
    <input
      type="text"
      value={query}
      onChange={e => setQuery(e.target.value)}
      placeholder="書籍を検索..."
    />
  );
}
```

▼ 動作:

- 検索バー (テキスト入力欄) が表示される
- キーボードで「Re」と打つと query が "" → "R" → "Re" に更新され、画面の入力欄もそれに追従する
- これは **useState** がブラウザのメモリに値を保持し、変更ごとに React が再描画するため
- サーバーはこの入力プロセスにまったく関与しない

重要: **"use client"** はそのファイルとそこから import されるすべてのモジュールを Client Component の境界として宣言します。つまり、Client Component の子コンポーネントも自動的に Client Component になります。

▼ コードを1つずつ分解して解説

この Client Component には「**"use client"** の宣言」「**useState** での状態管理」「制御コンポーネント」という3つの大事な要素があります。順に見ていきましょう。

解説1: ファイル先頭の `"use client"` で「ブラウザ側で動く部品」にする

```
"use client";

import { useState } from "react";
```

- `"use client"`（ユーズ・クライアント）は、ファイルの一番上（import より前）に書く「おまじない（ディレクティブ）」です。これを書くと、このファイルは「ブラウザ側で動く Client Component」になります。
- App Router ではデフォルトが Server Component（サーバー側で動く）なので、`useState` やボタンのクリック処理など「ブラウザでの操作」を使いたいときは、この1行で明示的に切り替える必要があります。
- 切り替えた後は、`react` から `useState`（状態を覚えておく仕組み）などのフックを import して使えるようになります。

用語: ディレクティブ（directive） ... プログラムに対する「指示書き」。`"use client"` は「このファイルはクライアント側で動かして」という Next.js への指示です。

解説2: `useState` で「入力された文字」を覚えておく

```
const [query, setQuery] = useState("");
```

- `useState`（ユーズ・ステート）は「変化する値を覚えておく箱」を作るフックです。`const [今の値, 値を変える関数] = useState(初期値)` の形で使います。
- ここでは「今の値」を `query`（クエリ＝検索文字列）、「値を変える関数」を `setQuery` という名前にしています。初期値は `""`（空文字）なので、最初の `query` は空です。
- ユーザーが文字を打つたびに `setQuery(新しい文字)` を呼ぶと、`query` が更新され、Reactが画面を描き直します。

用語: state（ステート＝状態） ... コンポーネントが内部で持つ「変化するデータ」。state を更新すると、Reactが自動で画面を最新の状態に描き直します。

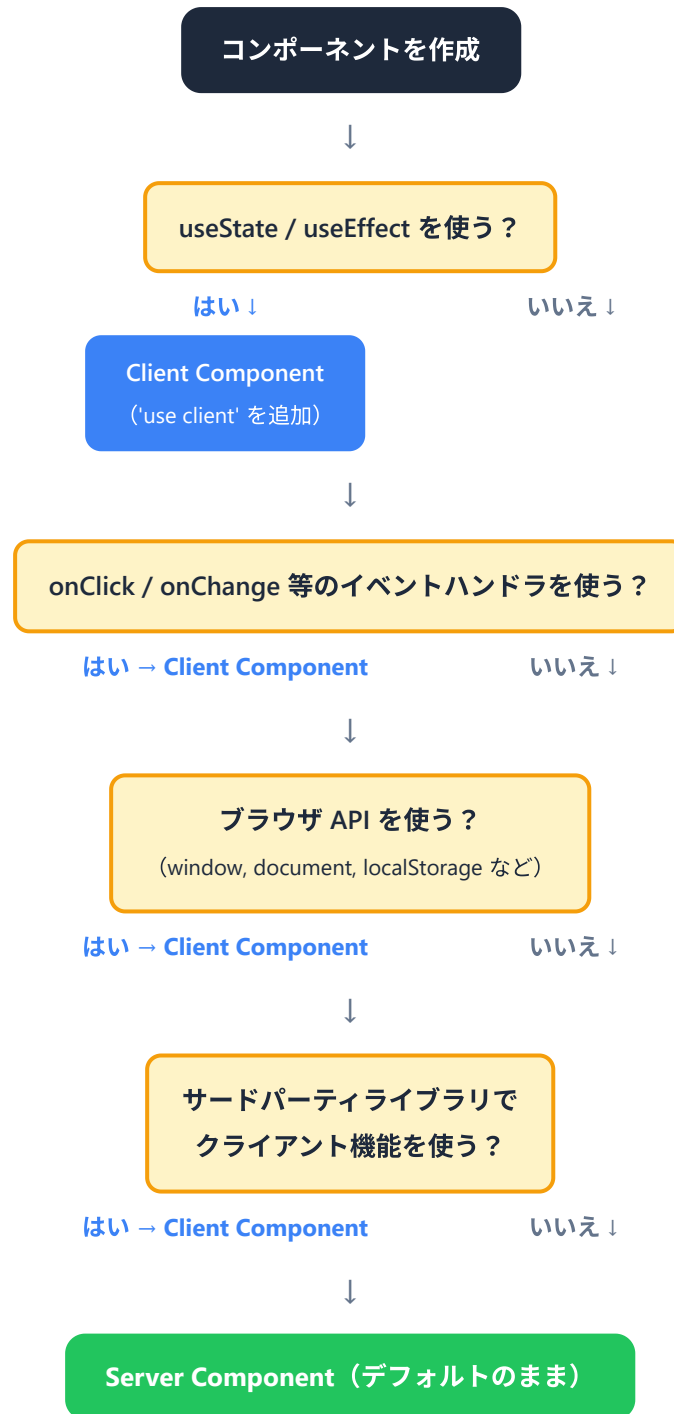
解説3: `value` と `onChange` をセットにする「制御コンポーネント」

```
return (  
  <input  
    type="text"  
    value={query}  
    onChange={(e) => setQuery(e.target.value)}  
    placeholder="書籍を検索..."  
  />  
);
```

- `value={query}` は「入力欄の表示内容を、state (`query`) と一致させる」指定です。
- `onChange={(e) => setQuery(e.target.value)}` は「入力欄の中身が変わるたびに呼ばれる関数」です。
`e.target.value` (イー・ターゲット・バリュー) が「今入力されている文字」で、それを `setQuery` に渡して state を更新しています。
- この「`value` で表示し、`onChange` で更新する」というセットの仕組みを **制御コンポーネント (controlled component)** と呼びます。入力欄の中身を常にReactの state が握っている状態です。
- `placeholder` (プレースホルダー) は、何も入力していないときに薄く表示される案内文です。

用語: 制御コンポーネント (controlled component) ... 入力欄の値をReactのstateで管理する方式。「画面の表示」と「stateの値」が常に一致し、入力内容をプログラムから自由に読み書きできます。

3.3 いつどちらを使うか（判断フローチャート）



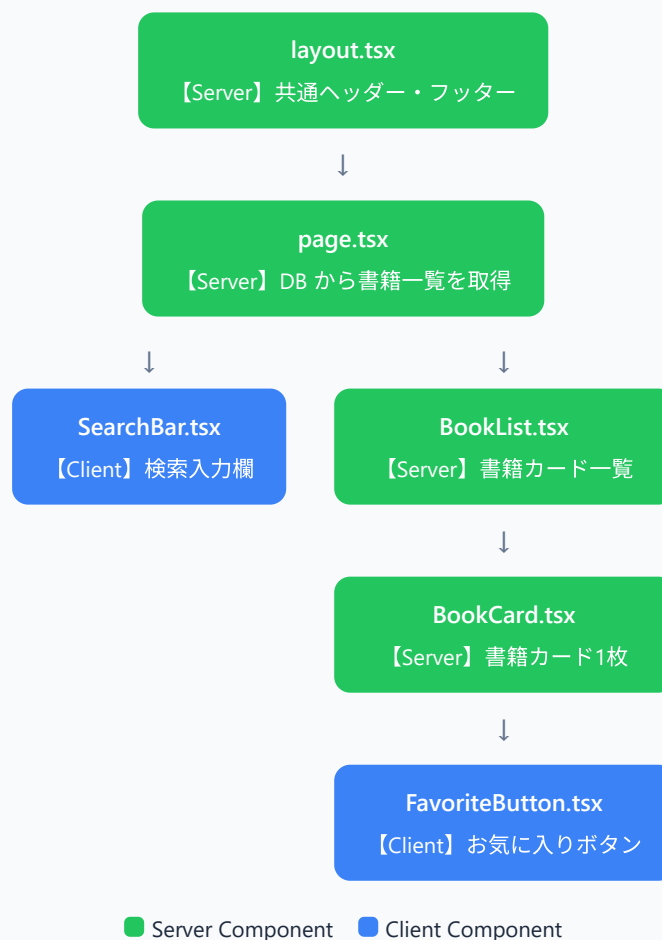
原則: できるだけ Server Component を使い、インタラクティブ性が必要な部分だけを Client Component にします。

用途	Server Component	Client Component
データの取得・表示	推奨	
データベースへの直接アクセス	推奨	不可
機密情報（APIキーなど）へのアクセス	推奨	不可
静的な UI の描画	推奨	
フォーム入力		必須
クリックイベント		必須
useState / useEffect		必須
ブラウザ API（localStorage 等）	不可	必須

3.4 書籍管理アプリでの使い分け例

書籍管理アプリの具体的なページで、どのようにコンポーネントを分割するか見てみましょう。

書籍一覧ページ /books — コンポーネント構成



▼このコードがやること（先に日本語で）：書籍一覧ページ本体を Server Component として作り、サーバー上で書籍データを取得してから画面を組み立てます。ポイントは「Server の骨組み（このページ）の中に、Client の部品（検索バー）と Server の部品（一覧表示）を組み合わせる」という App Router の基本パターンです。取得した **books** を子コンポーネントに props（受け渡しデータ）として渡します。詳細はコード内コメントを参照してください。


```

// ` /books ` のページ本体。"use client" なしなので Server Component
// app/books/page.tsx (Server Component)
// データベースから直接書籍一覧を取得する

// ` @/ ` はプロジェクトルートを示すエイリアス
import { SearchBar } from "@components/SearchBar";

// .../.../ のような相
// パス地獄を避けるための設定 (tsconfig.json で定義)
// 書籍リストを描画する別の自作コンポーネントを取り込む
import { BookList } from "@components/BookList";

// ` async ` 関数なので本体で ` await ` が使える
export default async function BooksPage() {

// この関数自体が
Server Component
// Server Component なので、サーバー上で直接データ取得ができる
// この処理はブラウザには送信されない
// ` fetch ` は Web 標準の HTTP 通信 API
const response = await fetch("https://api.example.com/books", {

// Next.js は fetch
// Next.js は fetch
// 他に "force-
cache" (SSG 的)、` next: { revalidate: N } ` (ISR 的) がある
});
// ` response.json() ` で本文 JSON を JS オブジェクトに変換
const books = await response.json();

return (
  <div>
    // ページタイトル
    <h1>書籍一覧</h1>
    // JSX 内のコメント記法
    { /* Client Component: ユーザーの入力を受け付ける */ }
    // 検索バー (Client)。ここから下は子コンポーネント
    <SearchBar />
    { /* Server Component: 取得したデータを表示する */ }
    // 取得した books 配列を props として渡す
    <BookList books={books} />

    // ` books =
  {books} ` は「books という prop に変数 books の値を渡す」の意
  </div>
);
}

```

▼ このコードがやること（先に日本語で）：入力した文字でページを検索結果に切り替える検索バーを作ります。Client Component なので入力状態を `useState` で覚えつつ、`useRouter`（ページ移動用の道具）で検索キーワード付きの URL に移動します。日本語や空白は URL にそのまま使えないため `encodeURIComponent` で変換する点も押さえてください。詳細はコード内コメントを参照してください。

```

// 再利用される検索バー部品
// components/SearchBar.tsx (Client Component)
// この行がある=このファイルは Client Component
"use client";

// ブラウザ側で動き、
useState やイベントが使える

// React の状態管理フック
import { useState } from "react";
// App Router 用のルーターフック
import { useRouter } from "next/navigation";

// ※
`next/router` ではない! (あちらは旧 Pages Router 用)

// 名前付き export。`{ SearchBar }` で import される
export function SearchBar() {
  // `useState` で「現在の検索文字列」を管理
  const [query, setQuery] = useState("");

  // 配列の分割代入:
  [現在値, 更新関数] = useState(初期値)
  // ルーター操作オブジェクトを取得 (push/replace/back/refresh等が使える)
  const router = useRouter();

  // フォーム送信時のハンドラ。型は React.FormEvent
  const handleSearch = (e: React.FormEvent) => {
    // ブラウザ標準のフォーム送信 (ページ全体リロード) を止める
    e.preventDefault();
    // 検索クエリ付きで遷移
    // `router.push` で別URLに移動 (履歴に追加)
    router.push(`/books?q=${encodeURIComponent(query)}`);

    //
    `encodeURIComponent` はURLに使えない文字 (空白や日本語) を %エンコード
  };

  return (
    // submit イベントを上のハンドラに紐付け
    <form onSubmit={handleSearch}>
      <input
        // テキスト入力欄
        type="text"
        // 制御コンポーネント: 表示値を React state と同期
        value={query}
        // 入力のたびに state を更新 (再描画される)
        onChange={(e) => setQuery(e.target.value)}
        // 空欄時の薄いガイドテキスト
        placeholder="タイトルで検索..."
      />
      // submit ボタン。クリックで form の onSubmit が走る
      <button type="submit">検索</button>
  );
}

```

```
    </form>
  );
}
```

▼ コードを1つずつ分解して解説

この検索バーには「`useRouter`」でのプログラムのなページ移動」「`encodeURIComponent`」でのURL変換」という、Next.js特有の要素があります。順に見ていきましょう。

解説1: `useRouter` を取り込むのは `next/navigation` から

```
"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
```

- `useRouter`（ユーズ・ルーター）は「プログラムからページを移動する」ための道具（フック）です。ボタンを押した後やフォーム送信後に、コードで「〇〇ページへ飛べ」と指示できます。
- 取り込み元が `next/navigation` である点が最重要です。よく似た `next/router` は古い Pages Router 用で、App Router では使えません（使うとエラーになります）。
- `useRouter` はブラウザ側の機能なので、ファイル先頭に `"use client"` が必須です。

用語: フック（hook） ... React で `use〇〇` という名前の特別な関数の総称。`useState`（状態）、`useRouter`（ページ移動）のように、コンポーネントに機能を「引っ掛けて」使います。

解説2: フォーム送信を受け取って検索結果ページへ移動する

```
const router = useRouter();

const handleSearch = (e: React.FormEvent) => {
  e.preventDefault();
  router.push(`/books?q=${encodeURIComponent(query)}`);
};
```

- `const router = useRouter()` で、ページ移動に使う「ルーター」を取り出します。
- `e.preventDefault()`（プリVENT・デフォルト）は、フォームの「送信するとページ全体が再読み込みされる」という昔ながらの動作を止めるおまじないです。これを書かないと画面がリロードされてしまいます。
- `router.push(...)` で、指定したURLへ移動します。`push` は「ブラウザの履歴に1つ追加しながら移動」なので、移動後に「戻る」ボタンで元のページに戻れます。

用語: preventDefault ... イベントの「ブラウザ標準の振る舞い」を打ち消すメソッド。フォーム送信では「ページのリロードを止める」ために、ほぼ必ず最初に書きます。

解説3: `encodeURIComponent` でURLに使えない文字を変換する

```
router.push(`/books?q=${encodeURIComponent(query)}`);
```

- バッククォート (```) で囲んだ文字列は **テンプレートリテラル** で、`${...}` の中に変数や式を埋め込みます。ここでは `/books?q=検索語` というURLを組み立てています。
- `encodeURIComponent(query)` (エンコード・ユーアールアイ・コンポーネント) は、「URLにそのまま使えない文字 (空白・日本語・記号など) を、安全な形に変換する」関数です。例えば空白は `%20`、日本語は `%E3%...` のような形に変換されます。
- これを通さないと、検索語に空白や日本語が含まれたときにURLが壊れてしまいます。`?q=` の後ろのような「URLに値を載せる」場面では必ず通す習慣をつけましょう。

用語: クエリパラメータ ... URLの `?` 以降に「キー=値」の形で付ける追加情報 (例: `?q=react`)。検索条件やページ番号などを次のページへ渡すのに使います。

▼ **このコードがやること (先に日本語で)** : 書籍1冊分の「カード」表示を Server Component で作ります。タイトルや著者など動かない部分はサーバーで描画し、クリックが必要な「お気に入りボタン」だけを Client Component として中に埋め込みます。`type Book = {...}` は「このオブジェクトはこういう形」と決める TypeScript の型定義で、入力ミスを防いでくれます。詳細はコード内コメントを参照してください。

```

// 書籍1冊分のカード。静的表示部分は Server で
// components/BookCard.tsx (Server Component)

// ※同じフォルダから import するときは "./" で始める
import { FavoriteButton } from "./FavoriteButton";

                                                                    // ここで Client
Component を取り込んでも、子だけが Client になる

// TypeScript の型定義 (=この形のオブジェクト)
type Book = {
  // 書籍ID (文字列)
  id: string;
  // タイトル
  title: string;
  // 著者
  author: string;
};

// props を分割代入で受け取り、型は `{ book: Book }`
export function BookCard({ book }: { book: Book }) {
  return (
    // カード枠 (CSSでスタイル付け)
    <div className="book-card">
      // タイトル表示
      <h3>{book.title}</h3>
      // 著者表示
      <p>{book.author}</p>
      {/* インタラクティブな部分だけ Client Component */}
      // お気に入りボタンには id だけ渡す (最小限のpropsで境界を作る)
      <FavoriteButton bookId={book.id} />
    </div>
  );
}

```

▼ このコードがやること（先に日本語で）：クリックで「お気に入り」の ON/OFF を切り替えるボタンを作ります。Client Component なので、状態を **useState** で覚え、クリックされたらまず画面表示をすぐ切り替え（楽観的更新）、その後サーバーに **fetch** で保存依頼を送ります。ボタンの文字も状態に応じて「☆」と「★」で切り替わります。詳細はコード内コメントを参照してください。

```

// クリック可能なお気に入りボタン
// components/FavoriteButton.tsx (Client Component)
// この1行で Client Component 化
"use client";

// 状態管理用のフック
import { useState } from "react";

// props は `{ bookId: string }`
export function FavoriteButton({ bookId }: { bookId: string }) {
  // 「お気に入りか？」の真偽値state。初期値は false
  const [isFavorite, setIsFavorite] = useState(false);

  // クリック時のハンドラ。async で await が使える
  const handleClick = async () => {
    // まず画面上の状態をトグル (楽観的更新)
    setIsFavorite(!isFavorite);

    // `!isFavorite`
    // 「現在の値の反対」
    // API を呼び出してお気に入り状態を保存
    // テンプレートリテラル (バッククォート ``) で URL を組み立てる
    await fetch(`/api/books/${bookId}/favorite`, {
      // HTTP メソッド: POST (新しい状態を送る)
      method: "POST",
      // JSオブジェクトをJSON文字列に変換してリクエストボディに乗せる
      body: JSON.stringify({ favorite: !isFavorite }),
    });
  };

  return (
    // ボタンクリックで handleClick を発動
    <button onClick={handleClick}>
      // 三項演算子: state によって表示を切り替える
      {isFavorite ? "★ お気に入り済み" : "☆ お気に入り"}
    </button>
  );
}

```

▼ コードを1つずつ分解して解説

このお気に入りボタンには「楽観的更新」「`fetch` でのサーバー保存」「三項演算子での表示切替」という、よく使うパターンが詰まっています。順に見ていきましょう。

解説1: クリックされたら「まず画面を切り替えてから」サーバーに送る

```
const [isFavorite, setIsFavorite] = useState(false);

const handleClick = async () => {
  setIsFavorite(!isFavorite);
  // API を呼び出してお気に入り状態を保存
  await fetch(`/api/books/${bookId}/favorite`, {
    method: "POST",
    body: JSON.stringify({ favorite: !isFavorite }),
  });
};
```

- `isFavorite` は「お気に入りかどうか」を表す `true` / `false` の状態です。初期値は `false`。
- `setIsFavorite(!isFavorite)` の `!` (ビックリマーク) は「反対の値にする」記号です。`true` なら `false` に、`false` なら `true` に切り替わります。
- 注目してほしいのは順番です。まず先に `setIsFavorite` で画面の見た目を切り替え、その後で `await fetch(...)` でサーバーに保存依頼を送っています。こうすると、サーバーの返事を待たずにボタンの見た目が即座に変わるので、ユーザーは「サクサク反応する」と感じます。これを **楽観的更新 (optimistic update)** と呼びます。

用語: 楽観的更新 (optimistic update) ... 「サーバー処理はどうせ成功するだろう」と楽観的に考え、サーバーの返事を待たずに先に画面を更新する手法。体感速度が上がります（失敗時は元に戻す処理を足すこともあります）。

解説2: `fetch` でサーバーに「お気に入り状態」を送る

```
await fetch(`/api/books/${bookId}/favorite`, {
  method: "POST",
  body: JSON.stringify({ favorite: !isFavorite }),
});
```

- `fetch` (フェッチ) は「サーバーと通信する」ためのブラウザ標準の関数です。第1引数が送り先のURL、第2引数が送り方の設定です。
- URLはテンプレートリテラル (```) で組み立てており、`${bookId}` の部分に、props で受け取った書籍IDが入ります。
- `method: "POST"` は「データを送って保存するときの通信方法」です（単に取得するだけなら `GET` ）。
- `body: JSON.stringify({ favorite: !isFavorite })` は、送る中身（本体）です。
`JSON.stringify(...)` は「JavaScriptのオブジェクトを、通信で送れる文字列（JSON）に変換する」関数です。

用語: JSON.stringify ... JavaScriptのオブジェクトや配列を「JSON形式の文字列」に変換するメソッド。サーバーへデータを送るときは、この文字列の形にしてから送ります。

解説3: 三項演算子でボタンの文字を切り替える

```
<button onClick={handleClick}>
  {isFavorite ? "★ お気に入り済み" : "☆ お気に入り"}
</button>
```

- `onClick={handleClick}` で、ボタンがクリックされたら上の `handleClick` 関数が呼ばれるように紐付けています。
- `{isFavorite ? "★ お気に入り済み" : "☆ お気に入り"}` は **三項演算子**です。「**条件 ? 真のときの値 : 偽のときの値**」の形で、`isFavorite` が `true` なら「★ お気に入り済み」、`false` なら「☆ お気に入り」を表示します。
- `isFavorite` の値が切り替わるたびにReactが再描画するので、ボタンの文字も自動で「☆」と「★」が入れ替わります。

用語: 三項演算子 (ternary operator) ... `条件 ? A : B` という形で「条件に応じてAかBを返す」式。`if` 文と違って「値を返す式」なので、JSXの `{ }` の中に直接書けます。

画面にはこう表示される: ページ上部に検索バー、その下に書籍カードが並んで表示されます。各カードには書籍タイトル、著者名、お気に入りボタンがあります。検索バーに文字を入力するとリアルタイムに反映され、お気に入りボタンをクリックすると「☆ お気に入り」が「★ お気に入り済み」に切り替わります。

4. レイアウトとテンプレート

4.1 ルートレイアウト

`app/layout.tsx` は **ルートレイアウト** と呼ばれ、アプリケーション全体に適用されます。`<html>` タグと `<body>` タグを含む **唯一** のレイアウトです。

▼ このコードがやること（先に日本語で）: アプリ全体の一番外側の枠（ヘッダー・本文・フッター）と、ブラウザタブのタイトルなどの基本情報（`metadata`）をまとめて定義します。`metadata` を export すると Next.js が `<head>` を自動で組み立ててくれるので、自分で `<title>` を書く必要がありません。Google Fonts の最適化読み込みも含む、実用的なルートレイアウトの完成形です。詳細はコード内コメントを参照してください。

```

// 予約名 `layout.tsx`。`app/` 直下のは「ルートレイアウト」と呼ばれる
// app/layout.tsx

// すべてのページを包む一番外側の枠
// になる（必須）

// `import type` は「型だけ取り込む」記法
import type { Metadata } from "next";

// 実行時のJSには残らないのでバンドルが軽くなる
// Google Fonts の Inter フォントを Next.js 経由で読み込む
import { Inter } from "next/font/google";

// `next/font/google` はビルド時にフォントをセルフホストして高速化してくれる
// 全ページに適用する CSS をインポート
import "./globals.css";

// 値として使わず副作用のためだけに読み込む書き方

// Google Fonts の Inter フォントを最適化して読み込む
// フォントのインスタンスを作る
const inter = Inter({ subsets: ["latin"] });

// `subsets: ["latin"]` は「ラテン文字だけ含む」指定でサイズ削減

// `metadata` という名前で export すると Next.js が <head> を自動生成
export const metadata: Metadata = {

// 自分で <title> や <meta> を書く必要がない
  title: {
    // 子ページのタイトルに自動で「 | 書籍管理アプリ」を付ける
    template: "%s | 書籍管理アプリ",

    // `%s` の部分が子の title に置き換わる
    // 子ページが title を指定しなかったときのデフォルト
    default: "書籍管理アプリ",
  },
  // <meta name="description"> として出力される。SEOで重要
  description: "あなたの読書記録を管理するアプリケーション",
};

// ルートレイアウトの本体関数。名前は慣習で `RootLayout`
export default function RootLayout({
  // children: このレイアウトに包まれる「各ページのJSX」
  children,
}: {
  // `React.ReactNode` は「JSX/文字列/数値/null など描画できる何でも」を表す型
  children: React.ReactNode;
}) {
  return (

```

```

// `<html>` はルートレイアウトでだけ書く
<html lang="ja">

// `lang="ja"` は日本語ページである
// `lang="ja" は日本語ページである

ことをブラウザや支援技術に伝える
// `<body>` もルートレイアウト専用
<body className={inter.className}>

// `inter.className` でフォント
用 CSS クラスを適用
  {/* ここにヘッダーを配置 */}
  // セマンティック要素 `<header>` : ページ上部のヘッダー領域
  <header className="site-header">
    // `<nav>` : ナビゲーション領域を示すセマンティック要素
    <nav>
      // 普通のアンカー。ロゴ的リンク（後で `<Link>` に置き換えるのがベター）
      <a href="/">書籍管理アプリ</a>
    </nav>
  </header>

  {/* children に各ページの内容が入る */}
  // `<main>` : ページの主要コンテンツ領域
  <main className="site-main">{children}</main>

  // ここに `app/page.tsx` や
  `app/books/page.tsx` 等の中身が差し込まれる

  {/* ここにフッターを配置 */}
  // `<footer>` : ページ下部のフッター領域
  <footer className="site-footer">
    // `&copy;` は © の HTML エンティティ（特殊文字）
    <p>&copy; 2024 書籍管理アプリ</p>
  </footer>
</body>
</html>
);
}

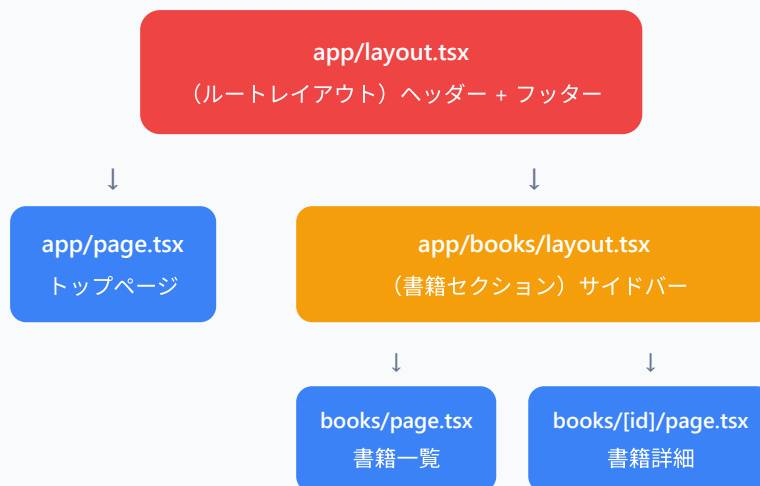
```

ポイント: - ルートレイアウトは **必須** です。削除するとエラーになります。 - `<html>` と `<body>` タグはルートレイアウトにのみ記述します。 - `metadata` オブジェクトで SEO 用のタイトル・説明を設定できます。 - `template: "%s | 書籍管理アプリ"` により、子ページのタイトルが「ページ名 | 書籍管理アプリ」の形式になります。

4.2 ネストされたレイアウト

各ルートセグメント（フォルダ）にも `layout.tsx` を置くことができ、そのセグメント以下のページに適用されます。

レイアウトのネスト構造



▼ このコードがやること（先に日本語で）： `/books` 以下のページだけに共通で付くサイドバー付きの枠（ネストされたレイアウト）を作ります。ルートレイアウトの内側にさらに重ねられるのがポイントで、トップページには出ず `/books` 配下でだけサイドバーが表示されます。ルート以外のレイアウトでは `<html>` / `<body>` は書かない点に注意してください。詳細はコード内コメントを参照してください。

```

// `app/books/` フォルダの中の `layout.tsx`
// app/books/layout.tsx
// `/books`, `/books/new`, `/books/[id]` 等で共通の枠になる
// /books 以下のすべてのページに適用されるレイアウト

// 型だけ import (Metadata 型を使うため)
import type { Metadata } from "next";

// この階層用のメタデータ
export const metadata: Metadata = {
  // 親 (RootLayout) の template と組み合わせ「書籍管理 | 書籍管理アプリ」になる
  title: "書籍管理",
};

// ネストされたレイアウトの本体関数
export default function BooksLayout({
  // children: この階層の page.tsx またはさらに深い layout.tsx の中身
  children,
}: {
  // 描画可能な何でも、を意味する型
  children: React.ReactNode;
}) {
  return (
    // ※ルート以外の layout.tsx は <html>/<body> を書かない
    <div className="books-layout">
      {/* サイドバー */}
      // `<aside>`: 補足コンテンツ用のセマンティック要素
      <aside className="sidebar">
        // ナビゲーション領域
        <nav>
          // 箇条書きリスト
          <ul>
            // ※ 後ほど `<Link>` への置き換えを学ぶ
            <li><a href="/books">書籍一覧</a></li>
            <li><a href="/books/new">書籍を追加</a></li>
            <li><a href="/books/favorites">お気に入り</a></li>
          </ul>
        </nav>
      </aside>

      {/* メインコンテンツ */}
      // `<section>`: 意味的にひとまとまりの領域
      <section className="content">
        // ここに /books/page.tsx や /books/[id]/page.tsx の内容が入る
        {children}
      </section>
    </div>
  );
}

```

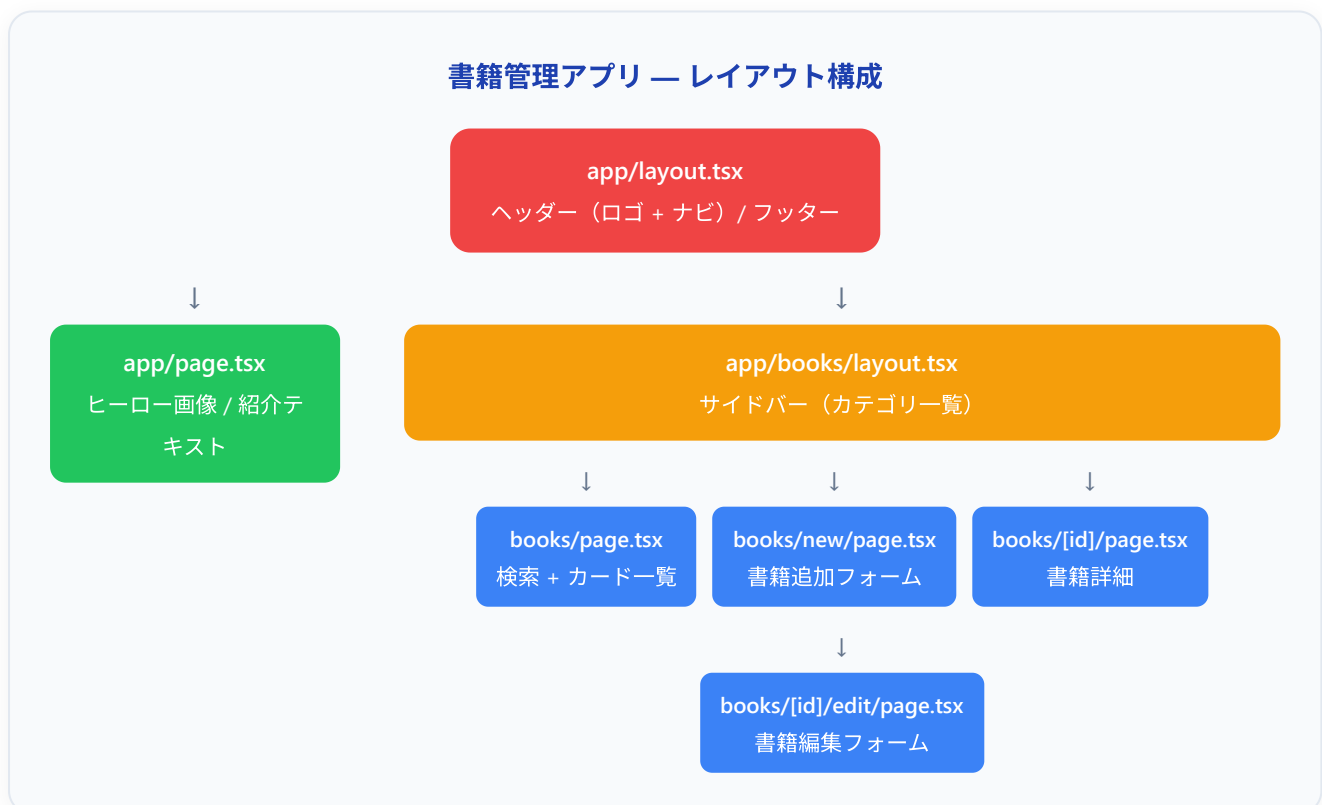
```
};  
}
```

画面にはこう表示される:



`/books` や `/books/123` にアクセスすると、ルートレイアウトのヘッダー・フッターに加えて、書籍セクション専用のサイドバーが表示されます。トップページ (`/`) にはサイドバーは表示されません。

4.3 書籍管理アプリのレイアウト構成



5. ナビゲーション

5.1 Link コンポーネント

Next.js では、ページ間の遷移に HTML の `<a>` タグではなく、`next/link` モジュールが提供する `<Link>` コンポーネントを使います。`<Link>` は内部的に `<a>` を描画しますが、クリック時にページ全体をリロードせず、必要な部分だけを差し替える **クライアントサイド遷移**（client-side navigation：ブラウザでURLだけ書き換えて描画を切り替える方式）を行います。

▼ このコードがやること（先に日本語で）：ページ間を移動するためのナビゲーション（メニュー）を、`next/link` の `<Link>` を使って作ります。`<a>` タグの代わりに `<Link>` を使うと、クリック時にページ全体を読み込み直さず必要な部分だけ差し替える「高速なページ移動」になります。`href` に行き先 URL を指定する書き方は `<a>` とよく似ています。詳細はコード内コメントを参照してください。

```
// ナビゲーション専用コンポーネント
// components/Navigation.tsx
// Next.js の Link コンポーネントを取り込む (default export)
import Link from "next/link";

// 名前付き export (中括弧で import される)
export function Navigation() {
  return (
    // セマンティック要素：ナビゲーション領域
    <nav>
      // リスト
      <ul>
        // 1項目
        <li>
          // `href` で行き先URLを指定。<a> と似た書き方
          <Link href="/">ホーム</Link>
        </li>
        <li>
          // `/books` への高速遷移リンク
          <Link href="/books">書籍一覧</Link>
        </li>
        <li>
          // `/books/new` への遷移
          <Link href="/books/new">書籍を追加</Link>
        </li>
      </ul>
    </nav>
  );
}
```

`<Link>` と `<a>` の違い:

特性	<code><a></code> タグ	<code><Link></code> コンポーネント
ページ遷移	ページ全体をリロード	クライアント側で遷移（高速）
プリフェッチ	なし	ビューポート内のリンクを自動プリフェッチ
状態の保持	リセットされる	レイアウトの状態が保持される
JavaScript	不要	必要

▼ このコードがやること（先に日本語で）：書籍ごとに異なる詳細ページ（例: `/books/42`）へのリンクを、データに応じて動的に組み立てます。`href={ /books/${book.id} }` のようにバッククォート（テンプレートリテラル）で URL に変数を埋め込むのがポイントです。1冊分の `book` を `props` で受け取り、その `id` からリンク先を作ります。詳細はコード内コメントを参照してください。

```
// 動的セグメント `[id]` を含むURLへのリンク例
// 動的ルートへのリンク
// Link コンポーネントを取り込む
import Link from "next/link";

// 書籍の型を簡易定義
type Book = {
  // 書籍ID
  id: string;
  // タイトル
  title: string;
};

// props として 1冊の book を受け取る
export function BookLink({ book }: { book: Book }) {
  return (
    // テンプレートリテラルで URL を組み立てる
    <Link href={`/${books}/${book.id}`}>
                                                                    // 例: book.id が "42" なら
"/books/42" が生成される
    // 表示テキストは book.title を埋め込み
    {book.title} の詳細を見る
    </Link>
  );
}
```

画面にはこう表示される: 「○○の詳細を見る」というリンクテキストが表示されます。クリックすると、ページ全体がリロードされることなく、スムーズに書籍詳細ページに遷移します。ヘッダーやサイドバーのレイアウトはそのまま残り、メインコンテンツ部分だけが切り替わります。

プリフェッチについて: `<Link>` コンポーネントはデフォルトで、ユーザーの画面（ビューポート）に表示されているリンク先のページを裏で先読みします。これにより、リンクをクリックした瞬間にページが表示されるような高速な体験が実現します。

▼ コードを1つずつ分解して解説

この動的リンクには「`<Link>` の import」「テンプレートリテラルでのURL組み立て」という2つのポイントがあります。

解説1: `next/link` から `<Link>` を取り込む

```
import Link from "next/link";

type Book = {
  id: string;
  title: string;
};
```

- `<Link>`（リンク）は、Next.js が用意している「ページ間を高速に移動するための部品」です。`next/link` から取り込みます。
- `import Link from ...`（中括弧なし）の形なのは、`<Link>` が **default export**（そのファイルの主演として公開）されているためです。
- `type Book = { ... }` は「書籍データの形」を決めるTypeScriptの型定義です。`id`（文字列）と `title`（文字列）を持つオブジェクト、と宣言しています。

用語: `<Link>` コンポーネント ... `<a>` タグの代わりに使うNext.jsの移動部品。クリック時にページ全体を読み込み直さず、必要な部分だけ差し替える「高速なページ移動」を実現します。

解説2: テンプレートリテラルで「行き先URL」を動的に組み立てる

```
export function BookLink({ book }: { book: Book }) {
  return (
    <Link href={`\books/${book.id}`}>
      {book.title} の詳細を見る
    </Link>
  );
}
```

- `{ book }` は分割代入で、props から `book`（書籍1冊分のデータ）を取り出しています。型は `{ book: Book }`。

- `href={ /books/${book.id} }` がこのコードのキモです。バッククォート (```) で囲んだ **テンプレートリテラル**を使い、`${book.id}` の部分に書籍IDを埋め込んでいます。`book.id` が `"42"` なら `/books/42` というURLが組み立てられます。
- `<Link>` の内側に書いた `{book.title} の詳細を見る` が、画面に表示されるリンクの文字になります。

用語: テンプレートリテラル ... バッククォートで囲む文字列で、`${変数}` の形で値を埋め込みます。URLやメッセージを「固定部分 + 変化する値」で組み立てるときに便利です。

5.2 useRouter フック

プログラムからページ遷移を行いたい場合（ボタンクリック後やフォーム送信後など）は、`useRouter` フックを使います。**Client Component でのみ使用可能です。**

▼ このコードがやること（先に日本語で）: `<Link>` のクリックではなく「処理が終わった後にプログラムから自動でページを移動する」方法を学びます。フォーム送信後に `useRouter` の `router.push("/books")` を呼んで一覧ページへ飛ばす例です。`useRouter` は Client Component でしか使えず、取り込み元は `next/navigation`（旧 `next/router` ではない）である点が要注意です。詳細はコード内コメントを参照してください。

```
// 書籍フォーム部品
// components/BookForm.tsx
// useRouter は Client Component でしか使えないため必須
"use client";

// ⚠️ next/router ではない!
// App Router 用は `next/navigation`、旧 Pages Router 用は `next/router`
import { useRouter } from "next/navigation";

export function BookForm() {
  // ルーターインスタンスを取得
  const router = useRouter();

  // フォーム送信時のハンドラ
  const handleSubmit = async (e: React.FormEvent) => {
    // デフォルトの送信動作（ページリロード）を防ぐ
    e.preventDefault();

    // 書籍を保存する処理（省略）
    // 例: await fetch("/api/books", { method: "POST", ... });

    // 保存後、書籍一覧ページに遷移
    // プログラム的に画面遷移。履歴に「/books」が追加される
    router.push("/books");
  };

  return (
    // submit イベントを上記関数に紐付け
    <form onSubmit={handleSubmit}>
      // 実際には input 要素などを並べる
      { /* フォームの内容（省略） */ }
      // submit ボタン
      <button type="submit">保存</button>
    </form>
  );
}
```

useRouter の主要メソッド:

▼ このコードがやること（先に日本語で）: **useRouter** が持つ4つの代表的な機能を、ボタンで呼び分けて比較します。**push**（履歴を残して移動）、**replace**（履歴を上書きして移動＝戻れない）、**back**（1つ前に戻る）、**refresh**（今のページのデータだけ取り直す）の違いを押さえるのが目的です。それぞれをいつ使うかはこの後の表にもまとまっています。詳細はコード内コメントを参照してください。

```

// Client Component 化
"use client";

// App Router 用のフック
import { useRouter } from "next/navigation";

export function NavigationExample() {
  // ルーターを取得
  const router = useRouter();

  return (
    <div>
      {/* 指定したパスに遷移（履歴に追加） */}
      // `push`: 履歴を1つ増やしながらか遷移（ブラウザの戻るで前ページに戻れる）
      <button onClick={() => router.push("/books")}>
        書籍一覧へ
      </button>

      {/* 指定したパスに遷移（履歴を置換） */}
      // `replace`: 現在の履歴エントリを上書きして遷移（戻れない）
      <button onClick={() => router.replace("/books")}>
        // ログイン後にログインページを履歴に残したくないときに使う
        書籍一覧へ（履歴を置換）
      </button>

      {/* ブラウザの「戻る」と同じ */}
      // `back`: 履歴を1つ戻る
      <button onClick={() => router.back()}>
        戻る
      </button>

      {/* ページのデータを再取得（再レンダリング） */}
      // `refresh`: 現在のルート of Server Component を再実行・再描画
      <button onClick={() => router.refresh()}>
        // ※ ページ全体リロードではない（state は保持される）
        更新
      </button>
    </div>
  );
}

```

メソッド	説明	使用例
<code>router.push(url)</code>	指定した URL に遷移	フォーム送信後のリダイレクト
<code>router.replace(url)</code>	現在の履歴エントリを置換して遷移	ログイン後のリダイレクト
<code>router.back()</code>	ブラウザの戻るボタンと同じ	詳細ページから一覧に戻る
<code>router.refresh()</code>	現在のページを再取得	データ更新後の再読み込み
<code>router.prefetch(url)</code>	指定した URL を先読み	近く遷移しそうなページの準備

5.2.1 usePathname / useSearchParams (補足)

`next/navigation` には `useRouter` 以外にも、現在の URL に関する情報を取り出すフックがあります。どちらも **Client Component 専用** です。

▼ このコードがやること（先に日本語で）：「今どの URL にいるか」「URL の `?sort=price` のような追加情報（クエリパラメータ）は何か」をプログラムから読み取ります。`usePathname` で現在のパス、`useSearchParams` でクエリの値（`.get("sort")` で取得）を取り出す例です。`??`（ヌル合体演算子）は「値が無いときの代わりの値」を指定する書き方です。詳細はコード内コメントを参照してください。

```
// Client Component 必須
"use client";

// 2つのフックを取り込む
import { usePathname, useSearchParams } from "next/navigation";

export function CurrentUrlInfo() {
  // 例: "/books/123" のような現在のパス（クエリは含まない）
  const pathname = usePathname();
  // クエリパラメータを扱う Readonly な URLSearchParams 風オブジェクト
  const searchParams = useSearchParams();

  // `?sort=price` なら "price" を返す。なければ null
  const sort = searchParams.get("sort");
  // `??` は左辺が null/undefined のとき右辺を使う「ヌル合体演算子」
  const page = searchParams.get("page") ?? "1";

  return (
    <p>
      現在のパス: {pathname}（並び順: {sort ?? "デフォルト"}、ページ: {page}）
    </p>
  );
}
```

5.3 リダイレクト

リダイレクト（redirect：別のURLに自動転送すること）は、Server Component の中で `redirect()` を呼ぶか、後述する `middleware.ts` で行います。

Server Component でのリダイレクト

▼ このコードがやること（先に日本語で）：書籍詳細ページで「条件によって別ページへ自動転送する」処理を学びます。データが見つからなければ `notFound()` で404ページを、非公開の書籍なら `redirect("/books")` で一覧へ飛ばす例です。どちらも呼ぶとそこで処理が止まる（戻ってこない）ため、`return` の前に置くだけで分岐できます。詳細はコード内コメントを参照してください。

```

// 動的セグメントを含む書籍詳細ページ
// app/books/[id]/page.tsx
// 2つのヘルパー関数を取り込む
import { redirect, notFound } from "next/navigation";

// redirect: 任意の
// notFound: 404扱い

URL へ転送

にして not-found.tsx を表示

// async Server Component
export default async function BookDetailPage({
  params,
}: {
  // Next.js 15+ では params は Promise でラップ
  params: Promise<{ id: string }>;
}) {
  // await で id を取り出す
  const { id } = await params;
  // 仮想のデータ取得関数
  const book = await fetchBook(id);

  // 書籍が見つからない場合は 404 ページを表示
  // データが null/undefined なら
  if (!book) {
    // この呼び出しで以降の処理は中断され not-found.tsx が表示される
    notFound();
  }

  // 関数の戻り値は
  never (戻ってこない関数の型)
  }

  // 非公開の書籍は一覧ページにリダイレクト
  // 条件次第で
  if (book.isPrivate) {
    // `/books` に転送 (これも never を返す)
    redirect("/books");
  }

  // 上の if を通り抜けた場合だけここに到達
  return (
    <div>
      // タイトル表示
      <h1>{book.title}</h1>
    </div>
  );
}

// データ取得を模した関数
async function fetchBook(id: string) {
  // データベースから書籍を取得する処理 (仮)

```

```
// 本来は DB クエリや API 呼び出し
// ダミーデータ
return { title: "サンプル書籍", isPrivate: false };
}
```

▼ コードを1つずつ分解して解説

このリダイレクト処理には「`notFound()` で404を出す」「`redirect()` で別ページへ飛ばす」という、Server Component ならではの分岐があります。順に見ていきましょう。

解説1: `redirect` と `notFound` を `next/navigation` から取り込む

```
import { redirect, notFound } from "next/navigation";
```

- `redirect` (リダイレクト) は「別のURLへ自動転送する」関数、`notFound` (ノットファウンド) は「404ページ (見つかりません) を表示する」関数です。どちらも `next/navigation` から取り込みます。
- これらは Server Component の中で呼ぶことを想定した関数です。ユーザーの操作を待たずに「サーバー側で表示前に転送先を決める」ときに使います。

用語: リダイレクト (`redirect`) ... あるURLにアクセスしたユーザーを、自動的に別のURLへ飛ばすこと。ログインが必要なページや、移動した古いページなどで使われます。

解説2: 見つからなければ404、非公開なら一覧へ飛ばす

```
const { id } = await params;
const book = await fetchBook(id);

if (!book) {
  notFound();
}

if (book.isPrivate) {
  redirect("/books");
}
```

- `await fetchBook(id)` で書籍データを取りに行き、結果を `book` に入れます。
- `if (!book) { notFound(); } ... !book` は「`book` が空 (`null / undefined`) なら」という意味です。データが見つからなければ `notFound()` を呼び、404ページ (`not-found.tsx`) を表示します。
- `if (book.isPrivate) { redirect("/books"); } ... 非公開の書籍なら redirect("/books") で一覧ページへ転送します。`

- ポイント: `notFound()` も `redirect()` も「呼んだらそこで処理が止まり、後ろの行には進まない」特殊な関数です。だから `else` を書かなくても、下の `return` まで到達するのは「見つかった & 公開されている」場合だけになります。

用語: 早期リターン的な中断 ... `notFound()` / `redirect()` は戻り値の型が `never` (= 戻ってこない関数) で、呼んだ時点で処理が打ち切られます。そのため `if` で条件を満たしたらすぐ抜ける書き方ができます。

middleware.ts でのリダイレクト

`middleware.ts` (ミドルウェア: リクエストとレスポンスの間に挟まる処理。Next.js では Edge Runtime (軽量な実行環境) で動く) をプロジェクトルートに置くことで、ページが描画される**前の段階**でリダイレクトを行えます。認証チェック、地域別の振り分け、A/Bテストなどに使えます。

▼ このコードがやること (先に日本語で): ページが表示される「前の段階」で割り込み、未ログインのユーザーを `/login` に転送する関所 (ミドルウェア) を作ります。リクエストの `Cookie` からログイン状態を判定し、条件に合えば転送、合わなければそのまま通します。最後の `config.matcher` で「どの URL にこの関所を効かせるか」を指定する点もポイントです。詳細はコード内コメントを参照してください。

```
// ファイル名は固定。app/ の外、プロジェクト直下に置く
// middleware.ts (プロジェクトルート)

// Edge ランタイム用のレスポンスヘルパー
import { NextResponse } from "next/server";
// 型だけ取り込み (リクエスト型)
import type { NextRequest } from "next/server";

// `middleware` という名前で export すると Next.js が自動呼び出し
export function middleware(request: NextRequest) {
  // 例: 未ログインユーザーを /books ページからリダイレクト
  // Cookie から `session` の値を取得 (あればログイン中とみなす)
  const isLoggedIn = request.cookies.get("session");

  // 未ログイン かつ /books 配下にアクセスしている
  if (!isLoggedIn && request.nextUrl.pathname.startsWith("/books")) {
    // /login に転送するレスポンスを返す
    return NextResponse.redirect(new URL("/login", request.url));
  }

  URL("/login", request.url)` はベースURLを保ったまま絶対URLを作る

  // `new

  // 条件に合わない場合は普通に次の処理 (=ページ描画) へ進める
  return NextResponse.next();
}

// ミドルウェアを適用するパスを指定
// `config` という名前で設定オブジェクトを export
export const config = {
  // この matcher にマッチしたURLでだけ middleware が動く
  matcher: ["/books/:path*"],
};

// `:path*` は「任意
の階層を含む」の意味
};
```

6. データフェッチ

6.1 Server Component でのデータ取得

Server Component ではコンポーネント関数を `async` にして、直接 `await` でデータを取得できます。これが Next.js App Router の最も強力な機能の1つです。

▼ このコードがやること（先に日本語で）：Server Component の最大の魅力である「コンポーネント関数を `async` にして、その場で `await` でデータを取得する」書き方を学びます。取得した書籍配列を `map`（1件ずつ変換）でリスト表示する、データ取得の王道パターンです。データが入った状態の HTML がサーバーから届くので表示が速く SEO にも強い、という普通の React との違いも意識してください。詳細はコード内コメントを参照してください。

```
// =====
// ファイルパス: app/books/page.tsx
// 役割      : 書籍一覧ページ。サーバーで全書籍を取ってきてHTMLにする。
// 種類      : Server Component ("use client" を書いていないので自動でこちら)
// -----

// Server Component の最大の魅力は「コンポーネント関数自体を async にできて、
// 直接 await でデータを取ってこられる」こと。
// そのデータはサーバーで埋め込まれた状態でブラウザに届くので、
// 初回表示が速く、SEOにも有利。
// =====

// (1) 書籍データ1件分の「形」を定義
//      Book 型を1度ここで決めておくと、map のコールバック内などで
//      book.〇〇 と書いたとき VS Code が補完してくれる。
type Book = {
  // 一意なID
  id: string;
  // 書籍タイトル
  title: string;
  // 著者名
  author: string;
  // 出版年
  publishedYear: number;
};

// (2) APIから書籍配列を取ってくる関数
//      async を付けると、関数の中で await が使えるようになる。
//      戻り値は「Book型の配列が将来届く」を意味する Promise<Book[]>。
async function getBooks(): Promise<Book[]> {

  // fetch は Web標準のHTTP通信API。
  // 第2引数の cache オプションで Next.js のキャッシュ動作を指定する。
  const response = await fetch("https://api.example.com/books", {
    // ← 毎リクエストで最新データを取得する (=SSR動作)
    cache: "no-store",
    // cache: "force-cache",
    // ← ビルド時に1度だけ取得 (=SSG動作)
    // next: { revalidate: 60 },
    // ← 60秒ごとに再取得 (=ISR動作)
  });

  // (2-1) 通信は成功したけどステータスが 4xx/5xx の場合は失敗扱いにする。
  // response.ok は status が 200~299 のとき true、それ以外で false。
  if (!response.ok) {
    throw new Error("書籍データの取得に失敗しました");
  }
}
```

```

// (2-2) JSON 文字列を JS オブジェクトに変換して返す
return response.json();
}

// (3) ページコンポーネント本体
// 関数自体に async を付けられるのが Server Component の特権。
// Client Component ("use client"あり) では関数本体を async にできない。
export default async function BooksPage() {

  // この行はサーバーで実行される。
  // 取得処理が終わるまでブラウザにはHTMLが送られない（その間 loading.tsx が表示される）。
  const books = await getBooks();

  // (4) 取れたデータを使ってJSXを組み立てる
  // books.length は配列の要素数（書籍が3件なら3）。
  // books.map((book) => ...) で1件ずつJSXに変換する。
  // key={book.id} は React がリストの差分検出に使う必須の属性。
  return (
    <div>
      <h1>書籍一覧 ({books.length}冊) </h1>
      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <h3>{book.title}</h3>
            <p>著者: {book.author} ({book.publishedYear}年) </p>
          </li>
        ))}
      </ul>
    </div>
  );
}

```

▼ APIが3件のデータを返した場合のブラウザ表示:

書籍一覧（3冊）

- ・ リーダブルコード
著者: Dustin Boswell (2012年)
- ・ プロを目指す人のためのTypeScript入門
著者: 鈴木 僚太 (2022年)
- ・ 達人プログラマー
著者: David Thomas (2016年)

▼ 通信失敗時:

`throw new Error(...)` が走ると、Next.js は同じセグメント内の `error.tsx` を自動で表示します。`error.tsx` を作っていない場合は親のエラーバウンダリにバブリングします。

▼ ブラウザに届くHTMLの様子（要点だけ抜粋）：

```
<div>
  <h1>書籍一覧（3冊）</h1>
  <ul>
    <li><h3>リーダブルコード</h3><p>著者：Dustin Boswell（2012年）</p></li>
    <li><h3>プロを目指す人のためのTypeScript入門</h3><p>著者：鈴木 僚太（2022年）</p></li>
    <li><h3>達人プログラマー</h3><p>著者：David Thomas（2016年）</p></li>
  </ul>
</div>
```

▼ 普通のReactとの最大の違い: クライアントサイドの `useState/useEffect` でデータを取りに行くのとは違って、最初からデータが入ったHTMLが返ってくるので、表示がカクカクしない・SEOに強い・JSバンドルも小さく済む。

▼ コードを1つずつ分解して解説

このデータ取得コードには「`async` 関数での `fetch`」「`cache` オプション」「`response.ok` でのエラー判定」「`await` でのページ内データ取得」という要素があります。順に見ていきましょう。

解説1: `async` 関数の中で `fetch` してデータを取りに行く

```
async function getBooks(): Promise<Book[]> {
  const response = await fetch("https://api.example.com/books", {
    cache: "no-store",
  });
}
```

- `getBooks` は「APIから書籍一覧を取ってくる」関数です。`async` が付いているので、中で `await` が使えます。
- `: Promise<Book[]>` は戻り値の型注釈で、「`Book` 型の配列が、将来（非同期で）届く」という意味です。
`Promise<...>` は「後で値が届く」ことを表します。
- `await fetch(...)` で、指定したURLにデータを取りに行き、返事（`response`）が届くまで待ちます。

用語: `fetch`（フェッチ） ... サーバーとHTTP通信してデータをやり取りするブラウザ標準の関数。Next.js ではこれを拡張して、後述のキャッシュ機能を追加しています。

解説2: `cache` オプションで「いつ取り直すか」を決める

```
const response = await fetch("https://api.example.com/books", {
  // ← 毎リクエストで最新データを取得する (=SSR動作)
  cache: "no-store",
  // cache: "force-cache",
  // ← ビルド時に1度だけ取得 (=SSG動作)
  // next: { revalidate: 60 },
  // ← 60秒ごとに再取得 (=ISR動作)
});
```

- `fetch` の第2引数の `cache` (キャッシュ) は、Next.js が標準の `fetch` に追加した独自オプションです。「取得したデータをどれくらい使い回すか」を決めます。
- `cache: "no-store"` は「使い回さず、毎回サーバーから最新を取る」設定で、常に新しいデータが必要なページ (= SSR動作) に向きます。
- コメントアウトされた `"force-cache"` は「1度取ったら使い回す」 (= SSG動作)、`next: { revalidate: 60 }` は「60秒ごとに取り直す」 (= ISR動作) です。この1行を変えるだけで、ページの性質を切り替えられるのが Next.js の特徴です。

用語: キャッシュ (cache) ... 一度取得したデータを保存しておき、次回はそれを再利用する仕組み。通信を減らして高速化できますが、その分データが古くなる可能性があります。

解説3: `response.ok` で通信の成否を確認する

```
if (!response.ok) {
  throw new Error("書籍データの取得に失敗しました");
}

return response.json();
```

- `response.ok` (レスポンス・オーケー) は、通信のステータスが正常 (200~299番台) なら `true`、それ以外 (404や500などのエラー) なら `false` になります。
- `if (!response.ok)` は「正常でなければ」という意味で、`throw new Error(...)` でエラーを発生させます。Next.js ではこのエラーが起きると、自動的に同じフォルダの `error.tsx` が表示されます。
- 最後の `response.json()` は「返ってきた本文 (JSON文字列) を、JavaScriptのオブジェクトに変換する」処理です。これで使える形の書籍配列が返ります。

用語: throw (スロー) ... 「エラーを投げる」命令。`throw new Error("メッセージ")` で意図的にエラーを発生させ、エラー処理 (`error.tsx` など) に流れを移せます。

```
export default async function BooksPage() {
  const books = await getBooks();

  return (
    <div>
      <h1>書籍一覧 ({books.length}冊) </h1>
      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <h3>{book.title}</h3>
            <p>著者: {book.author} ({book.publishedYear}年) </p>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

- ページコンポーネント自体が `async` なので、本体で `const books = await getBooks()` と書いてデータ取得の完了を待てます。この待っている間、ユーザーには `loading.tsx` が表示されます。
- `books.length` は配列の件数（3冊なら3）です。見出しに「書籍一覧（3冊）」のように埋め込んでいます。
- `books.map((book) => (...))` で、配列の各書籍を1件ずつ `<li>`（リスト項目）のJSXに変換しています。`key={book.id}` は、Reactがリストの各項目を見分けるための必須の目印です。

用語: map（マップ） ... 配列の各要素を1つずつ別の形に作り替え、新しい配列を作るメソッド。「データの配列」を「JSX要素の配列」に変換してリスト表示するのが定番の使い方です。

fetch のキャッシュ戦略

Next.js の `fetch` には、標準仕様にはない独自のオプションが追加されています。これらを使い分けることで SSR / SSG / ISR（インクリメンタル静的再生成）を切り替えられます。


```
// 毎回サーバーで最新データを取りに行く動作
// SSR - リクエストのたびに最新データを取得
// `cache: "no-store"` でキャッシュを使わない
fetch(url, { cache: "no-store" });

// ビルド時に1回だけ取得し、以降は同じ結果を返す
// SSG - ビルド時にデータを取得し、キャッシュ（デフォルト）
// `cache: "force-cache"` で強制キャッシュ
fetch(url, { cache: "force-cache" });

// 一定間隔で再取得する中間方式
// ISR - 60秒ごとにキャッシュを再検証
// `next.revalidate` は Next.js が追加した独自オプション
fetch(url, { next: { revalidate: 60 } });
```

// 60 は秒数。指定間隔ごとに次のアクセス時に再取得

// 60 は秒

戦略	説明	使用例
<code>cache: "no-store"</code>	毎回取得（SSR）	ユーザー固有のデータ
<code>cache: "force-cache"</code>	ビルド時に取得（SSG）	変更の少ないデータ
<code>next: { revalidate: N }</code>	N秒ごとに再検証（ISR）	ニュース記事など

6.2 Client Component でのデータ取得

Client Component では、従来の React と同じように `useEffect` と `useState` を使うか、サードパーティのデータ取得ライブラリ（SWR や TanStack Query）を使います。

▼ このコードがやること（先に日本語で）：ブラウザ側でリアルタイムに検索する部品を、従来の React と同じ `useState` + `useEffect` で作ります。入力のたびに毎回通信すると重いので、「入力が止まって300ミリ秒後だけに検索する」デバウンスという工夫を入れています。読み込み中・エラー・結果という3つの状態をそれぞれ管理する、実戦的な例です。詳細はコード内コメントを参照してください。

```

// ブラウザでリアルタイム検索する部品
// components/BookSearch.tsx
// Client Component。useState/useEffect を使うため必須
"use client";

// 状態管理と副作用管理のフック
import { useState, useEffect } from "react";

// 検索結果1件の型
type Book = {
  // 書籍ID
  id: string;
  // タイトル
  title: string;
  // 著者
  author: string;
};

export function BookSearch() {
  // 検索文字列の state (初期値は空文字)
  const [query, setQuery] = useState("");
  // 検索結果の配列。ジェネリック `<Book[]>` で要素の型を指定
  const [books, setBooks] = useState<Book[]>([]);
  // 読み込み中フラグ
  const [isLoading, setIsLoading] = useState(false);
  // エラーメッセージ (なければ null)
  const [error, setError] = useState<string | null>(null);

  // query が変わるたびに走る副作用
  useEffect(() => {
    // 検索クエリが空なら何もしない
    // `trim()` で前後の空白を取り除いて空かチェック
    if (!query.trim()) {
      // 空の結果に
      setBooks([]);
      // ここで打ち切り
      return;
    }

    // デバウンス: 入力が止まって300ms後に検索を実行
    // 300ms後に走るタイマーを仕掛ける
    const timer = setTimeout(async () => {
      // 入力が続いている間
      // クリーンアップで毎回キャンセルされる (=デバウンス)
      // 「読み込み中」に切り替え
      setIsLoading(true);
      // エラー表示はリセット
      setError(null);
    }, 300);
  }, [query]);
}

```

```

// 例外の可能性のある処理を try で囲む
try {
  // 内部APIに GET リクエスト
  const response = await fetch(
    // クエリパラメータ q に検索文字列を入れる
    `/api/books/search?q=${encodeURIComponent(query)}`
  );
  // HTTP ステータスが 200番台でなければ
  if (!response.ok) {
    // 例外を投げて catch に飛ばす
    throw new Error("検索に失敗しました");
  }
  // レスポンス本文 (JSON) を JS オブジェクトに変換
  const data = await response.json();
  // state に保存 → 再描画
  setBooks(data);
  // 通信失敗や上の throw を捕まえる
} catch (err) {
  setError(err instanceof Error ? err.message : "エラーが発生しました");
  // err が Error 型
}

// ならその message を、それ以外なら汎用文を表示
// 成功・失敗どちらでも最後に走る
} finally {
  // 読み込み中フラグを解除
  setIsLoading(false);
}

// 300ms = デバウンス時間
}, 300);

// クリーンアップ: 次の入力 cameたらタイマーをキャンセル
// useEffect が return する関数は「次のeffect実行前」または「unmount時」に呼ばれる
return () => clearTimeout(timer);
// 依存配列. query が変わったときだけ effect を再実行
}, [query]);

return (
  <div>
    <input
      // 1行テキスト入力欄
      type="text"
      // 表示値を state と同期 (制御コンポーネント)
      value={query}
      // 入力変更時に state を更新
      onChange={e => setQuery(e.target.value)}
      // 空欄ガイド
      placeholder="書籍を検索..."
    />

    // 短絡評価: isLoading が true のときだけ <p> を描画
    {isLoading && <p>検索中...</p>}
  </div>
);

```

```
// エラー文字列があるときだけ表示
{error && <p className="error">{error}</p>}

<ul>
  // 配列を JSX 要素にマッピング
  {books.map((book) => (
    // `key` は React に「どの項目が同じか」を伝える必須属性
    <li key={book.id}>
      // タイトル - 著者 の形式で表示
      {book.title} - {book.author}
    </li>
  ))}
</ul>
</div>
);
}
```

画面にはこう表示される: 検索入力欄が表示されます。ユーザーが「村上」と入力すると、入力が止まって300ミリ秒後に「検索中...」というメッセージが一瞬表示され、その後「村上春樹」の著書一覧がリスト形式で表示されます。

▼ コードを1つずつ分解して解説

このクライアント検索には「複数の state 管理」「`useEffect`」での副作用「デバウンス」「`try/catch/finally`」での通信処理」という要素が組み合わさっています。順に見ていきましょう。

解説1: 4つの state で「入力・結果・読み込み中・エラー」を管理する

```
const [query, setQuery] = useState("");
const [books, setBooks] = useState<Book[]>([]);
const [isLoading, setIsLoading] = useState(false);
const [error, setError] = useState<string | null>(null);
```

- Client Component で通信を伴う検索を作るには、複数の状態を同時に管理する必要があります。ここでは4つの `useState` を使っています。
- `query` : 入力された検索文字列（初期値は空文字）。
- `books` : 検索結果の配列。`useState<Book[]>([])` の `<Book[]>` は「`Book` 型の配列を入れる箱」という型指定で、初期値は空配列 `[]`。
- `isLoading` : 読み込み中かどうかの `true / false`。
- `error` : エラーメッセージ。`<string | null>` は「文字列、またはエラーなしの `null`」という型で、初期値は `null`。

用語: ジェネリック (<...>) ... `useState<Book[]>(...)` の `<Book[]>` のように、型を `< >` で指定する書き方。「この箱には何の型が入るか」をTypeScriptに明示できます。

解説2: `useEffect` で「`query` が変わるたびに検索する」

```
useEffect(() => {
  if (!query.trim()) {
    setBooks([]);
    return;
  }
  // ... (タイマーの仕掛け) ...
}, [query]);
```

- `useEffect` (ユーズ・エフェクト) は「画面の描画とは別に行う処理 (副作用)」を書くフックです。データ取得はこの代表例です。
- 末尾の `[query]` は 依存配列 で、「`query` が変わったときだけ、この中の処理を実行する」という指定です。つまり入力が変わるたびに検索処理が走ります。
- `if (!query.trim())` は「入力が空 (または空白だけ) なら」の判定です。その場合は結果を空にして `return` で打ち切り、無駄な通信を避けます。

用語: 副作用 (side effect) / 依存配列 ... 副作用は「画面を描く以外の処理 (通信・タイマーなど)」のこと。依存配列はその副作用を「どの値が変わったら実行し直すか」を指定するリストです。

解説3: デバウンス —「入力が止まって300ミリ秒後」にだけ検索する

```
const timer = setTimeout(async () => {
  // ...通信処理...
}, 300);

return () => clearTimeout(timer);
```

- `setTimeout(関数, 300)` は「300ミリ秒後にこの関数を実行する」タイマーを仕掛けます。
- 最後の `return () => clearTimeout(timer)` は クリーンアップ関数 で、「次の入力が来たら (= effectが再実行される直前に) 前のタイマーを取り消す」処理です。
- この「タイマーを仕掛ける一次の入力で取り消す」の繰り返しにより、ユーザーが文字を打ち続けている間は検索が走らず、手が止まって300ミリ秒経ったときだけ検索が実行されます。これを デバウンス (debounce) と呼びます。毎文字ごとに通信する無駄を防げます。

用語: デバウンス (debounce) ... 連続して起きるイベント（キー入力など）を間引き、「最後の操作から一定時間経ったとき」だけ処理を実行する手法。検索や入力補完で多用されます。

解説4: `try / catch / finally` で通信の成功・失敗・後始末を分ける

```
try {
  const response = await fetch(
    `/api/books/search?q=${encodeURIComponent(query)}`
  );
  if (!response.ok) {
    throw new Error("検索に失敗しました");
  }
  const data = await response.json();
  setBooks(data);
} catch (err) {
  setError(err instanceof Error ? err.message : "エラーが発生しました");
} finally {
  setIsLoading(false);
}
```

- `try { ... }` には「失敗するかもしれない処理（通信）」を書きます。成功すれば、結果を `setBooks(data)` で state に保存します。
- `catch (err) { ... }` は「`try` の中でエラーが起きたとき」に実行されます。`err instanceof Error ? err.message : "エラーが発生しました"` は「`err` がちゃんとした Error なら、そのメッセージを、そうでなければ汎用メッセージを使う」という三項演算子です。
- `finally { ... }` は「成功しても失敗しても必ず最後に実行される」ブロックです。ここで `setIsLoading(false)` を呼び、どちらの場合も「読み込み中」表示を確実に解除しています。

用語: try / catch / finally ... エラーが起きうる処理を安全に扱う構文。`try` で試し、`catch` でエラーを受け止め、`finally` で後始末（必ず実行）をします。

Server Component と Client Component のデータ取得の比較:

特性	Server Component	Client Component
コード量	少ない (async/await のみ)	多い (useState + useEffect)
ローディング状態	loading.tsx が自動管理	自分で管理が必要
エラー処理	error.tsx が自動管理	自分で管理が必要
データの安全性	APIキー等が露出しない	APIキーは使えない
SEO	HTML にデータが含まれる	初回は空
リアルタイム更新	ページ再読み込みが必要	可能

6.3 loading.tsx による読み込み状態

`loading.tsx` を配置するだけで、Server Component のデータ取得中に自動的にローディング UI が表示されます。

▼ このコードがやること（先に日本語で）：読み込み中に、実際のコンテンツと同じ形をした灰色の枠（スケルトン UI）を表示するローディング画面を作ります。データが届いた瞬間に画面がガタつかないようにするための工夫です。

`Array.from({ length: 6 }).map(...)` で同じカードを6個並べる書き方や、`style` 属性にオブジェクトを渡す JSX 特有の書き方を学べます。詳細はコード内コメントを参照してください。

```

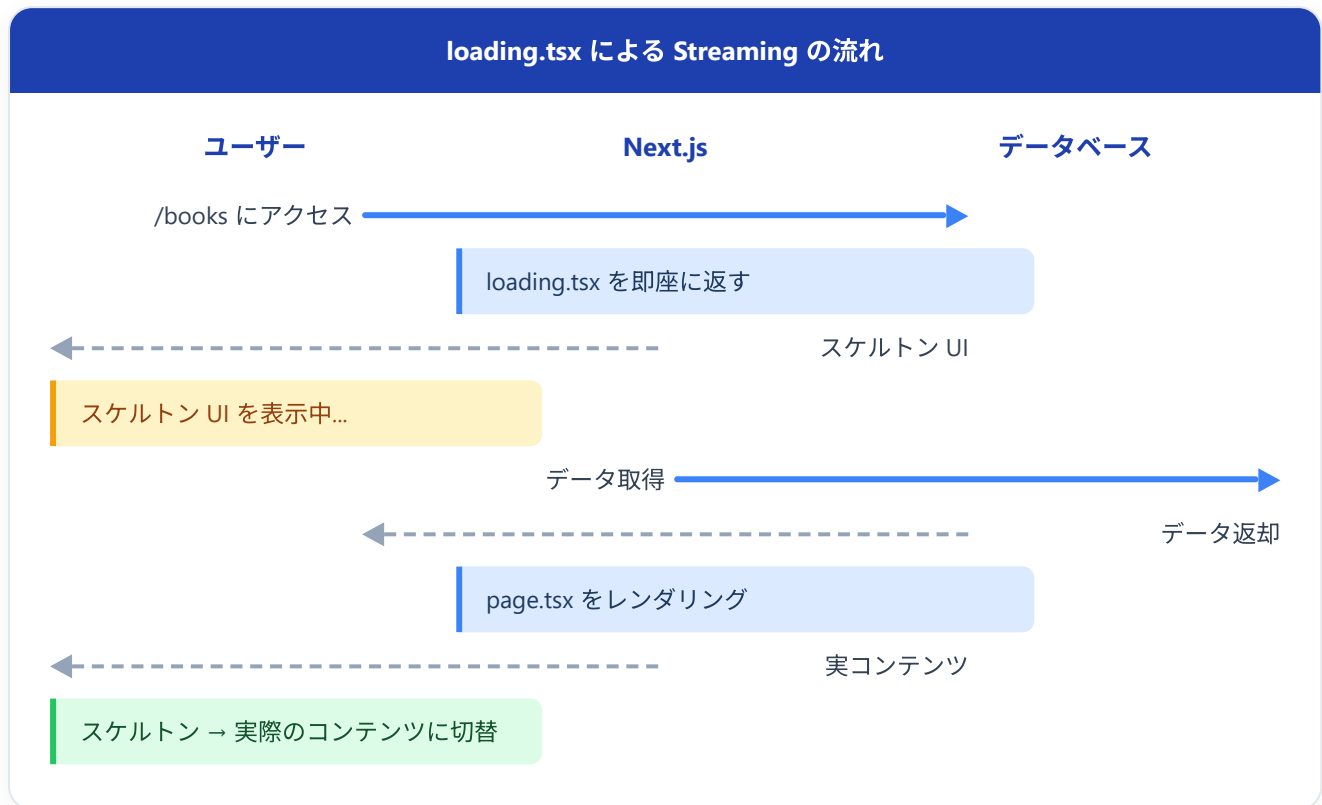
// 予約名 `loading.tsx`。/books のロード中UI
// app/books/loading.tsx

// default export 関数
export default function BooksLoading() {
  return (
    <div className="loading-container">
      {/* スケルトンUI: 実際のコンテンツと同じ形状の灰色の枠 */}
      <h1 className="skeleton" style={{ width: "200px", height: "32px" }} />
      // `style` 属性はオブ
      ジェクトを渡す (CSSのキャメルケース表記)
      // `{{ ... }}` の外
      側はJSXの式、内側はJSオブジェクト

      <div className="book-grid">
        {/* 6個のスケルトンカード */}
        // `Array.from({ length: 6 })` で要素6個の配列を作る
        {Array.from({ length: 6 }).map((_, i) => (
          // 第1引数 `_` は使わ
          ないので慣習的にアンダースコア
          // 第2引数 `i` は
          0,1,2,...,5 のインデックス
          // key にインデックスを使う (並び順が変わらないリストならOK)
          <div key={i} className="skeleton-card">
            <div
              className="skeleton"
              // 表紙画像の代わりに灰色ブロック
              style={{ width: "100%", height: "200px" }}
            />
            <div
              className="skeleton"
              style={{ width: "80%", height: "20px", marginTop: "8px" }}
              // タイトル代わりに細
              長い灰色ブロック
            />
            <div
              className="skeleton"
              style={{ width: "60%", height: "16px", marginTop: "4px" }}
              // 著者代わりにさらに
              小さなブロック
            />
          </div>
        )]}
      </div>
    </div>
  );
}

```


画面にはこう表示される: データの読み込み中、見出しの位置に灰色のバーが1つ、その下に灰色のカード状のブロックが6個並びます。これらは実際のコンテンツと同じ大きさ・位置で表示されるため、読み込みが完了するとスムーズに実際のコンテンツに切り替わり、画面がガタつきません（この手法を **スケルトンUI** と呼びます）。



6.4 Route Handlers（補足）

App Router で「自前の HTTP API」を作りたいときは、`app/api/.../route.ts` というファイルを置きます。これを **Route Handler**（ルートハンドラ：URLとHTTPメソッドに対応する関数を書くファイル）と呼びます。Server Component から直接DBにアクセスできる App Router では出番は減りましたが、外部から `fetch` で叩かれるエンドポイントを公開したい場合に使います。

▼ このコードがやること（先に日本語で）：`app/api/.../route.ts` という予約名のファイルで、外部から `fetch` で呼べる自前の HTTP API を作ります。`GET` や `POST` という名前の関数を `export` すると、それぞれが同名の HTTP メソッドの処理になるのがポイントです。`NextResponse.json(...)` で JSON を返す、API の基本形を学べます。詳細はコード内コメントを参照してください。

```
// 予約名 `route.ts`。`/api/books` の HTTP API になる
// app/api/books/route.ts

// page.tsx と同じフォ
ルダには置けない（ルートが衝突するため）

// レスポンスを作るヘルパー
import { NextResponse } from "next/server";

// 関数名 `GET` が HTTP メソッドに対応（POST/PUT/DELETE等も同様）
export async function GET() {
  // 仮データ
  const books = [
    { id: "1", title: "リーダブルコード" },
    { id: "2", title: "達人プログラマー" },
  ];
  // 配列を JSON にしてレスポンスとして返す
  return NextResponse.json(books);
}

// POST メソッド用。引数は Web 標準の Request
export async function POST(request: Request) {
  // リクエストボディの JSON を解釈
  const body = await request.json();
  // ここで body を使ってDB保存などを行う
  return NextResponse.json({ created: true, body }, { status: 201 });
  // 201 Created を返す
}

例
}
```

6.5 next/image（補足）

画像は `next/image` の `<Image>` コンポーネントを使うと、自動でサイズ最適化・WebP変換・遅延ロードが行われます。

▼このコードがやること（先に日本語で）：画像を `<img>` ではなく `next/image` の `<Image>` で表示し、サイズ最適化・WebP変換・遅延読み込みを自動で効かせます。`width / height` はレイアウト崩れ防止のために必須、`alt`（代替テキスト）もアクセシビリティのために必須です。最重要画像には `priority` を付ける、という使い方を学べます。詳細はコード内コメントを参照してください。

```
// default export の Image コンポーネント
import Image from "next/image";

export function BookCover() {
  return (
    <Image
      // public/covers/book.jpg を指す (public 直下は `/` で参照)
      src="/covers/book.jpg"
      // 代替テキスト (必須)
      alt="書籍の表紙"
      // 元画像のアスペクト比計算用
      width={200}
      // 画像のレイアウト崩れを防ぐ
      height={300}
      // LCP (最大コンテンツ描画) に関わる重要画像はこれを付ける
      priority
    />
  );
}
```

public/ フォルダ: 画像や favicon などの静的ファイルは **public/** に置きます。URL では **/** から直接参照します (例: **public/logo.png** → **/logo.png**)。

6.6 next.config.js / next.config.ts (補足)

プロジェクトルートの **next.config.js** (または TypeScript 版 **next.config.ts**) は Next.js 全体の設定ファイルです。画像の許可ドメインや実験的機能の有効化などをここで指定します。

▼このコードがやること (先に日本語で) : プロジェクト全体の設定ファイル **next.config.ts** の基本形を学びます。ここでは **next/image** で外部サイトの画像を表示してよいドメインを **remotePatterns** で許可する例を示します (許可しないと外部画像は表示されません)。設定オブジェクトを作り **export default** する、という決まった書き方を押さえてください。詳細はコード内コメントを参照してください。

```
// 設定ファイル本体
// next.config.ts
// 型を取り込み
import type { NextConfig } from "next";

// 設定オブジェクト
const nextConfig: NextConfig = {
  // next/image の設定
  images: {
    // 外部URLを許可するリスト（明示しないと表示できない）
    remotePatterns: [
      { protocol: "https", hostname: "example.com" },
    ],
  },
  // experimental: { ... }
  // 実験的機能はここで ON にする
};

// default export することで Next.js が読み込む
export default nextConfig;
```

7. Server Actions

7.1 Server Actions とは

Server Actions は、Client Component から直接サーバー上の関数を呼び出せる 仕組みです。API ルートを別途作成する必要がなく、フォーム処理やデータの変更（作成・更新・削除）をシンプルに書けます。

従来の方法

Client Component
(フォーム)

fetch('/api/books')
→

API Route
(route.ts)

DB操作
→

データベース

Server Actions (新しい方法)

Client Component
(フォーム)

関数を直接呼び出し
→

Server Action
(サーバー上で実行)

DB操作
→

データベース

Server Actions は `"use server"` ディレクティブで宣言します。 `"use client"` がファイル全体を Client Component にするのと同じ要領で、 `"use server"` はファイルや関数を「サーバー上で動く関数」として印を付ける役割を持ちます。

7.2 フォーム処理での利用

基本的な Server Action

▼ このコードがやること（先に日本語で）：フォーム送信を処理する関数（Server Action）を作ります。ファイル先頭の `"use server"` が「この中の関数はサーバー上で動く」という宣言で、これにより別途 API を作らなくてもフォームの保存処理が書けます。送信された値を `formData.get("title")` で取り出し、入力チェック（バリデーション）してから保存する流れを学べます。詳細はコード内コメントを参照してください。

```

// Server Action 用の専用ファイル
// app/books/new/actions.ts
// ファイル先頭の `use server` で「この中の関数は全部サーバー実行」と宣言
"use server";

// この関数はサーバー上でのみ実行される
// クライアントには関数の中身は送信されない
// 引数 `FormData` は Web標準のフォーム値コンテナ
export async function createBook(formData: FormData) {
  // <form action=
  {createBook}> の送信内容が自動で渡される
  // `formData.get("title")` で name="title" の値を取り出す
  const title = formData.get("title") as string;
  // 戻り値は
  `FormDataEntryValue | null` なので `as string` で型を絞り込む
  // 同じく著者
  const author = formData.get("author") as string;
  // 数値に変換 (formData の値は基本文字列)
  const publishedYear = Number(formData.get("publishedYear"));

  // バリデーション
  // 必須項目が空ならエラーを返す
  if (!title || !author) {
    // オブジェクトを返すと呼び出し側で state として扱える
    return {
      error: "タイトルと著者は必須です",
    };
  }

  // データベースに保存 (例)
  // 本番ではここで Supabase や Prisma で INSERT
  // const book = await db.book.create({
  //   data: { title, author, publishedYear },
  // });

  // サーバーのコンソールに出力 (ブラウザには出ない)
  console.log("書籍を追加:", { title, author, publishedYear });

  // 成功したら書籍一覧にリダイレクト
  // redirect("/books");
  // `redirect()` を呼べば一覧ページに自動遷移
  // 成功フラグを返す
  return { success: true };
}

```

▼ コードを1つずつ分解して解説

この Server Action には「`"use server"` 宣言」「`FormData` からの値の取り出し」「バリデーション」という、フォーム処理の基本が詰まっています。順に見ていきましょう。

解説1: ファイル先頭の `"use server"` で「サーバー専用の関数」と宣言する

```
"use server";

export async function createBook(formData: FormData) {
```

- `"use server"`（ユーズ・サーバー）は、ファイルの一番上を書くディレクティブです。これを書くと、このファイルの関数は「サーバー上でだけ実行される」関数（Server Action）になります。
- `"use client"` がファイルを「ブラウザ側」に切り替えるのと同になる存在です。`"use server"` で印を付けた関数の中身はブラウザに送られないので、DB接続情報などの機密を安全に扱えます。
- `createBook` は `async` 関数で、引数に `FormData` を1つ受け取ります。この関数を `<form action={createBook}>` のように指定すると、フォーム送信時にサーバーで実行されます。

用語: Server Action（サーバーアクション） ... `"use server"` を付けた、サーバー上で実行される関数。別途 API（route.ts）を作らなくても、フォーム送信やデータ更新の処理を直接書けます。

解説2: `FormData` から `get()` で入力値を取り出す

```
const title = formData.get("title") as string;
const author = formData.get("author") as string;
const publishedYear = Number(formData.get("publishedYear"));
```

- `formData`（フォームデータ）は、送信されたフォームの入力値が全部入った「Web標準の箱」です。
- `formData.get("title")` は「`name="title"` の入力欄の値を取り出す」という意味です。取り出すキーは、各 `<input>` の `name` 属性と一致します（`id` ではない点に注意）。
- `as string` は型アサーションで、「`get()` の戻り値（文字列 | null というあいまいな型）を、文字列として扱ってね」とTypeScriptに伝えています。
- `Number(formData.get("publishedYear"))` は、取り出した値を数値に変換しています。フォームの値は基本すべて文字列なので、数値が必要なときは `Number(...)` で変換します。

用語: `FormData` ... HTMLフォームの入力値を「キー（name）→値」の形でまとめて持つWeb標準のオブジェクト。`.get("キー名")` でそれぞれの値を取り出します。

```
if (!title || !author) {  
  return {  
    error: "タイトルと著者は必須です",  
  };  
}
```

- `if (!title || !author)` は「タイトルが空、または著者が空なら」という条件です。`!title` は「`title` が空（falsy）なら true」、`||` は「または」です。
- 必須項目が欠けていれば、`{ error: "..."` というオブジェクトを `return` します。このオブジェクトは、呼び出し側（フォーム）で「エラーメッセージの表示」に使えます（後述の `useActionState` の例で活用します）。
- ここを通り抜けたとき（＝両方入力されているとき）だけ、下のDB保存処理に進みます。

用語: バリデーション（validation） ... 入力された値が正しいか（必須項目が空でないか等）をチェックすること。不正なデータがDBに保存されるのを防ぐ、フォーム処理の必須ステップです。

Server Component のフォーム（JavaScript 不要）

▼ このコードがやること（先に日本語で）：先ほどの Server Action を使う「書籍追加フォーム」を Server Component として作ります。`<form action={createBook}>` のように `action` 属性に関数を直接渡せるのが Next.js の拡張で、JavaScript が動かない環境でも送信できる堅牢な作りになります（Progressive Enhancement）。各 `<input>` の `name` 属性が、サーバー側で値を取り出すときのキー名になる点が重要です。詳細はコード内コメントを参照してください。


```

// 新規追加ページ。"use client" を書かないので Server
// app/books/new/page.tsx (Server Component)

// 同じフォルダの actions.ts から Server Action を取り込む
import { createBook } from "../actions";

// 同期関数でOK (このページ自体は async データ取得をしない)
export default function NewBookPage() {
  return (
    <div>
      <h1>書籍を追加</h1>

      // `<form action={...}>` に関数を渡せるのが Next.js の拡張
      {/* action 属性に Server Action を渡す */}
      {/* JavaScript が無効でも動作する (Progressive Enhancement) */}
      <form action={createBook}>
        <div>
          // `htmlFor` は HTML の `for` 属性のJSX版 (forはJSの予約語)
          <label htmlFor="title">タイトル</label>
          <input
            // テキスト入力
            type="text"
            // label と紐付けるための id
            id="title"
            // ※ formData.get で取り出すキー名はこの name 属性
            name="title"
            // 入力必須 (ブラウザの標準バリデーション)
            required
          />
        </div>

        <div>
          <label htmlFor="author">著者</label>
          <input
            type="text"
            id="author"
            name="author"
            required
          />
        </div>

        <div>
          <label htmlFor="publishedYear">出版年</label>
          <input
            // 数値入力欄。上下ボタンが付く
            type="number"

```

```

        id="publishedYear"
        name="publishedYear"
      />
    </div>

    // submit ボタン
    <button type="submit">追加する</button>
  </form>
</div>
);
}

```

画面にはこう表示される: 「書籍を追加」という見出しの下に、タイトル、著者、出版年の入力フィールドと「追加する」ボタンが表示されます。各フィールドに入力して「追加する」ボタンを押すと、フォームのデータがサーバーに送信され、`createBook` 関数がサーバー上で実行されます。

Client Component のフォーム（ローディング状態の管理）

▼ このコードがやること（先に日本語で）: 同じフォームを、送信中の状態やエラー表示まで扱える Client Component 版に作り変えます。React 19 の `useActionState` フックを使うと、Server Action と連携しながら「現在の状態・加工済みの送信関数・送信中フラグ」の3つを受け取れます。送信中はボタンを「追加中...」にしてクリック不可にし、エラーがあれば画面に出す、という親切な UI が作れます。詳細はコード内コメントを参照してください。

```

// クライアント側でローディング状態を扱えるフォーム
// components/BookFormClient.tsx
// useActionState は React のフックなので Client 必須
"use client";

// React 19 で追加された Server Action 連携用フック
import { useActionState } from "react";

// 旧名は
useFormState (同じ用途)
// 別ファイルの Server Action を取り込む
import { createBook } from "@app/books/new/actions";

export function BookFormClient() {
  const [state, formAction, isPending] = useActionState(createBook, null);
  // 戻り値は [現在の
state, ラップ済み formAction, 送信中フラグ]
  // 第2引数の null は
state の初期値
  // formAction は
<form action={...}> に渡せるよう加工された関数

  return (
    // 元の createBook ではなく、ラップ済みの formAction を渡す
    <form action={formAction}>
      <div>
        <label htmlFor="title">タイトル</label>
        // name="title" が formData のキーになる
        <input type="text" id="title" name="title" required />
      </div>

      <div>
        <label htmlFor="author">著者</label>
        <input type="text" id="author" name="author" required />
      </div>

      <div>
        <label htmlFor="publishedYear">出版年</label>
        <input type="number" id="publishedYear" name="publishedYear" />
      </div>

      {/* エラーメッセージの表示 */}
      // `?.` はオプションチェイニング (state が null でも安全)
      {state?.error && (
        // Server Action が返した error メッセージを表示
        <p className="error-message">{state.error}</p>
      )}

      {/* 送信中はボタンを無効化 */}
      // `disabled={isPending}` で送信中はクリック不可

```

```

<button type="submit" disabled={isPending}>
  // ボタンラベルも切り替え
  {isPending ? "追加中..." : "追加する"}
</button>
</form>
);
}

```

画面にはこう表示される: フォームは前の例と同じ見た目ですが、「追加する」ボタンを押すと、ボタンのテキストが「追加中...」に変わり、ボタンがグレイアウトして再クリックできなくなります。バリデーションエラーがあれば、ボタンの上にエラーメッセージが赤字で表示されます。

▼ コードを1つずつ分解して解説

このクライアントフォームの核心は React 19 の `useActionState` フックです。「3つの戻り値」「ラップ済み関数の使い方」「送信中の状態表示」を順に見ていきましょう。

解説1: `useActionState` で Server Action と連携する

```

"use client";

import { useActionState } from "react";
import { createBook } from "@/app/books/new/actions";

export function BookFormClient() {
  const [state, formAction, isPending] = useActionState(createBook, null);

```

- `useActionState` (ユーズ・アクション・ステート) は、React 19 で追加された「Server Action とフォームをつなぐ」フックです (旧名は `useFormState`)。ブラウザ側のフックなので `"use client"` が必須です。
- 第1引数に Server Action (`createBook`)、第2引数に state の初期値 (`null`) を渡します。
- 戻り値は3つの値の配列で、分割代入で受け取ります。
- `state` : Server Action が `return` した値 (例: `{ error: "..."}`)。
- `formAction` : `<form action={...}>` に渡せるよう加工済みの送信関数。
- `isPending` : 送信処理中かどうかの `true` / `false`。

用語: `useActionState` ... Server Action の「実行結果(state)」「送信用の関数」「送信中フラグ」をまとめて受け取れるReactフック。フォームの送信状態を簡単に扱えます。

```
return (  
  <form action={formAction}>  
    <div>  
      <label htmlFor="title">タイトル</label>  
      <input type="text" id="title" name="title" required />  
    </div>  
    { /* ...他の入力欄... */ }
```

- `<form action={formAction}>` のように、`action` 属性には元の `createBook` ではなく、`useActionState` が返した `formAction` を渡します。これにより、送信時に state の更新や `isPending` の管理が自動で行われます。
- 各 `<input>` の `name` 属性（`name="title"` など）が、Server Action 側で `formData.get("title")` として値を取り出すときのキーになります。`required` はブラウザ標準の「入力必須」チェックです。

用語: action 属性 ... Next.js では `<form>` の `action` に「関数」を直接渡せます（通常のHTMLでは送信先URLを書く場所）。これがServer Actionと連携する入口になります。

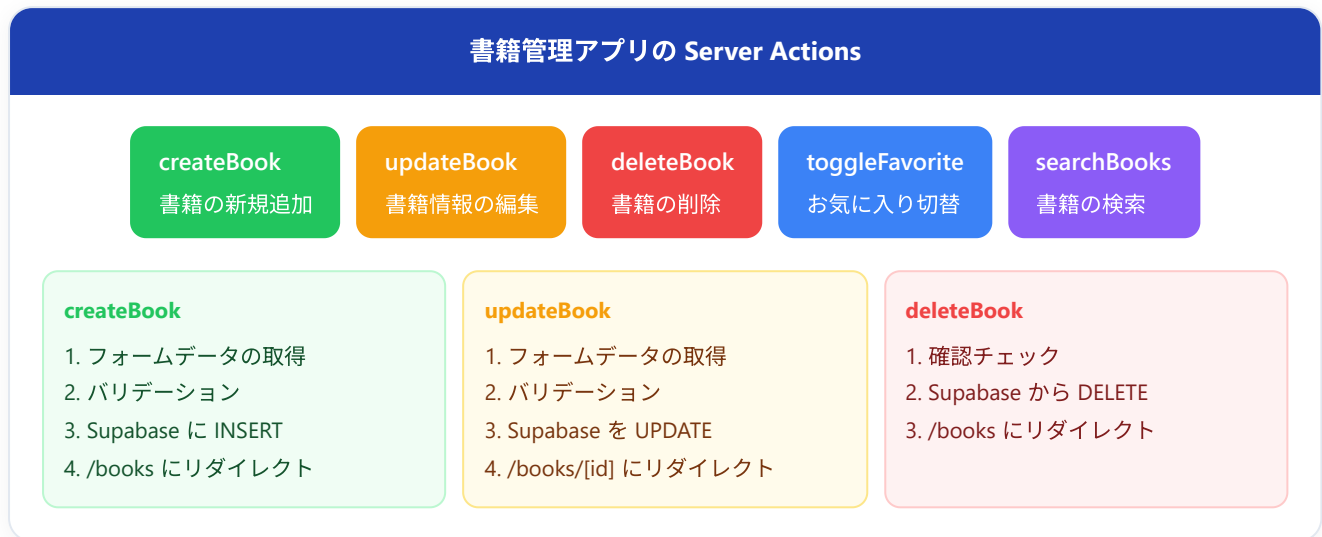
```
{state?.error && (  
  <p className="error-message">{state.error}</p>  
)}  
  
<button type="submit" disabled={isPending}>  
  {isPending ? "追加中..." : "追加する"}  
</button>
```

- `{state?.error && (...)}` ... `state?.error` の `?.`（オプショナルチェイニング）は「`state` が `null` でも安全に `error` を読む」書き方です。エラーメッセージがあるときだけ、その右の `<p>` を表示します（`&&` による条件付き表示）。
- `disabled={isPending}` ... 送信処理中（`isPending` が `true`）の間、ボタンをクリック不可（グレーアウト）にします。これで二重送信を防げます。
- `{isPending ? "追加中..." : "追加する"}` ... 三項演算子で、送信中はボタンの文字を「追加中...」に切り替えます。

用語: オプショナルチェイニング（?.） ... `state?.error` のように書くと、`state` が `null / undefined` のときはエラーにならず `undefined` を返します。「値があるか分からないオブジェクト」を安全にたどれます。

7.3 書籍の追加/編集での利用予告

次章以降で、Server Actions を使って以下の機能を実装します:



予告: 第6章「Supabase との連携」で、実際のデータベースと接続してこれらの Server Actions を完成させます。

8. 環境変数

8.1 .env.local の設定方法

Next.js では、プロジェクトルートに `.env.local` ファイルを作成して環境変数を設定します。

▼ このコードがやること（先に日本語で）: APIキーやデータベース接続情報などの設定値を、コードに直接書かずに `.env.local` というファイルにまとめて管理します。 `KEY="値"` の形で1行に1つ書きます。変数名の先頭に `NEXT_PUBLIC_` を付けるとブラウザにも公開され、付けるとサーバー側だけで使える（＝機密情報を守る）という違いが最大のポイントです。詳細はコード内コメントを参照してください。

```

# .env.local (プロジェクトルートに作成)                                # `#` から始まる行はコメント。シェル/dotenv 共通の記法
# データベース接続情報 (サーバーサイドのみ)                          # NEXT_PUBLIC_ なしなのでブラウザには漏れない
# `KEY="VALUE"` の形で1行1変数。`=` の前後にスペースを入れない
DATABASE_URL="postgresql://user:password@localhost:5432/bookapp"

# Supabase 接続情報 (サーバーサイドのみ)                            # 機密キーは絶対NEXT_PUBLIC_ を付けない
# Supabase プロジェクトのURL
SUPABASE_URL="https://xxxxx.supabase.co"
# サービスロールキー (DB全権限。絶対に公開してはいけない)
SUPABASE_SERVICE_ROLE_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6I..

# Supabase 接続情報 (クライアントサイドでも使用可能)              # NEXT_PUBLIC_ プレフィックスでブラウザJSにも埋め込まれる
# 同じURL でも公開してよい想定なら NEXT_PUBLIC_ を付ける
NEXT_PUBLIC_SUPABASE_URL="https://xxxxx.supabase.co"
# 匿名キー (Row Level Security で制限される前提なので公開OK)
NEXT_PUBLIC_SUPABASE_ANON_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6I.."

```

重要: `.env.local` は `.gitignore` に含まれているため、Git にコミットされません。APIキーやパスワードなどの機密情報を安全に管理できます。

8.2 NEXT_PUBLIC_ プレフィックス

環境変数のアクセス範囲は、変数名のプレフィックスによって決まります。

NEXT_PUBLIC_ あり	NEXT_PUBLIC_ なし
NEXT_PUBLIC_SUPABASE_URL	SUPABASE_SERVICE_ROLE_KEY
✓ Server Component	✓ Server Component
✓ Client Component	✗ Client Component (undefined)
⚠ ブラウザの JS に含まれる	✓ ブラウザには絶対に送信されない

▼このコードがやること（先に日本語で）：Server Component の中で環境変数を読み取る例です。サーバー側では `process.env.変数名` で、`NEXT_PUBLIC_` 付きでも無しでも、すべての環境変数にアクセスできます。機密キーもここでなら安全に使えます（ブラウザには送られない）。`console.log` の出力もブラウザではなくサーバーのターミナルに出る点を意識してください。詳細はコード内コメントを参照してください。

```
// サーバー側ではすべての環境変数にアクセスできる
// Server Component での使用
// app/books/page.tsx

export default async function BooksPage() {
  // ✅ どちらもアクセス可能
  // `process.env` は Node.js 標準の環境変数オブジェクト
  const url = process.env.SUPABASE_URL;
  // 機密キー。Server Component なので安全に取得可能
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;
  // 公開用キーも同様に取れる
  const publicUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;

  // "https://xxxxx.supabase.co"
  // サーバーのコンソールに出力（ブラウザDevToolsには出ない）
  console.log(url);
  // "eyJhbGciOi..."
  // ※ 本番ではこれをログに出さないこと
  console.log(serviceKey);
  // "https://xxxxx.supabase.co"
  console.log(publicUrl);

  // 省略表記
  return <div>...</div>;
}
```

▼このコードがやること（先に日本語で）：同じ `process.env` でも、Client Component（ブラウザ側）では `NEXT_PUBLIC_` 付きの変数しか読めない、という違いを確かめる例です。`NEXT_PUBLIC_` なしの機密キーを参照すると `undefined`（値なし）になり、これは「漏らさないための安全な仕様」です。どの変数がどちらで使えるかを区別できるようになるのが目標です。詳細はコード内コメントを参照してください。


```
// ブラウザ側では NEXT_PUBLIC_ 付きしか参照できない
// Client Component での使用
// components/SomeClient.tsx
// Client Component 化
"use client";

export function SomeClient() {
  // ✅ NEXT_PUBLIC_ 付きはアクセス可能
  // ビルド時に値がJSバンドルに埋め込まれる
  const publicUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
  // "https://xxxxx.supabase.co"
  // ブラウザのコンソールに表示される
  console.log(publicUrl);

  // ❌ NEXT_PUBLIC_ なしは undefined
  // バンドルに含まれない=undefined
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;
  // undefined (安全!)
  // 機密情報が漏れない設計になっている
  console.log(serviceKey);

  return <div>...</div>;
}
```

使い分けの原則:

環境変数の種類	プレフィックス	使用例
機密情報	なし	<code>SUPABASE_SERVICE_ROLE_KEY</code> 、 <code>DATABASE_URL</code>
公開しても安全な情報	<code>NEXT_PUBLIC_</code>	<code>NEXT_PUBLIC_SUPABASE_URL</code> 、 <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>

8.3 Supabase の接続情報の管理

書籍管理アプリでは、Supabase を使います。環境変数の設定例は以下の通りです。

▼ このコードがやること（先に日本語で）：書籍管理アプリで実際に使う Supabase（データベースサービス）の接続情報を `.env.local` に設定する具体例です。URL と匿名キーは公開してよいので `NEXT_PUBLIC_` を付け、全権限を持つサービスロールキーは絶対に公開しないため `NEXT_PUBLIC_` を付けません。この付け分けがそのままセキュリティの境界になります。詳細はコード内コメントを参照してください。

```

# .env.local                                     # 開発機ローカル用の環
境変数ファイル

# Supabase プロジェクト URL (公開OK)             # ブラウザにも露出して
OKな値
# NEXT_PUBLIC_ 付きでClient/Server 両方で使える
NEXT_PUBLIC_SUPABASE_URL="https://abcdefghijklm.supabase.co"

# Supabase 匿名キー (公開OK - Row Level Security で保護される)
# 公開キー。RLS で安全に絞られている前提
NEXT_PUBLIC_SUPABASE_ANON_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

# Supabase サービスロールキー (絶対に公開してはいけない)
# NEXT_PUBLIC_ なし=Server 限定
SUPABASE_SERVICE_ROLE_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

```

▼このコードがやること (先に日本語で) : ブラウザ側 (Client Component) から Supabase に接続するための「クライアント」を作る関数を定義します。公開してよい `NEXT_PUBLIC_` 付きの URL と匿名キーだけを使うのがポイントです。! (non-null assertion) は「この値は undefined ではないと TypeScript に約束する」記号で、設定漏れがあると実行時にエラーになります。詳細はコード内コメントを参照してください。

```

// ブラウザ側で動く Supabase クライアント
// lib/supabase/client.ts
// Client Component 用の Supabase クライアント

// '@supabase/ssr' パッケージの import
import { createBrowserClient } from "@supabase/ssr";

// クライアントを生成する関数 (毎回新しいインスタンスを作る)
export function createClient() {
  return createBrowserClient(
    // `!` は TypeScript の non-null assertion (「undefinedではないと断言」)
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    // 設定漏れがあるとここで undefined になり実行時エラーになる
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}

```

▼ このコードがやること（先に日本語で）：サーバー側（Server Component）から Supabase に接続するクライアントを作ります。クライアント版との違いは、ログイン状態を保つために Cookie を読み書きする仕組み（`cookies()` を使ったアダプタ）を渡している点です。`cookies()` が Promise を返すため関数を `async` にして `await` する必要があります。やや複雑ですが、いまは「サーバー用は Cookie 連携が要る」とだけ掴めば十分です。詳細はコード内コメントを参照してください。

```

// サーバー側で動く Supabase クライアント
// lib/supabase/server.ts
// Server Component 用の Supabase クライアント

// サーバー用ファクトリ関数
import { createServerClient } from "@supabase/ssr";
// Next.js が提供する、リクエストの Cookie 読み書きAPI
import { cookies } from "next/headers";

// async にする必要があるのは cookies() が Promise を返すため
export async function createClient() {
  // 現在のリクエストの Cookie ストアを取得
  const cookieStore = await cookies();

  return createServerClient(
    // URL
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    // 匿名キー
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      // Cookie 入出力のアダプタを渡す
      cookies: {
        // 全 Cookie を返す関数
        getAll() {
          return cookieStore.getAll();
        },
        // Supabase がセッション更新時に呼ぶ関数
        setAll(cookiesToSet) {
          try {
            // 配列を1つずつ処理
            cookiesToSet.forEach(({ name, value, options }) =>
              // Next.js の cookie ストアに書き込み
              cookieStore.set(name, value, options)
            );
          } catch {
            // Server Component からの呼び出し時は
            // Server Component から cookie を書き込もうとすると例外が出るが
            // cookie のセットができないが、問題ない
            // 実害がないため握りつぶしてよい
          }
        },
      },
    },
  );
}

```

▼ コードを1つずつ分解して解説

このサーバー用 Supabase クライアントは、クライアント版と違って「Cookie 連携」が必要なため少し複雑です。「なぜ `async` なのか」「`cookies()` の取得」「Cookie アダプタ」を順に見ていきましょう。

解説1: `cookies()` を `await` するために関数を `async` にする

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();
```

- `createServerClient` は「サーバー側で動く Supabase クライアントを作る」関数です。`@supabase/ssr` パッケージから取り込みます。
- `cookies`（クッキー）は Next.js が提供する「現在のリクエストの Cookie を読み書きする」関数です。`next/headers` から取り込みます。
- `createClient` 関数が `async`なのは、`await cookies()` を呼ぶためです。`cookies()` は Promise を返すので、`await` で中身（Cookieストア）を取り出してから使います。

用語: Cookie（クッキー） ... ブラウザとサーバーの間で受け渡しされる小さなデータ。ログイン状態（セッション）を保持するのに使われ、Supabaseはこれを使ってユーザーを識別します。

解説2: URL・匿名キーと一緒に「Cookie の入出力アダプタ」を渡す

```
return createServerClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
  {
    cookies: {
      getAll() {
        return cookieStore.getAll();
      },
      setAll(cookiesToSet) {
        // ... (Cookie の書き込み処理) ...
      },
    },
  },
);
```

- 第1・第2引数は、接続先のURLと匿名キーです。 `process.env.NEXT_PUBLIC_...` で環境変数から読み込んでいます。末尾の `!` (non-null assertion) は「この値は `undefined` ではないと断言する」記号で、設定漏れがあると実行時にエラーになります。
- 第3引数の `cookies` オブジェクトが、クライアント版にはないサーバー版だけの追加設定です。Supabase に「Cookie を読むときはこれ、書くときはこれを使って」と、読み書きの方法（アダプタ）を渡しています。
- `getAll()` : 今のリクエストの全Cookieを返す関数。
- `setAll(...)` : Supabase がセッションを更新するとき、新しいCookieを書き込む関数。

用語: アダプタ (adapter) ... 「形の違う2つの仕組みをつなぐ橋渡し役」のこと。ここでは「Supabaseが期待するCookie操作」と「Next.jsのCookie API」をつなぐ役割を果たします。

解説3: 書き込み時の例外を `try/catch` で握りつぶす理由

```
setAll(cookiesToSet) {
  try {
    cookiesToSet.forEach(({ name, value, options }) =>
      cookieStore.set(name, value, options)
    );
  } catch {
    // Server Component からの呼び出し時は
    // cookie のセットができないが、問題ない
  }
},
```

- `cookiesToSet.forEach(...)` は、Supabaseが渡してきた複数のCookieを1つずつ取り出して、`cookieStore.set(...)` で書き込んでいます。 `({ name, value, options }) =>` は分割代入で、各Cookieの名前・値・オプションを取り出しています。
- これを `try/catch` で囲んでいるのは、Server Component から Cookie を書き込もうとすると Next.js が例外を出すためです。
- ただし、これは実害がないので、`catch {}` の中を空にして意図的に握りつぶしています（エラーを無視しています）。コメントの通り「セットできないが問題ない」状況です。

用語: 例外を握りつぶす ... `try/catch` でエラーを受け止めつつ、`catch` の中で何もしないこと。「起きても問題ないと分かっているエラー」に限って使う手法で、むやみに多用するのは禁物です。

次章で詳しく解説: Supabase のセットアップと実際の接続方法は、第5章「Supabase 入門」で詳しく扱います。ここでは環境変数の管理方法だけ理解しておいてください。

9. プロジェクト構成のベストプラクティス

9.1 ディレクトリ構成例

bookshelf-app/

app app/ (ルーティング・ページ)

layout.tsx

page.tsx

globals.css

books/

page.tsx

loading.tsx

error.tsx

layout.tsx

actions.ts

new/

page.tsx

[id]/

page.tsx

edit/

page.tsx

components components/ (再利用可能なUI)

ui/ (汎用UIパーツ)

Button.tsx | Input.tsx

books/ (書籍関連)

BookCard.tsx | BookForm.tsx | SearchBar.tsx

layout/ (レイアウト関連)

Header.tsx | Footer.tsx | Sidebar.tsx

lib lib/ (ユーティリティ)

supabase/

client.ts | server.ts

utils.ts

types types/ (型定義)

book.ts | user.ts

public public/ (静的ファイル)

images/

9.2 各ディレクトリの役割

ディレクトリ	役割	含まれるもの
<code>app/</code>	ルーティングとページ	page.tsx, layout.tsx, loading.tsx, error.tsx, Server Actions
<code>components/</code>	再利用可能な UI コンポーネント	ボタン、カード、フォーム、ヘッダー等
<code>lib/</code>	ビジネスロジックとユーティリティ	DB接続、ヘルパー関数、外部API呼び出し
<code>types/</code>	TypeScript 型定義	アプリ全体で使う型 (Book, User 等)
<code>public/</code>	静的ファイル	画像、フォント、favicon 等

9.3 書籍管理アプリのファイル構成

以下は、書籍管理アプリの最終的なファイル構成の全体像です。

▼ このコードがやること（先に日本語で）：アプリ全体で使い回す「書籍データの形」を TypeScript の型として1か所にまとめて定義します。型を決めておくと、`book.title` のように書いたときにエディタが補完してくれたり、入力ミスを事前に警告してくれます。`string | null`（文字列か空のどちらか）や `?`（省略可能）といった、型を柔軟に表す書き方も学べます。詳細はコード内コメントを参照してください。

```

// 型定義の置き場
// types/book.ts
// アプリ全体で使う書籍の型定義

// 名前付き export。`import { Book } from "@types/book"` で使う
export type Book = {
  // データベース上の主キー (UUID または文字列ID)
  id: string;
  // 書籍タイトル
  title: string;
  // 著者名
  author: string;
  // 出版年。スネークケースは Supabase の列名と合わせるため
  published_year: number;
  // 説明文 (null 許容)。`string | null` は型のユニオン
  description: string | null;
  // 表紙画像のURL (任意)
  cover_image_url: string | null;
  // お気に入りフラグ
  is_favorite: boolean;
  // 作成日時 (ISO 8601 文字列)
  created_at: string;
  // 更新日時
  updated_at: string;
};

// フォーム送信時に使う型 (id や日時は不要)
// 新規追加・編集フォーム専用の入力データ型
export type BookFormData = {
  title: string;
  author: string;
  published_year: number;
  // `?` で「省略可能」のオプションプロパティ
  description?: string;
};

```

▼ このコードがやること（先に日本語で）：アプリのあちこちで使う「便利な小さな関数（ユーティリティ）」を1か所にまとめます。日付文字列を「2024年1月15日」の形に整える `formatDate` と、複数のクラス名を空白区切りでつなげる `cn` の2つを定義します。...`classes`（レストパラメータ）で引数を何個でも受け取る書き方や、`filter(Boolean)` で空の値を除く書き方が学べます。詳細はコード内コメントを参照してください。

```

// 汎用ヘルパー置き場
// lib/Utils.ts
// 汎用ユーティリティ関数

/**
 * 日付文字列を日本語のフォーマットに変換する
 * @example formatDate("2024-01-15") → "2024年1月15日" // `@example` は
JSDoc の例示タグ。IDE で吹き出しに表示される
 */
// 引数も戻り値も string と明示
export function formatDate(dateString: string): string {
  // 文字列を Date オブジェクトに変換
  const date = new Date(dateString);
  // ロケール指定で日本語フォーマット
  return date.toLocaleDateString("ja-JP", {
    // 年は数字 (2024)
    year: "numeric",
    // 月は「1月」のような長い表記
    month: "long",
    // 日は数字
    day: "numeric",
  });
}

/**
 * クラス名を結合する (falsy な値は除外)
 * @example cn("base", isActive && "active", "extra") → "base active extra"
 */
export function cn(...classes: (string | false | undefined | null)[]): string {
  // `...classes` はレ
  // ストパラメータ。可変長の引数を配列として受け取る
  // 型は「string か
  // false か undefined か null の配列」
  // `filter(Boolean)` で false/undefined/null/"" を除外
  return classes.filter(Boolean).join(" ");
  // `join(" ")` で半
  // 角スペース区切りに連結
}

```

▼ このコードがやること（先に日本語で）：色や種類を切り替えられる、使い回し可能な汎用ボタン部品を作ります。variant（primary/secondary/danger）や disabledなどを props で受け取り、種類に応じて CSS クラスを組み立てます。variant = "primary" のように引数にデフォルト値を持たせる書き方や、props の形を型（ButtonProps）で定義する書き方が学べます。詳細はコード内コメントを参照してください。

```

// 再利用可能なボタンコンポーネント
// components/ui/Button.tsx
// 汎用ボタンコンポーネント

// props の型定義
type ButtonProps = {
  // ボタン内の表示内容 (テキストやアイコン)
  children: React.ReactNode;
  // ボタンの種類。`?` で省略可能、`|` でユニオン型
  variant?: "primary" | "secondary" | "danger";
  // 無効化フラグ
  disabled?: boolean;
  // HTML の button 要素の type
  type?: "button" | "submit";
  // クリック時のハンドラ (任意)
  onClick?: () => void;
};

// 分割代入で props を取り出す
export function Button({
  children,
  // デフォルト値の指定。指定がなければ "primary"
  variant = "primary",
  disabled = false,
  // デフォルトは "button" (submit を防ぐ)
  type = "button",
  onClick,
}: ButtonProps) {
  // 全ボタン共通のクラス
  const baseClass = "btn";
  // 種類別のクラス (例: "btn-primary")
  const variantClass = `btn-${variant}`;

  return (
    <button
      // HTML 属性に渡す
      type={type}
      // テンプレートリテラルで2つのクラスを連結
      className={`${baseClass} ${variantClass}`}
      disabled={disabled}
      onClick={onClick}
    >
      // ボタンの中身を表示
      {children}
    </button>
  );
}

```

パスエイリアスの設定

`@/` というパスエイリアスを使うと、ディレクトリの深さに関係なく、プロジェクトルートからの絶対パスで import できます。Next.js のプロジェクト作成時に自動で設定されます。

```
// ❌ 相対パスでの import (ディレクトリが深いと地獄)
// `../` の数を間違えると即エラー
import { BookCard } from "../../components/books/BookCard";
// ファイルを移動するたびに書き直しが必要
import { formatDate } from "../../lib/utils";

// ✅ パスエイリアスでの import (常にわかりやすい)
// `@/` がプロジェクトルートを指す
import { BookCard } from "@components/books/BookCard";
// どんなに深い階層からでも同じ書き方でOK
import { formatDate } from "@lib/utils";
```

この設定は `tsconfig.json` (TypeScript のコンパイル設定ファイル) に記述されています:

```
{
  // TypeScript コンパイラのオプション
  "compilerOptions": {
    // パスエイリアスのマッピング
    "paths": {
      // `@/` を「プロジェクトルートからの相対パス」に展開
      "@/*": ["./*"]
    }
  }
}
```

発展: アプリでは使っていない重要なNext.js機能

ここまでで、このチュートリアル of 書籍管理アプリを作るのに必要な機能はひとつおりました。ですが、Next.js には**実際の本番アプリでよく使われるのに、このアプリでは出番がなかった重要な機能**がまだあります。この「発展」セクションでは、それらを独立した最小サンプルで1つずつ紹介します。「いつ使うか」「なぜ便利か」を中心に、初心者でも読めるように解説していきます。

読み方のヒント: ここで紹介する機能は、いきなり全部覚える必要はありません。「こういう機能がある」という存在だけ頭の片隅に置いておき、必要になったときに戻ってくれば十分です。すべて App Router (`app/` フォルダ) 前提のコードです。

generateMetadata と generateStaticParams（ページごとのタイトル設定とビルド時のページ生成）

▼ このコードがやること（先に日本語で）：書籍ごとに違う詳細ページ（`/books/1` , `/books/2` ...）で、ブラウザのタブやSNS共有に出るタイトルを「その書籍のタイトル」に変える仕組みと、どの書籍ページを事前に用意しておくかを `Next.js` に教える仕組みの2つを作ります。前者は検索結果やSNSで「この記事は何の話か」を正しく見せるため（SEO対策）に重要で、後者はアクセス前にページを作っておくことで表示を最速にするためのものです。詳細はコード内コメントを参照してください。

```

// 動的ルート。/books/1, /books/2, ... に対応
// app/books/[id]/page.tsx
// [id] に入った値ごとに別ページになる
// URL: /books/1, /books/2, ...

// タイトルなどの型を読み込む（実行時には消える型だけのimport）
import type { Metadata } from "next";

// -----
// (A) 書籍データを取得する仮の関数（本来はDBやAPIから取る）
// -----
// id を受け取って書籍1冊分のデータを返す
async function getBook(id: string) {
  // 実際にはここで fetch やデータベース問い合わせを書く
  return { id, title: `サンプル書籍 ${id}`, author: "山田太郎" };
}

// -----
// (B) generateMetadata: ページごとに <title> などを動的に作る
// -----
// この名前で export すると、Next.js がページ表示前に呼んで <head> を組み立てる。
// 予約名の関数。async にできる
export async function generateMetadata({
  // page.tsx と同じく動的セグメントの値が入る
  params,
}: {
  // Next.js 15+ では params は Promise
  params: Promise<{ id: string }>;
// 戻り値は Metadata 型 (title など)
}): Promise<Metadata> {
  // await で id を取り出す (文字列)
  const { id } = await params;
  // その id の書籍データを取得
  const book = await getBook(id);
  return {
    // ブラウザタブ・検索結果に出るタイトル
    title: `${book.title} | 書籍管理アプリ`,
    // SNS共有や検索結果の説明文
    description: `${book.author} 著「${book.title}」の詳細ページ`,
  };
}

// -----
// (C) generateStaticParams: ビルド時に作っておくページの一覧
// -----
// この名前で export すると、Next.js は「ここで返した id のページ」を
// ビルド時にあらかじめHTML化しておく（＝アクセスが最速になる）。
// 予約名の関数
export async function generateStaticParams() {

```

```

// 事前生成したい書籍IDの一覧（本来はDBから取得）
const ids = ["1", "2", "3"];
// [{ id: "1" }, { id: "2" }, { id: "3" }] を返す
return ids.map((id) => ({ id }));
}

// -----
// (D) ページ本体
// -----

export default async function BookDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  // ページ本体でも params から id を取り出す
  const { id } = await params;
  // 書籍データを取得
  const book = await getBook(id);
  return (
    <div>
      { /* 画面に出る見出し */ }
      <h1>{book.title}</h1>
      <p>著者: {book.author}</p>
    </div>
  );
}

```

画面にはこう表示される: `/books/1` を開くと本文に「サンプル書籍 1」と表示され、さらにブラウザのタブには「サンプル書籍 1 | 書籍管理アプリ」と出ます。`/books/2` なら本文もタブも「2」に変わります。`generateStaticParams` で `"1"~"3"` を返しているので、この3ページはビルド時に作られ、表示が最速になります。

▼ コードを1つずつ分解して解説

このコードには「`generateMetadata` でタイトルを動的に作る」「`generateStaticParams` で事前生成リストを返す」という2つの新しい仕組みが入っています。順番に見ていきましょう。

解説1: `generateMetadata` でページごとに違うタイトルを付ける

```
export async function generateMetadata({
  params,
}: {
  params: Promise<{ id: string }>;
}): Promise<Metadata> {
  const { id } = await params;
  const book = await getBook(id);
  return {
    title: `${book.title} | 書籍管理アプリ`,
    description: `${book.author} 著「${book.title}」の詳細ページ`,
  };
}
```

- これまで（`layout.tsx`）では `export const metadata = { ... }` という固定値でタイトルを付けていました。でも書籍詳細ページは「URLごとに中身が違う」ので、タイトルも本ごとに変えたいですね。そこで使うのが `generateMetadata`（ジェネレート・メタデータ）という決まった名前の関数です。
- この関数は `page.tsx` と同じく `params`（URLの `[id]` の値）を受け取れます。`await params` で `id` を取り出し、その本のデータを取得して、そのデータを使ったタイトルを `return` で返します。
- 返したタイトルや説明文を、Next.js が自動で `<head>` の中に入れてくれます。検索エンジンやSNS（X・Facebook など）はこの `<head>` を読んで「このページは何か」を判断するので、SEO（検索エンジン対策）やSNS共有の見栄えに直結します。

いつ使う？ なぜ便利？ ブログ記事・商品ページ・プロフィールページなど「URLごとに内容が変わるページ」では、`generateMetadata` でタイトルや説明文を中身に合わせて変えるのが基本です。これをしないと全ページが同じタイトルになり、検索結果で見分けがつかず、クリックされにくくなります。

用語: SEO（エスイーオー） ... Search Engine Optimization の略。Google などの検索結果で自分のページが上位に・分かりやすく表示されるよう整えること。タイトルと説明文はその一番の基本です。

解説2: `generateStaticParams` で「事前に作っておくページ」を伝える

```
export async function generateStaticParams() {
  const ids = ["1", "2", "3"];
  return ids.map((id) => ({ id }));
}
```

- 動的ルート（`[id]`）は、本来「アクセスが来たそのとき」にページを組み立てます。でも `generateStaticParams`（ジェネレート・スタティック・パラムス）という名前の関数を置くと、ビルド時（公開前の準備段階）にまとめてHTMLを作っておけます。

- 返す形は `[{ id: "1" }, { id: "2" }, { id: "3" }]` のような「`params` のオブジェクトの配列」です。
`ids.map((id) => ({ id }))` はこの形を作っているだけです（`{ id }` は `{ id: id }` の省略形）。
- こうしておくと、`/books/1` ～ `/books/3` は完成済みHTMLとして返るので、**表示が最速**になりサーバー負荷も減ります（第1章で学んだSSGの仕組みです）。

いつ使う？ なぜ便利？ 「ページ数があらかじめ分かっている、頻繁には変わらない」もの（公開済みのブログ記事一覧、商品カタログなど）に向いています。アクセスを待たずに作っておくぶん、ユーザーは待たされません。逆に「数が膨大」「毎回内容が変わる」ページには向きません。

ISR（一定時間ごとにページを自動で作り直す）

▼ このコードがやること（先に日本語で）：「ページを事前に作っておく（最速・SSG）」の良さを保ちつつ、一定時間ごとに自動で最新版に作り直す設定です。たった1行 `export const revalidate = 60;` を書くだけで「このページは60秒たった次のアクセスのタイミングで作り直してね」とNext.jsに伝えられます。常に最新でなくてもいいけれど、たまには更新したいページにぴったりです。詳細はコード内コメントを参照してください。

```
// ニュース一覧ページの例
// app/news/page.tsx
// URL: /news

// -----
// この1行が ISR の本体。
// 「このページは最大60秒は使い回し、60秒を過ぎたら次のアクセス時に裏で作り直す」
// という意味。数字（秒数）は自由に変えられる。
// -----
// 60秒ごとに作り直す（予約名の export）
export const revalidate = 60;

// ニュースを取得する仮の関数
async function getNews() {
  // 実際にはここで外部APIやDBから最新ニュースを取る
  // 「ページを作った時刻」を確認用に作る
  const now = new Date().toLocaleTimeString("ja-JP");
  return [{ title: "新刊が入荷しました", builtAt: now }];
}

// ページ本体 (Server Component)
export default async function NewsPage() {
  // データ取得
  const news = await getNews();

  return (
    <div>
      <h1>お知らせ</h1>
      <ul>
        // 取得した記事を一覧表示
        {news.map((item, i) => (
          <li key={i}>{item.title} (生成時刻: {item.builtAt}) </li>
        ))}
      </ul>
    </div>
  );
}
```

画面にはこう表示される:「お知らせ」見出しの下にニュース項目が並びます。「生成時刻」の部分に注目すると、ページを何度リロードしても60秒間は同じ時刻のまま（＝作り置きを使い回している）で、60秒を過ぎてからアクセスすると新しい時刻に更新されます。これが「一定時間ごとに作り直す」動きです。

▼ コードを1つずつ分解して解説

ISR は新しい関数を覚える必要はなく、「`revalidate`」という決まった名前の数字を export するだけというのがポイントです。

解説1: `export const revalidate = 60` の1行がすべて

```
export const revalidate = 60;
```

- ISR（アイエスアール）は「一度作ったページを使い回しつつ、決めた秒数ごとに裏でこっそり作り直す」仕組みです。これを有効にするのが、この `revalidate`（リバリデート＝再検証）という**決まった名前**の値を export する1行です。
- 数字は「何秒間そのページを使い回すか」を表します。`60` なら「60秒は作り置きを返す。60秒を過ぎた後の最初のアクセスをきっかけに、裏で新しいページを作って差し替える」という動きになります。
- 重要なのは、**作り直しはユーザーを待たせない**点です。秒数が過ぎても、そのアクセスにはまず古いページをすぐ返し、作り直しはその裏で進みます。次のアクセスから新しいページに切り替わります。

いつ使う？ なぜ便利？「最新であってほしいけれど、1秒単位の鮮度までは要らない」ページ（ニュース、ランキング、在庫数の目安など）に最適です。アクセスのたびに毎回作り直す SSR より速く・安く、完全に作り置き SSG より新しい、という「いいとこ取り」ができます。

用語: ISR（Incremental Static Regeneration） ... 「少しずつ（インクリメンタルに）静的ページを再生成する」方式。第1章の表で出てきた SSG と SSR の中間にあたる方法だと考えると分かりやすいです。

Route Handlers（自分のアプリの中にAPIを作る）

▼ このコードがやること（先に日本語で）：Next.js のアプリの中に、他のプログラムがデータをやり取りするための「窓口（API）」を作ります。`app/api/.../route.ts` という決まった場所にファイルを置くと、「データを取りに来たとき（GET）」と「データを送ってきたとき（POST）」の処理を書けます。スマホアプリや外部サービスと連携したり、ブラウザ側のJavaScriptからデータだけを取りに行きたいときに使います。詳細はコード内コメントを参照してください。

```

// 予約名 route.ts。URLは /api/books になる
// app/api/books/route.ts
// page.tsx ではなく route.ts を置くのがポイント
// URL: /api/books

// JSONなどを返すための Next.js 標準ヘルパー
import { NextResponse } from "next/server";

// 仮の書籍データ（本来はDBに保存する）
const books = [
  { id: 1, title: "吾輩は猫である" },
  { id: 2, title: "坊っちゃん" },
];

// -----
// (1) GET: データを「取りに来た」ときの処理
// -----
// 関数名を GET にすると、HTTPのGETリクエストに対応する。
export async function GET() {
  // NextResponse.json(...) でデータをJSON形式にして返す
  // /api/books に GET すると books 一覧が返る
  return NextResponse.json(books);
}

// -----
// (2) POST: データを「送ってきた」ときの処理
// -----
// 関数名を POST にすると、HTTPのPOSTリクエストに対応する。
// 送られてきた内容は引数 request に入る
export async function POST(request: Request) {
  // 送信された本文 (JSON) を取り出す
  const body = await request.json();
  // 新しい書籍を組み立てる
  const newBook = { id: books.length + 1, title: body.title };
  // 一覧に追加（本来はDBに保存）
  books.push(newBook);
  // 201 は「新しく作成しました」を表すHTTPステータスコード
  // 作成した書籍を返す
  return NextResponse.json(newBook, { status: 201 });
}

```

画面にはこう表示される: これは画面（ページ）ではなくデータの窓口です。ブラウザで

`http://localhost:3000/api/books` を開くと、`[{"id":1,"title":"吾輩は猫である"}, {"id":2,"title":"坊っちゃん"}]` のようなJSONデータがそのまま表示されます。POST のほうは、ブラウザのJavaScriptや外部ツールから「タイトルを送る」と新しい書籍が追加され、追加後のデータが返ってきます。

▼ コードを1つずつ分解して解説

このファイルには「`route.ts` という置き場所のルール」「`GET` / `POST` という関数名のルール」「`NextResponse.json` で返すルール」の3点が詰まっています。

解説1: `route.ts` を置くと、その場所が「APIの窓口」になる

```
// app/api/books/route.ts
// URL: /api/books
```

- これまで作ってきたページは `page.tsx` という名前でした。それに対し、`route.ts`（ルート・ティーエス）という名前のファイルを置くと、そこは「画面を出す場所」ではなく「データをやり取りする窓口（API）」になります。
- 場所はフォルダ構成のままURLになります。`app/api/books/route.ts` なら URL は `/api/books` です（`api` というフォルダ名は必須ではありませんが、APIだと分かるよう慣習的にこう置きます）。
- このように Next.js だけで API も作れるので、別途サーバー（Express など）を立てなくてよいのが大きな利点です。

用語: API（エーピーアイ） ... プログラム同士がデータをやり取りするための「窓口・受付」のこと。人が見る画面（HTML）ではなく、機械が読むためのデータ（多くはJSON）を返します。

用語: JSON（ジェイソン） ... `{"title": "坊っちゃん"}` のような「キーと値」でデータを表す、プログラム間によく使われる共通フォーマット。

解説2: 関数名 `GET` / `POST` がそのまま「リクエストの種類」に対応する

```
export async function GET() {
  return NextResponse.json(books);
}

export async function POST(request: Request) {
  const body = await request.json();
  const newBook = { id: books.length + 1, title: body.title };
  books.push(newBook);
  return NextResponse.json(newBook, { status: 201 });
}
```

- Webのデータのやり取りには「種類」があります。代表的なのが `GET`（取りに行く）と `POST`（送り込む）です。`route.ts` では、関数の名前をそのまま `GET` や `POST` にすることで、それぞれの種類に対応する処理を書けます。
- `GET` は引数なしでもよく、`NextResponse.json(books)` のように返したいデータをJSONにして返すだけです。

- `POST` では、送られてきた内容が引数 `request` に入っています。`await request.json()` で送信された本文を取り出し、新しいデータを作って返しています。`{ status: 201 }` は「新しく作成しました」を意味する番号（HTTPステータスコード）です。

いつ使う？ なぜ便利？ スマホアプリや他のWebサービスに自分のデータを提供したいとき、ブラウザ側のJavaScriptから一部のデータだけを取りに行きたいとき、Webhook（外部サービスからの通知）を受け取りたいときなどに使います。なお、このアプリのようにフォーム送信をサーバーで処理するだけなら、第7章の **Server Actions** のほうが手軽な場合も多いです。用途で使い分けます。

Middleware（アクセス前に割り込んで認証チェックする）

▼ このコードがやること（先に日本語で）：ページが表示される前に「割り込み」を入れて、「この人はログイン済みか？」をチェックします。もしログインしていなければ、目的のページを見せずに**ログイン画面へ自動的に送り返し（リダイレクト）**ます。会員専用ページや管理画面を「ログインしていない人に見せない」ための、見張り番のような仕組みです。詳細はコード内コメントを参照してください。

```
// 置き場所が重要。プロジェクトの一番上 (app/ と同じ階層) に置く
// middleware.ts
// ※ app/ の中ではなく、ルート直下に1つだけ置く

// リダイレクトや「通過OK」を返すためのヘルパー
import { NextResponse } from "next/server";
// 届いたリクエストの型 (型だけのimport)
import type { NextRequest } from "next/server";

// -----
// middleware: ページに届く前に必ず通る「関所」
// 予約名 middleware で export する必要がある。
// -----
// request にアクセス内容 (URLやCookieなど) が入る
export function middleware(request: NextRequest) {
  // Cookie から「ログインの証 (トークン)」を読む
  // ログイン時に保存しておいた印を確認
  const token = request.cookies.get("token");

  // ログインの証が無い=未ログインなら、ログイン画面へ送り返す
  if (!token) {
    // リダイレクト先のURLを組み立てる
    const loginUrl = new URL("/login", request.url);
    // ログイン画面へ強制移動させる
    return NextResponse.redirect(loginUrl);
  }

  // ログイン済みなら、そのまま目的のページへ通す
  // 「通過OK」を返す (何も止めない)
  return NextResponse.next();
}

// -----
// config.matcher: この見張りを「どのURLに効かせるか」
// -----
export const config = {
  // /mypage と、その下のすべてのページ (/mypage/settings など) にだけ適用する
  matcher: ["/mypage/:path*"],
};
```

画面にはこう表示される: ログインしていない状態で `/mypage` を開こうとすると、`/mypage` の中身は一切見えないまま、一瞬で `/login` (ログイン画面) に飛ばされます。逆にログイン済み (Cookie にトークンがある) なら、何ごともなく `/mypage` がそのまま表示されます。

▼ コードを1つずつ分解して解説

Middleware には「`middleware.ts` の置き場所」「`middleware` という関数名」「`config.matcher` で対象を絞る」という3つの約束があります。

解説1: `middleware.ts` は「ページの手前にある関所」

```
// middleware.ts (プロジェクトのルート直下に置く)
export function middleware(request: NextRequest) {
  const token = request.cookies.get("token");
  if (!token) {
    const loginUrl = new URL("/login", request.url);
    return NextResponse.redirect(loginUrl);
  }
  return NextResponse.next();
}
```

- Middleware（ミドルウェア）は、ユーザーがページにたどり着く直前に必ず通る「関所」です。`middleware.ts` という名前のファイルをプロジェクトの一番上（`app/` フォルダと同じ階層）に1つだけ置きます。
- 中では `request`（届いたアクセス情報）を調べられます。ここでは `request.cookies.get("token")` で「ログインの証」を探しています。
- 証が無ければ `NextResponse.redirect(...)` でログイン画面へ送り返し、有れば `NextResponse.next()` でそのまま通します。`next()` は「邪魔せず通過させる」という意味です。

用語: リダイレクト ... アクセスしてきた人を、別のURLへ自動的に送り直すこと。「この扉は今開けられないので、あちらの受付へどうぞ」と案内するイメージです。

用語: Cookie（クッキー）／トークン ... Cookieはブラウザに小さなデータを保存しておく仕組み。ログインすると「あなたはログイン済みです」という印（トークン）をCookieに入れておき、次回以降そのトークンの有無でログイン状態を判断します。

解説2: `config.matcher` で「どのページに効かせるか」を絞る

```
export const config = {
  matcher: ["/mypage/:path*"],
};
```

- Middleware は放っておくとすべてのアクセスに割り込んでしまい、画像やトップページにまで関所がかかって無駄が出ます。そこで `config` の `matcher`（マッチャー＝対象を選ぶもの）で、「効かせたいURLだけ」を指定します。
- `"/mypage/:path*"` は「`/mypage` とその下のすべてのページ」という意味です。これで会員専用ページにだけログインチェックがかかり、トップページやログイン画面はチェックなしで開けます。

いつ使う？ なぜ便利？「ログインしていない人に会員ページ・管理画面を見せたくない」というのが代表例です。各ページに毎回チェックを書くより、入口でまとめて見張るほうが書き漏れがなく安全です。ほかにも、言語ごとのページ振り分けや、地域によるアクセス制限などにも使われます。

next/image の詳細（画像を自動で最適化する）

▼ このコードがやること（先に日本語で）：普通の `<img>` タグの代わりに Next.js の `<Image>` を使い、画像を自動で軽く・速く・きれいに表示します。`fill`（親要素いっぱいに広げる）、`priority`（最優先で読み込む）、`sizes`（画面幅ごとの表示サイズ指定）、`placeholder`（読み込み中にぼかしを出す）といった便利なオプションの使い方を1つにまとめた例です。画像が多いページの表示速度を大きく改善できます。詳細はコード内コメントを参照してください。

```
// どのページでも使える例として page.tsx に書く
// app/page.tsx

// Next.js 専用の画像コンポーネントを読み込む
import Image from "next/image";

export default function HomePage() {
  return (
    <div>
      {/* —— (1) サイズが分かっている画像: width / height を指定 —— */}
      <Image
        // public フォルダ内の画像 (/ から始める)
        src="/cover.jpg"
        // 画像の説明文 (読み上げ・SEO・表示失敗時に必須)
        alt="本の表紙"
        // 元画像の幅 (ガタつき防止のため必須)
        width={300}
        // 元画像の高さ
        height={450}
        // このページで「最初に見える重要画像」を最優先で読み込む
        priority
      />

      {/* —— (2) サイズが可変な画像: fill で親要素いっぱいに広げる —— */}
      <div style={{ position: "relative", width: "100%", height: "300px" }}>
        {/* fill を使うときは、親要素に position: relative とサイズが必要 */}
        <Image
          src="/banner.jpg"
          alt="キャンペーンバナー"
          // width/height の代わりに「親要素を埋める」
          fill
          // 画面幅ごとに必要な画像サイズを指定 (後述)
          sizes="(max-width: 768px) 100vw, 50vw"
          // 読み込み中はぼかし画像を表示
          placeholder="blur"
          // ぼかしに使う極小画像データ
          blurDataURL="data:image/png;base64,iVBORw0K..."
          // はみ出した部分を切り取って枠に合わせる
          style={{ objectFit: "cover" }}
        />
      </div>
    </div>
  );
}
```

画面にはこう表示される: 見た目は普通の画像と同じですが、裏側では Next.js が「画面サイズに合った小さめの画像」「対応ブラウザには軽い形式 (WebP など)」を自動で配って、読み込みを速くしています。 `placeholder="blur"` を

付けた画像は、読み込みが終わるまでぼんやりした状態で表示され、完了するとくっきり切り替わるので、待ち時間が気になりにくなります。

▼ コードを1つずつ分解して解説

`<Image>` には便利なオプションがいくつかあります。よく使う `width/height` ・ `priority` ・ `fill` ・ `sizes` ・ `placeholder` を順に見ていきましょう。

解説1: `width` / `height` と `priority`

```
<Image
  src="/cover.jpg"
  alt="本の表紙"
  width={300}
  height={450}
  priority
/>
```

- `next/image` の `<Image>` は、普通の `<img>` の代わりに使うと `Next.js` が画像を自動で最適化してくれる部品です。`src` は画像の場所、`alt` は画像の説明文（読み上げソフトや、表示に失敗したときに使われるので必ず書きます）。
- `width` と `height` は元画像の縦横の大きさです。これを指定しておく、画像が読み込まれる前から「ここに300×450の場所を空けておく」ことができ、読み込み後に他の文字や要素がガタツとずれるのを防げます。
- `priority`（プライオリティ＝優先）を付けると、その画像を最優先で読み込みます。ページを開いてすぐ目に入る大きな画像（記事のトップ画像など）に付けると、表示が体感で速くなります。逆に、下のほうにあって最初は見えない画像には付けません（付けすぎると逆効果）。

用語: alt（オルト）テキスト ... 画像の内容を文字で説明したもの。目の不自由な人向けの読み上げや、画像が表示できなかったときの代わりの表示に使われ、SEOにも効きます。

解説2: `fill` で「親要素いっぱい」に広げる

```
<div style={{ position: "relative", width: "100%", height: "300px" }}>
  <Image
    src="/banner.jpg"
    alt="キャンペーンバナー"
    fill
    style={{ objectFit: "cover" }}
  />
</div>
```

- 画像の正確なサイズが分からない・画面幅に合わせて伸び縮みさせたい、という場合は `width/height` の代わりに `fill`（フィル＝満たす）を使います。これは「親要素の大きさにいっぱい画像を広げる」オプションです。
- `fill` を使うときの約束として、親要素に `position: relative`（位置の基準にする指定）とサイズ（高さなど）が必要です。これがないと画像が正しく広がりません。
- `objectFit: "cover"` は「枠からはみ出す部分は切り取って、枠をきれいに埋める」指定です。これで縦横比が違う画像でも崩れずに表示できます。

解説3: `sizes` で「画面幅ごとに最適な画像」を配る

```
sizes="(max-width: 768px) 100vw, 50vw"
```

- `sizes`（サイズ）は、「画面の幅ごとに、この画像が実際どれくらいの大きさで表示されるか」を Next.js に教えるオプションです。これを伝えと、Next.js はスマホには小さい画像、PCには大きい画像といったように、ちょうどよいサイズの画像を選んで配れます。
- `(max-width: 768px) 100vw, 50vw` は「画面幅が768px以下（スマホ）なら画面幅いっぱい（100vw）、それ以外（PC）なら画面の半分（50vw）の大きさで表示する」という意味です。`vw` は「画面幅に対する割合」を表す単位です。
- `fill` を使うときは、適切な画像を選ぶために `sizes` を一緒に指定するのが推奨です。スマホで巨大な画像をわざわざダウンロードせずに済むので、通信量の節約と高速化につながります。

解説4: `placeholder="blur"` で読み込み中にぼかしを出す

```
placeholder="blur"
blurDataURL="data:image/png;base64,iVBORw0K..."
```

- `placeholder="blur"`（プレースホルダー＝仮置き、blur＝ぼかし）を付けると、画像の読み込みが終わるまでの間、ぼんやりした仮の画像を表示します。完了すると本物にスッと切り替わります。
- 「真っ白な空白」が一瞬出るより、ぼかしでも何か見えているほうが、ユーザーは「ちゃんと読み込まれている」と感じて待ち時間が気になりにくなります。
- `blurDataURL` は、そのぼかしに使うごく小さな画像データです（ここでは省略して `...` と書いています）。`public` フォルダ内の画像をインポートして使う場合は、Next.js がこのぼかし用データを自動生成してくれるので自分で用意しなくて済みます。

いつ使う？ なぜ便利？（next/image 全体） 画像はWebページの表示が遅くなる最大の原因の1つです。

`<img>` をそのまま使うと「大きすぎる画像をそのまま配ってしまう」「読み込み中にレイアウトがずれる」といった問題が起きがちですが、`next/image` を使えばサイズ調整・形式変換・遅延読み込みなどを Next.js が自動でやってくれます。画像を多く使うページほど効果が大いなので、Next.js では原則 `<Image>` を使うのがおすすめです。

10. よくあるエラーと対処法

エラー1: "useState" は Server Component で使用できない

Error: useState only works in Client Components. Add the "use client" directive at the top of the file to use it.

原因: Server Component (デフォルト) で `useState` や `useEffect` を使おうとしている。

対処法: ファイルの先頭に `"use client"` を追加する。

```
// "use client" がないので Server Component 扱い
// ❌ エラーになる
// React のフックを取り込み
import { useState } from "react";

export default function Counter() {
  // ← ここでエラー
  // Server Component では useState を呼べない
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
  // onClick も
  Server Component では使えない
}

// ✅ 修正版
// ← これを追加
// ファイル先頭のディレクティブで Client Component 化
"use client";

import { useState } from "react";

export default function Counter() {
  // OK: Client では useState が使える
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

エラー2: "async/await" は Client Component で使用できない

```
Error: async/await is not yet supported in Client Components, only Server Components.
```

原因: `"use client"` を付けたコンポーネントで `async` 関数としてエクスポートしている。

対処法: データ取得を Server Component に移動するか、`useEffect` を使う。

```

// ❌ エラーになる
// Client Component なのに
"use client";

// 関数本体に async を付けている
export default async function BooksPage() {
  // ← async は Client Component で不可
  const books = await fetch("/api/books");

  // App Router の
  Client では関数を async にできない
  return <div>...</div>;
}

// ✅ 修正版 1: Server Component にする ("use client" を削除)
// "use client" がなければ Server Component
export default async function BooksPage() {
  // Server なら async でも OK
  const books = await fetch("https://api.example.com/books");
  return <div>...</div>;
}

// ✅ 修正版 2: Client Component のまま useEffect を使う
"use client";

import { useState, useEffect } from "react";

// 関数本体は同期のままにする
export default function BooksPage() {
  // 結果を state に保持
  const [books, setBooks] = useState([]);

  // マウント後にデータ取得
  useEffect(() => {
    fetch("/api/books")
      // Promise チェーン
      .then((res) => res.json())
      // 取れた配列を state に格納
      .then(setBooks);
    // 依存配列が空なので初回マウント時に1度だけ実行
  }, []);

  return <div>...</div>;
}

```


エラー3: "next/router" と "next/navigation" の混同

```
Error: NextRouter was not mounted.
```

原因: App Router で `next/router` を import している。App Router では `next/navigation` を使う。

```
// ❌ App Router では使えない
// 旧 Pages Router 用の useRouter
import { useRouter } from "next/router";

// ✅ App Router ではこちらを使う
// 新しい App Router 用。push/replace/back/refresh などを持つ
import { useRouter } from "next/navigation";
```

エラー4: metadata と "use client" の共存

```
Error: You are attempting to export "metadata" from a component marked with
"use client", which is unsupported.
```

原因: `"use client"` のファイルで `metadata` をエクスポートしている。メタデータは Server Component でのみエクスポート可能。

```
// ❌ エラーになる
// Client Component なのに
"use client";

// ← Client Component では不可
export const metadata = { title: "書籍一覧" };

// metadata は
// Server Component の機能なので Client では宣言できない

// ✅ 修正版: metadata は Server Component (page.tsx や layout.tsx) に置く
// app/books/page.tsx (Server Component)
// この行でブラウザタブのタイトルが「書籍一覧」になる
export const metadata = { title: "書籍一覧" };

export default function BooksPage() {
  return <div>...</div>;
}
```

補足: generateMetadata : 動的にメタデータを決めたいとき（例: 書籍IDから書籍タイトルを取って `<title>` に入れる）は、`generateMetadata` という名前で `async` 関数を `export` します。引数として `params` を受け取れます。同様に「ビルド時に動的セグメントの値を事前列挙したい」場合は `generateStaticParams` を `export` します。

エラー5: 環境変数が `undefined` になる

原因: Client Component で `NEXT_PUBLIC_` プレフィックスのない環境変数を使っている。

```
// ❌ Client Component では undefined になる
"use client";

// undefined
// NEXT_PUBLIC_ なしの環境変数はブラウザバンドルに含まれない
const apiKey = process.env.API_SECRET_KEY;

// ✅ NEXT_PUBLIC_ を付ける（ただし機密情報には使わない！）
// 値が取得できる
// 公開してよい値だけに NEXT_PUBLIC_ を付ける
const publicUrl = process.env.NEXT_PUBLIC_API_URL;
```

注意: 機密情報に `NEXT_PUBLIC_` を付けてはいけません。Server Action や API Route 経由でアクセスしてください。

エラー6: "Text content does not match server-rendered HTML" (Hydration エラー)

```
Warning: Text content did not match. Server: "2024年3月15日" Client: "3/15/2024"
```

原因: サーバーとクライアントで異なる出力が生成されている。日時のフォーマットやランダム値でよく発生する。

```
// ❌ サーバーとクライアントで結果が異なる可能性がある
export default function DateDisplay() {
  // サーバーとブラウザで実行時刻が違えば表示も違う
  return <p>{new Date().toLocaleString()}</p>;
}

// → Hydration エラ
-になる
}

// ✅ 修正案 1: suppressHydrationWarning を使う
export default function DateDisplay() {
  return <p suppressHydrationWarning>{new Date().toLocaleString()}</p>;
}

// React の
Hydration 不一致警告を抑制する属性
// 中身が動的でもよい
場合の応急処置
}

// ✅ 修正案 2: Client Component にして useEffect で設定
// Client 化
"use client";

import { useState, useEffect } from "react";

export default function DateDisplay() {
  // 初期値は空文字 (サーバーでも同じ値)
  const [dateStr, setDateStr] = useState("");

  // ハイドレーション後=ブラウザ側でのみ走る
  useEffect(() => {
    // ここで日付を設定 → 再描画
    setDateStr(new Date().toLocaleString("ja-JP"));
    // 初回1回だけ
  }, []);

  // サーバー描画時は空、ブラウザで時刻に置き換わる
  return <p>{dateStr}</p>;
}
```

エラー7: fetch でのキャッシュに関する警告

Warning: fetch for "https://..." on "/" used "no-store" and should not be called inside a layout or template.

対処法: `layout.tsx` 内では `cache: "no-store"` を避け、`page.tsx` で使う。

エラーの調査方法（一般的なアドバイス）

1. **ターミナルのエラーメッセージを読む**: Next.js のエラーメッセージは非常に詳しく、多くの場合、解決策まで提示してくれます。
 2. **ブラウザのコンソールを確認する**: 開発者ツール（F12）の Console タブにもエラーが表示されます。
 3. **Server Component と Client Component の境界を確認する**: 多くのエラーは、この2つの境界での誤りが原因です。
 4. **Next.js の公式ドキュメントを参照する**: <https://nextjs.org/docs> には詳細な説明とトラブルシューティングがあります。
-

まとめ

この章では Next.js の基礎を広く学びました。

トピック	学んだこと
Next.js とは	React のフルスタックフレームワーク。SSR/SSG/CSR に対応
App Router	ファイルベースルーティング。page.tsx, layout.tsx 等の特殊ファイル
Server/Client Components	デフォルトは Server。インタラクティブ性が必要なら "use client"
レイアウト	ルートレイアウトとネストされたレイアウトで共通UIを効率的に管理
ナビゲーション	Link コンポーネント（宣言的）と useRouter フック（命令的）
データフェッチ	Server Component なら async/await で直接。Client は useEffect
Server Actions	"use server" でサーバー処理を直接呼び出し。フォーム処理に最適
環境変数	NEXT_PUBLIC_ の有無でアクセス範囲が変わる
プロジェクト構成	app/, components/, lib/, types/ の4層構造

次の章では: Supabase のセットアップを行い、実際のデータベースと接続します。書籍管理アプリのバックエンドを構築し、この章で学んだ Server Components や Server Actions を使ってデータの読み書きを実装していきます。

確認問題

以下の問いに答えて、理解度を確認してみましょう。

1. **Server Component と Client Component の違い** を3つ挙げてください。

2. 以下のコンポーネントは Server / Client のどちらにすべきですか？ - データベースから書籍一覧を取得して表示するコンポーネント - 検索バー（ユーザーの入力をリアルタイムに反映） - 書籍カードの静的な表示（クリックイベントなし）
3. `NEXT_PUBLIC_API_KEY` と `API_SECRET_KEY` はそれぞれどこで使用できますか？
4. `layout.tsx` と `page.tsx` の違いは何ですか？
5. Server Actions を使う利点を2つ挙げてください。

▶ 解答を見る